

Data Science for Cloud Security

Machine learning for analysing cyber threats, and predictive models for cyber threat intelligence

STEP 1

Raw cloud logs

STEP 2

Behaviour features

STEP 3

Learning models

STEP 4

Threat intelligence

16 modules

228 slides

Worked examples, code and checkpoints

ORIENTATION

How to read this tutorial: six kinds of box

PLAIN ENGLISH

An everyday analogy that explains a technical idea before the formal definition arrives. Read these first.

KEY TERM

A word you must know, defined in one sentence. Every key term is repeated in the glossary at the end.

WORKED EXAMPLE

Real numbers pushed through a formula, one step at a time, so you can redo the sum on paper.

WATCH OUT

A trap that catches beginners and experienced teams alike. Most real project failures live in these boxes.

IN CODE · PYTHON

Short Python you can type and run. Each snippet has a line-by-line explanation beside it.

AT MERIDIAN

How the idea plays out at Meridian Retail, the fictional company we follow from the first log line to the final model.

Checkpoints close every module with five questions. The answers sit in the speaker notes of the checkpoint slide.

ORIENTATION

Who this tutorial is for

The security analyst

You know alerts and incidents. You want to understand what the models behind your tools are doing, and to build your own.

The data scientist

You know models and metrics. You need the security vocabulary, the log sources and the traps that are unique to adversarial data.

The cloud engineer or student

You build or study cloud systems. You want one path that joins cloud, security and machine learning into a single skill.

What you need to bring

- Arithmetic, percentages and the patience to follow a sum
- Curiosity about how attackers think
- Optional: a little Python. Every line of code here is explained

WHAT YOU DO NOT NEED

No previous machine learning. No cloud certificate. No calculus. Each idea is built from the ground up.

ORIENTATION

The roadmap: sixteen modules in four phases

01 · Foundations Cloud foundations	02 · Foundations Security and CTI basics	03 · Foundations Data science foundations	04 · Data Cloud security data
05 · Data Exploring the data	06 · Data Feature engineering	07 · Models Supervised learning	08 · Models Evaluating detectors
09 · Models Anomaly detection	10 · Models Time, sequences and graphs	11 · Models Language models for CTI	12 · Intelligence Predictive threat intelligence
13 · Practice Cloud use cases	14 · Practice Models in production	15 · Practice Attacks on the models	16 · Practice Capstone project

CTI means cyber threat intelligence. Modules 1 to 3 can be skimmed if you already know that field; from Module 4 onward each module builds on the last.

No team can read its way to safety.

432,000,000

events a day from a cloud estate that logs
5,000 events every second

2,880

events one analyst can read in an 8-hour
shift at 10 seconds per event

150,000

analyst shifts to read a single day of logs:
about 400 years for one person

Illustrative arithmetic: $5,000 \times 86,400 \text{ seconds} = 432,000,000$. Then $432,000,000 \div 2,880 = 150,000$. Machines must do the first reading; people decide what it means.

ORIENTATION

Five things you will be able to do

1

Read cloud logs as data

Know what each log source records and how to turn it into a table.

2

Describe behaviour with features

Convert millions of events into numbers that capture who did what, when and how unusually.

3

Train and judge detectors

Choose an algorithm, train it, and measure it honestly when attacks are one event in a million.

4

Find threats nobody has labelled

Use anomaly detection, sequences and graphs to surface attacks that match no known signature.

5

Predict and prioritise

Forecast which vulnerabilities, accounts and assets are most likely to be attacked next, and turn that into decisions a security team can act on.

Meet Meridian Retail



Meridian Retail

A fictional company, invented for this tutorial

Business: an online retailer whose shop runs entirely in a public cloud.

Estate: a web shop in containers, customer records in a managed database, images and exports in object storage.

People: 400 staff, 60 engineers with cloud access, and a security team of four.

Data: every action in the cloud account is logged, far more than four people can review.

Three things keep the team awake



Stolen access keys. An engineer's credentials used by somebody else.



Customer data leaving. Storage read in bulk by an account that never did so before.



Hijacked compute. Servers started by an intruder to mine cryptocurrency on Meridian's bill.

Look for the "At Meridian" boxes: each one applies the idea on that slide to these three worries.

MODULE 01

Cloud foundations

Before we can defend a cloud, we need to know what one is, how it is built and where it can be attacked.

01

YOU WILL LEARN

What cloud computing is, in plain language, and the three ways it is sold

YOU WILL LEARN

Who is responsible for securing which layer: you or the provider

YOU WILL LEARN

How a real cloud attack unfolds, step by step, and the trail it leaves in the logs

What "the cloud" actually is

KEY TERM

Cloud computing

Renting computing resources (servers, storage, databases, networks and software) over the internet, on demand, and paying only for what you use.

PLAIN ENGLISH

A century ago factories ran their own generators. Today they plug into the grid and pay by the unit. The cloud did the same to computing: the power station is a provider's data centre, and the socket is the internet.

The five essential characteristics, from the NIST definition of cloud computing

On-demand self-service

Start a server in minutes, with no phone call.

Broad network access

Reach it from anywhere over the internet.

Resource pooling

Many customers share the same hardware.

Rapid elasticity

Capacity grows or shrinks as demand changes.

Measured service

Usage is metered and billed like a utility.

Three service models: how much do you rent?

IaaS

INFRASTRUCTURE AS A SERVICE

You rent: raw servers, storage and networks.

You manage: operating system, applications, data.

Examples: Amazon EC2, Azure Virtual Machines, Google Compute Engine

PaaS

PLATFORM AS A SERVICE

You rent: a ready platform that runs your code or database.

You manage: your code and your data.

Examples: AWS Elastic Beanstalk, Azure App Service, Google App Engine

SaaS

SOFTWARE AS A SERVICE

You rent: finished software, used through a browser.

You manage: your users, settings and data.

Examples: Microsoft 365, Google Workspace, Salesforce

PLAIN ENGLISH

Pizza. IaaS is renting a kitchen and cooking it yourself. PaaS is a take-and-bake pizza: you only bake. SaaS is the restaurant: you only eat.

Security consequence

The more the provider does, the less you can see. Your logs get thinner as you move from IaaS to SaaS.

Four deployment models: where does it run?

Public cloud

Shared infrastructure owned by a provider and rented to anyone.

For the defender: rich provider logs, but everything is reachable through internet-facing APIs.

Private cloud

Cloud-style infrastructure dedicated to one organisation.

For the defender: full control of the stack, and full responsibility for logging all of it.

Hybrid cloud

Public cloud joined to an organisation's own data centre.

For the defender: attackers cross the join, so identities and logs must be linked across both sides.

Multi-cloud

Two or more public providers used side by side.

For the defender: every provider names its logs and fields differently, so data must be normalised first.

KEY TERM

Tenant: one customer's isolated slice of a shared cloud. Keeping tenants apart is the provider's job; everything inside your tenant is yours to secure.

The four building blocks of any cloud



Compute

Where code runs: virtual machines, containers and serverless functions.

Goes wrong when software is left unpatched or an intruder starts machines of their own.



Storage

Where data rests: object storage buckets, disks and managed databases.

Goes wrong when a bucket is left open to the public or is read in bulk by a thief.



Network

How things connect: virtual networks, firewall rules, load balancers, DNS.

Goes wrong when a rule exposes an admin port to the whole internet.



Identity

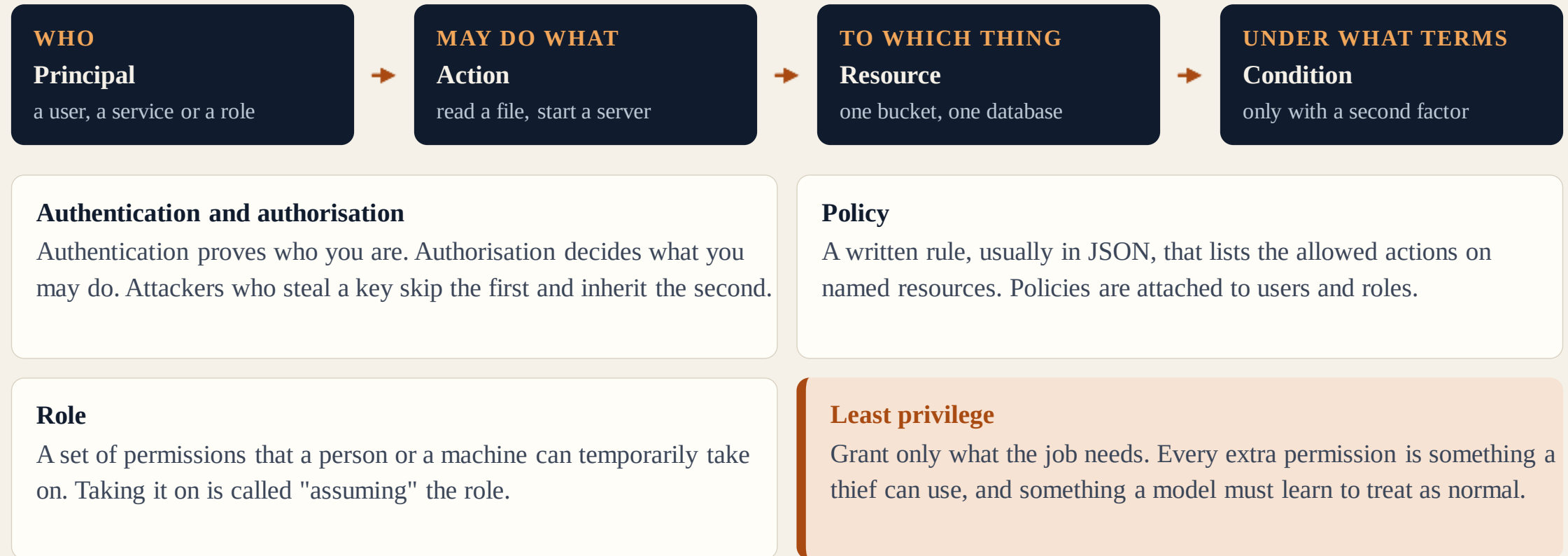
Who may do what: users, roles, policies and access keys.

Goes wrong when a key leaks or a role has more power than it needs. This block guards the other three.

AT MERIDIAN

The shop runs in containers (compute), product images and customer exports sit in buckets (storage), a load balancer faces the internet (network), and 60 engineers hold roles (identity).

Identity and access management in one sentence



Two planes: managing things and using things

Control plane

Actions that **create, change or delete** cloud resources.

Start a server. Open a firewall rule. Attach a policy to a user. Make a bucket public.

Recorded in audit logs: AWS CloudTrail management events, Azure Activity Log, Google Cloud Admin Activity logs. On by default, modest in volume.

Data plane

Actions that **use** the resources once they exist.

Read a file from a bucket. Run a database query. Send packets between two servers. Resolve a domain name.

Recorded in data-access logs, storage access logs, flow logs and DNS logs. Often off by default, and enormous in volume.

PLAIN ENGLISH

The control plane is the building manager's office: who was given which key, which walls were moved. The data plane is what people do inside the rooms.

Why a data scientist cares

The two planes give different datasets. Privilege abuse shows in the control plane; data theft shows in the data plane. A model fed only one of them is blind to the other.

The shared responsibility model

Layer to secure	IaaS	PaaS	SaaS
Data, and who may access it	You	You	You
Identities and configuration	You	You	You
Application code	You	You	Provider
Runtime and operating system	You	Provider	Provider
Virtualisation and network fabric	Provider	Provider	Provider
Buildings, power and hardware	Provider	Provider	Provider

A simplified view. Each provider publishes its own detailed version, and the exact line moves from service to service.

The provider secures the cloud itself.
You secure what you put in it.

WATCH OUT

The top two rows never move. Breach reports repeatedly trace cloud incidents to the customer's side of the line: a misconfiguration or a stolen credential. That is where our models will look.

Why the cloud changes the security problem

	IN THE OLD DATA CENTRE	IN THE CLOUD
Perimeter	A firewall around one building	Identity and API permissions
Lifespan	Servers lived for years and kept their addresses	Resources live for minutes; addresses are reused
Change	A ticket and an engineer with a cable	An API call, thousands of times a day, mostly by automation
Visibility	You owned the wire and could tap it	You see the logs the provider offers, if you turned them on
Speed of attack	Days to move through a network	Minutes: one stolen key can script an entire attack

So what for the data scientist?

Model identities, not machines. An IP address may belong to three different servers in one afternoon.

Learn behaviour, not rules. Static rules cannot keep up with thousands of legitimate changes a day.

Score fast. A detection that arrives tomorrow is a report, not a defence.

The cloud attack surface: six ways in

Misconfiguration

A bucket left public, a database port open to the world.

Data signal: configuration-change events and reads from unfamiliar addresses.

Stolen credentials

Keys leaked in code, passwords phished, tokens copied from a laptop.

Data signal: a known identity behaving in a way it never has before.

Vulnerable workloads

Unpatched software inside a virtual machine or container.

Data signal: odd processes, odd outbound connections, odd DNS lookups.

Insecure APIs

Application interfaces that trust their callers too much.

Data signal: request rates, error bursts and parameter patterns unlike real customers.

Supply chain

A poisoned software package, container image or third-party integration.

Data signal: a trusted component suddenly contacting somewhere new.

Insiders

A legitimate user who is careless, or who turns malicious.

Data signal: slow drift from the user's own history and from their peers.

Notice the pattern: every door leaves a different trace in the data. Matching the door to its trace is the whole craft of this tutorial.

Anatomy of a cloud breach, minute by minute

09:12 · Leak

An engineer commits an access key to a public code repository by mistake.

In the logs: nothing yet. The leak happens outside the cloud.

09:19 · First use

A scanning bot finds the key and tests whether it works.

In the logs: one "who am I?" call from an address never seen before.

09:24 · Discovery

The intruder lists users, roles, servers and buckets to map the account.

In the logs: a burst of List and Describe calls, many denied.

09:41 · Escalation

They attach an administrator policy to the stolen identity.

In the logs: a rare, powerful API call this engineer has never made.

10:05 · Hijack

Forty large servers are started in a region Meridian never uses, to mine cryptocurrency.

In the logs: new region, new instance type, sudden cost.

10:30 · Theft

Customer export files are copied out of object storage.

In the logs: bulk reads, far above this identity's daily norm.

78 minutes from leak to theft. Five of the six steps left a measurable trace. The rest of this tutorial is about catching those traces automatically.

Checkpoint 1: cloud foundations

Answer on paper first. The answers are in this slide's speaker notes.

- 1 Which service model leaves the operating system for you to patch?**
a) SaaS · b) PaaS · c) IaaS
- 2 Attaching a policy to a user is an action on which plane?**
a) Control plane · b) Data plane · c) Neither
- 3 In every service model, who is responsible for the data and for who may access it?**
a) The provider · b) The customer · c) Whoever built the application
- 4 Why is an IP address a weak way to identify a machine in the cloud?**
a) Addresses are encrypted · b) Resources are short-lived and addresses are reused · c) Providers hide them
- 5 In the Meridian breach, which step left no trace in the cloud logs?**
a) The key leak · b) The discovery burst · c) The bulk storage reads

MODULE 02

Security and threat intelligence basics

The vocabulary and the maps that defenders use to describe attackers.

02

YOU WILL LEARN

The words every security conversation rests on: threat, vulnerability, risk

YOU WILL LEARN

Four frameworks that describe how attacks unfold, including MITRE ATT&CK

YOU WILL LEARN

What cyber threat intelligence is, how it is produced and how it is shared

The three goals of security: the CIA triad



Confidentiality

Only the right people can read the data.

Broken by: a thief copying customer files out of a storage bucket.

A model watches: who reads what, how much, and from where.



Integrity

The data and systems are correct, and change only when authorised.

Broken by: an intruder altering prices, code or audit logs.

A model watches: who changes what, and whether that is normal for them.



Availability

The service works when people need it.

Broken by: a flood of traffic, ransomware, or deleted servers.

A model watches: traffic volume, error rates and resource deletions.

Every attack in this tutorial damages at least one of the three. When you design a detector, name which goal it protects: it tells you which logs to collect.

Six words that carry the whole subject

A **threat** uses an **exploit** against a **vulnerability** in an **asset**. The chance and cost of that is the **risk**. When it happens, it is an **incident**.

Asset

Anything worth protecting.
Meridian's customer database.

Threat

Anyone or anything that could cause harm.
A criminal group hunting for leaked keys.

Vulnerability

A weakness that a threat can use.
An access key committed to a public repository.

Exploit

The method or tool that takes advantage of the weakness.
A script that logs in with the leaked key.

Risk

How likely the harm is, multiplied by how bad it would be.
Likely and severe: a top priority.

Incident

An event that actually harms security.
The 78-minute breach on slide 18.

Measuring risk: likelihood times impact

Risk how much to worry	=	Likelihood how probable, scored 1 to 5	×	Impact how damaging, scored 1 to 5
----------------------------------	---	--	---	--

WORKED EXAMPLE

Leaked access key. Likelihood 4, impact 5. Risk = $4 \times 5 = 20$ out of 25.

Provider region loses power. Likelihood 1, impact 4. Risk = $1 \times 4 = 4$ out of 25.

In money: if one incident costs £40,000 and is expected once every two years (0.5 a year), the annual loss expectancy is $£40,000 \times 0.5 = £20,000$ a year.

Where data science enters

Scores of 1 to 5 are opinions. A model can replace the likelihood term with a probability estimated from evidence: 0.85 for this server, 0.02 for that one.

Module 12 does exactly this for vulnerabilities.

Who attacks, and why it changes the data

Actor	Motive	Typical behaviour in a cloud	What it means for detection
Opportunistic criminals	Quick money	Automated scanning for leaked keys and open buckets, then cryptomining	Loud and fast. Volume and novelty features catch them.
Ransomware and extortion groups	Large payouts	Steal data, delete backups, encrypt or threaten to publish	Watch bulk reads and deletion of snapshots and backups.
State-linked groups	Espionage, disruption	Patient and quiet; live inside legitimate identities for months	Hardest case. Needs long baselines and subtle drift.
Hacktivists	Ideology, publicity	Denial of service, defacement, leaking data	Traffic floods and public-facing changes.
Insiders	Grievance, money, or plain error	Use access they were properly given, for the wrong purpose	No "break-in" exists. Compare with own history and peers.



A model tuned for loud, fast criminals will miss a quiet, patient spy. Always ask which actor a detector is built for.

The Cyber Kill Chain: an attack in seven links

Stage	What the attacker does	A cloud example
1 · Reconnaissance	Studies the target	Scans for exposed buckets and leaked keys
2 · Weaponisation	Prepares a tool for the job	Writes a script that abuses the cloud API
3 · Delivery	Gets the tool to the target	Sends a phishing email to an engineer
4 · Exploitation	Triggers the weakness	Logs in with the stolen key
5 · Installation	Secures a lasting foothold	Creates a second access key or a hidden user
6 · Command and control	Opens a channel to steer the attack	A hijacked server calls home at regular intervals
7 · Actions on objectives	Does what they came to do	Copies customer data out; mines cryptocurrency

The idea worth keeping

Break any one link and the attack fails. The earlier the link, the cheaper the defence.

Its limit

Published by Lockheed Martin in 2011 for malware intrusions. Cloud attacks often skip or repeat stages.

MITRE ATT&CK: tactics, techniques, procedures

Tactic THE WHY

The attacker's goal at this moment. *Example: Credential Access.*

Technique THE HOW

A general method of reaching that goal. *Example: Unsecured Credentials, through the cloud instance metadata service.*

Procedure THE EXACT STEPS

How one particular group actually does it. *Example: a web request sent from a compromised server to read its role's temporary keys.*

KEY TERM

ATT&CK is a public knowledge base, maintained by MITRE, of the tactics and techniques seen in real attacks. Together they are called **TTPs**.

PLAIN ENGLISH

A burglar's tactic is "get inside". The technique is "pick the lock". The procedure is "this burglar uses a bump key on back doors at 3 a.m."

MODULE 2 • FRAMEWORK 2 OF 4

The fourteen ATT&CK tactics

1 • Reconnaissance

Gathering information about the target

2 • Resource Development

Setting up servers, accounts, tools

3 • Initial Access

Getting the first foothold

4 • Execution

Running the attacker's code

5 • Persistence

Keeping access over time

6 • Privilege Escalation

Gaining higher permissions

7 • Defense Evasion

Avoiding being detected

8 • Credential Access

Stealing passwords, keys, tokens

9 • Discovery

Mapping the environment from inside

10 • Lateral Movement

Moving on to other systems

11 • Collection

Gathering the data they want

12 • Command and Control

Steering compromised systems

13 • Exfiltration

Taking the data out

14 • Impact

Destroying, encrypting, disrupting

Reading the grid

Before, during, and the pay-off

AT MERIDIAN

The breach on slide 18 walked through Initial Access (3), Discovery (9), Privilege Escalation (6), Impact (14) and Exfiltration (13). Real intruders do not go in numerical order.

ATTACK in the cloud: techniques and their traces

Tactic	Cloud technique and ATT&CK ID	What it looks like in the logs
Initial Access	Cloud Accounts · T1078.004	A real identity signing in from a new place or device
Persistence	Additional Cloud Credentials · T1098.001	A new access key created for an existing user
Defense Evasion	Disable or Modify Cloud Logs · T1562.008	A call that stops or deletes an audit trail
Credential Access	Cloud Instance Metadata API · T1552.005	A server's role keys being used from outside the cloud
Discovery	Cloud Infrastructure Discovery · T1580	A burst of List and Describe calls across services
Collection	Data from Cloud Storage · T1530	Bulk object reads by an identity that rarely reads
Exfiltration	Transfer Data to Cloud Account · T1537	A snapshot or bucket shared with an unknown account
Impact	Resource Hijacking · T1496	Large servers launched in regions never used before

Why this table matters: each row is a detection you can build. The right-hand column tells you which features to engineer in Module 6.

Technique names and IDs are revised from time to time. Check the current ATT&CK matrix on the MITRE website before you cite one.

The Diamond Model: four corners of every intrusion



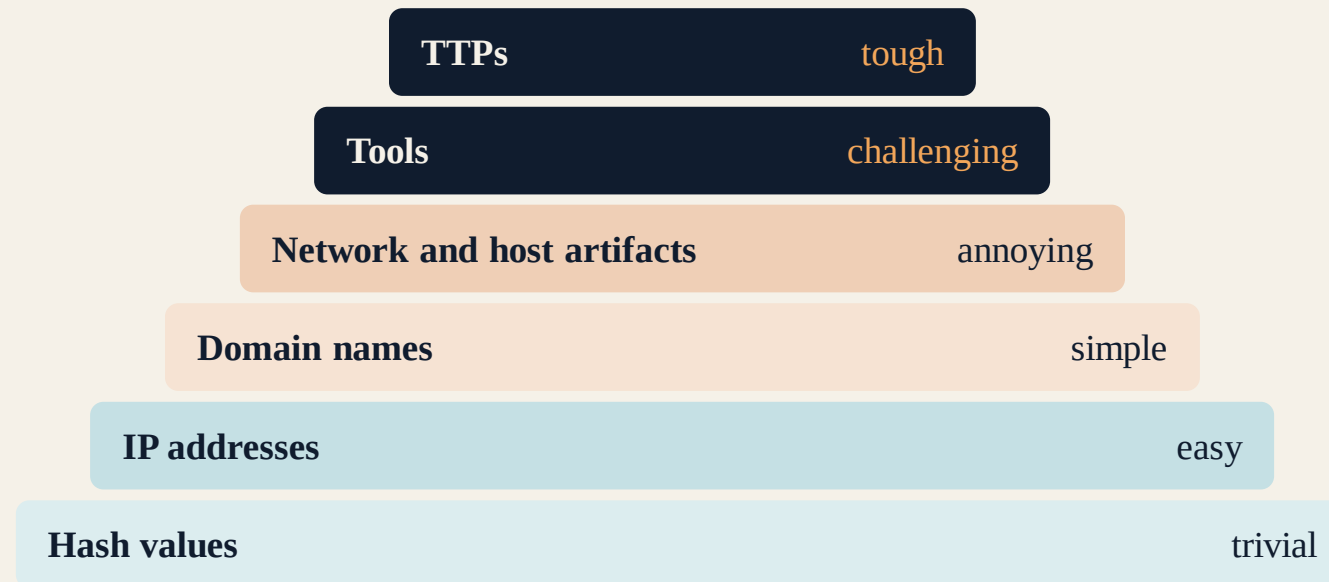
Its use: pivoting

Know one corner and you can reach the others. An attacking address (infrastructure) leads to the malware that called it (capability), which leads to other victims of the same adversary.

Why a data scientist smiles

Corners joined by edges are a graph. Module 10 turns this picture into graph analytics.

The Pyramid of Pain: what hurts an attacker most



Right-hand word: how hard it is for the attacker to change that indicator once you block it. After David Bianco, 2013.

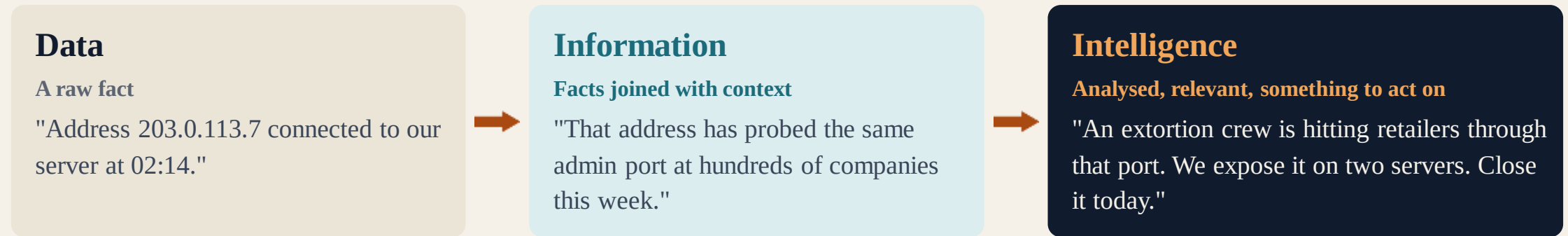
Bottom of the pyramid

Block a file hash and the attacker changes one byte. Signature tools live here: cheap to build, cheap to evade.

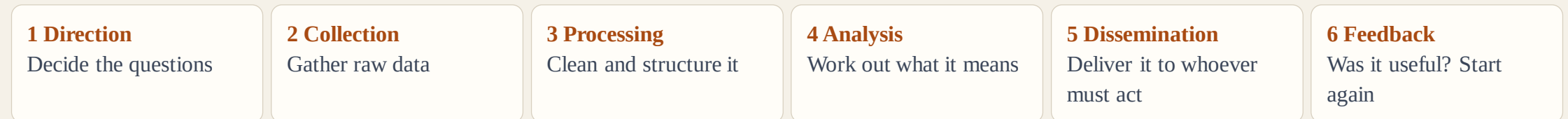
Top of the pyramid

Detect the behaviour itself and the attacker must change how they work. Machine learning on behaviour aims here.

From data to information to intelligence



The intelligence cycle: how that last box gets written, again and again



Four levels of threat intelligence

Strategic

Who is targeting our sector, and why?

Reader: executives and the board

Shelf life: months to years

Data science adds: trend analysis and risk forecasts

Operational

Which campaign is under way, and when might it reach us?

Reader: security managers and incident responders

Shelf life: weeks

Data science adds: clustering events into campaigns

Tactical

How do they operate? Which TTPs do they use?

Reader: detection engineers and threat hunters

Shelf life: weeks to months

Data science adds: behaviour models and TTP classifiers

Technical

Which exact addresses, domains and file hashes?

Reader: security tools, automatically

Shelf life: hours to days

Data science adds: indicator scoring and ageing



Read the cards from right to left and you climb the Pyramid of Pain: short-lived indicators at the technical level, durable behaviour and intent at the top.

Indicators, and how intelligence is shared

Indicator of compromise

IOC · EVIDENCE LEFT BEHIND

A file hash, an IP address, a domain name. It says "this known bad thing was here". Precise, but easy for the attacker to change.

Indicator of attack

IOA · BEHAVIOUR IN PROGRESS

A pattern of actions: "an identity lists every bucket, then reads them all". It says "this looks like an attack". Harder to evade.

THREE KEY TERMS FOR SHARING

STIX is a standard language, written in JSON, for describing threats: indicators, malware, threat actors and the relationships between them.

TAXII is the protocol that carries STIX between organisations over HTTPS.

TLP, the Traffic Light Protocol, labels how far a report may travel.

RED: named recipients only

AMBER: your organisation

GREEN: your community

CLEAR: anyone

For the data scientist: STIX gives threat reports a regular structure, which means they can be loaded, joined to your logs and used as model features.

MODULE 2 · SECURITY OPERATIONS

Where data science plugs into the security team

SOC · Security Operations Centre: the people who monitor, investigate and respond.

SIEM · the central system that stores security logs and raises alerts.

SOAR · the system that automates response steps, called playbooks.

1 · Collect

Bring every log into one place.

ML adds: parsing messy logs and matching identities across sources.

2 · Detect

Spot what might be an attack.

ML adds: classifiers and anomaly detectors beside the rules.

3 · Triage

Decide which alerts deserve a human.

ML adds: risk scores, ranking, grouping of duplicates.

4 · Investigate

Work out what really happened.

ML adds: threat-intel enrichment, related-entity graphs, summaries.

5 · Respond

Contain, remove, recover.

ML adds: recommended actions, with a person approving the risky ones.

0 · Predict

Before any of the five: forecast which vulnerabilities, accounts and assets are most likely to be attacked, so the team can fix them first. This is predictive threat intelligence, the subject of Module 12.

Checkpoint 2: security and CTI basics

Answer on paper first. The answers are in this slide's speaker notes.

- 1** In ATT&CK, "Credential Access" is a...
a) tactic · b) technique · c) procedure
- 2** Which indicator is hardest for an attacker to change once you detect it?
a) A file hash · b) An IP address · c) Their TTPs
- 3** A risk has likelihood 2 and impact 5 on five-point scales. What is its score?
a) 7 · b) 10 · c) 25
- 4** Which level of threat intelligence holds specific addresses and file hashes?
a) Strategic · b) Tactical · c) Technical
- 5** TAXII is...
a) a language for describing threats · b) a protocol for exchanging threat information · c) a label that limits sharing

MODULE 03

Data science foundations

How a machine learns from examples, and the small amount of mathematics the rest of the tutorial leans on.

03

YOU WILL LEARN

What "learning" means for a machine, and the three styles of it

YOU WILL LEARN

The working vocabulary: sample, feature, label, model, training

YOU WILL LEARN

Averages, spread, z-scores and Bayes' rule, each with a worked example

What data science is

Data science is the craft of turning data into decisions. It sits where three skills overlap.



Statistics

Knowing what the numbers can and cannot tell you, and how sure you are.

+



Computing

Collecting, cleaning and processing data at a scale no spreadsheet survives.

+



Domain knowledge

Knowing the field. Here: how clouds work and how attackers behave.

WATCH OUT

In security the third box is not optional. A model built without it finds statistical oddities, such as the backup job that runs at 2 a.m., and calls them threats. That is why Modules 1 and 2 came first.

AI, machine learning, deep learning: boxes in boxes

Artificial intelligence · any technique that makes a machine do a task that seems to need intelligence. Hand-written rules count.

Machine learning · the machine works out the rules from examples, instead of being told them.

Deep learning · machine learning with many-layered neural networks. Strong on raw text, images and long sequences.

Large language models · deep networks trained on text, able to read and write it.

In a security product

A rule that says "alert after five failed logins" is AI of the oldest kind. A model that learns each user's normal number of failures is machine learning.

Where this tutorial spends its time

Mostly in the machine learning box (Modules 7 to 10), because log data is tabular and classical methods are strong there. Deep learning and language models appear in Modules 7, 10 and 11.

How a machine learns

TRADITIONAL PROGRAMMING

Rules, written by a person

Data



Answers

"If failed logins exceed five, raise an alert." Someone had to think of the rule, and of the number five.

MACHINE LEARNING

Data

Answers from the past



Rules

Show it 10,000 past logins, each marked "fine" or "attack", and it works out the rule, including numbers nobody guessed.

PLAIN ENGLISH

You do not hand a new analyst a 500-page rulebook. You sit them beside a senior colleague, show them a thousand closed alerts, and they pick up the pattern. A model is trained the same way, only with numbers.

The catch

The model is only as good as the examples. Wrong answers in, wrong rules out.

Three ways of learning

Supervised

EXAMPLES WITH ANSWERS

Learns to map inputs to a known answer.

Like: flashcards with the answer on the back.

In security: is this email phishing? Is this file malware? Module 7.

Unsupervised

EXAMPLES ONLY, NO ANSWERS

Finds groups, structure and things that do not fit.

Like: sorting a drawer of unlabelled keys by shape.

In security: which identity is behaving unlike all the others? Module 9.

Reinforcement

TRIAL, ERROR AND REWARD

Learns which actions pay off by trying them.

Like: learning a game by playing it.

In security: automated attack simulation and response agents. Still mostly research.



The everyday reality is in between. Security teams have a few hundred confirmed incidents and billions of unlabelled events. Methods that combine both are called semi-supervised learning.

The working vocabulary of machine learning

Sample

One thing being judged: a row in the table.
One login. One file. One hour of one user's activity.

Feature

A measurable property of the sample: a column.
Hour of day. Bytes sent. Is the country new for this user?

Label

The answer we want predicted, when we know it.
"Malicious" or "benign", decided by an investigation.

Model

The learned rule that turns features into a prediction.
A formula, a tree of questions, or a neural network.

Training

Adjusting the model until its predictions match the labels.
Done once in a while, on historical data.

Inference

Using the trained model on new, unseen samples.
Done constantly, on live events. Also called scoring.

Two more you will meet soon: a **parameter** is a number the model learns for itself; a **hyperparameter** is a setting you choose before training, such as how deep a tree may grow.

Anatomy of a dataset

user	hour	new_country	failed_1h	mfa	label
amara	9	0	0	1	benign
li.wei	14	0	1	1	benign
li.wei	3	1	6	0	malicious
tomasz	10	0	0	1	benign
svc-backup	2	0	0	0	benign

Five sign-in events at Meridian. "failed_1h" counts failed attempts in the previous hour; "mfa" is 1 when a second factor was used; 1 means yes and 0 means no.

Each row is a sample. Here, one sign-in.

Each middle column is a feature. Together the features are written **X**.

The last column is the label, written **y**. Training means finding a rule that turns **X** into **y**.

Look at row 3 and row 5. Both happen in the small hours without a second factor. Only one is an attack. Night-time alone is not the pattern.

The machine learning workflow: six steps, one loop

1 Frame the problem

What decision will the model support? What does a mistake cost?

2 Understand the data

Which logs exist? What is in them? Modules 4 and 5.

3 Prepare the data

Clean it and build features. Module 6. Most of the effort goes here.

4 Train models

Try a simple one first, then better ones. Modules 7, 9, 10.

5 Evaluate honestly

On data the model has never seen, with the right metric. Module 8.

6 Deploy and monitor

Score live events; watch for decay; retrain. Module 14.

It is a loop. Evaluation sends you back to the features; monitoring sends you back to the problem. This sequence follows the industry's CRISP-DM process model.

Watch out: beginners rush to step 4. Projects are won or lost in steps 1 and 3, long before an algorithm is chosen.

Three piles of data, split by time

Training set · about 70%

January to August. The model learns its rules from these events.

Validation

September. You tune settings here.

Test

October. Used once, at the end.

older events —————> newer events

PLAIN ENGLISH

Training data is the textbook exercises. Validation is the mock exam you may sit many times. The test set is the final exam: sit it once, or the mark means nothing.

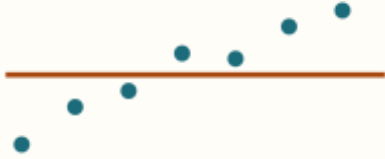
Why by time?

In production the model always predicts the future from the past. The evaluation must copy that, so the test events must come after the training events.

WATCH OUT

A random shuffle puts pieces of one attack in both training and test. The model "recognises" it and scores brilliantly, then fails on the next, new attack.

Too simple, about right, too clever



Underfitting

"Every login after midnight is an attack."

The rule is too crude. It is wrong on the training data and wrong on new data.

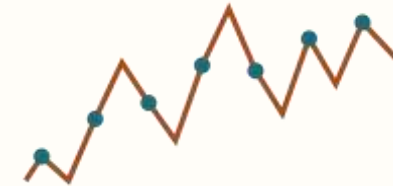
Fix: add better features or use a richer model.



A good fit

"A new country, after several failures, with no second factor, is suspicious."

It captures the pattern and ignores the noise. Good on training data and on new data.



Overfitting

"Attacks come from 203.0.113.7 at 03:14 on Tuesdays."

It memorised the examples. Perfect on training data, useless on anything new.

Fix: more data, a simpler model, or regularisation.

KEY TERMS

Bias is the error from being too simple. **Variance** is the error from clinging to the training examples. **Regularisation** is any penalty that discourages needless complexity.

Typical, spread, and how unusual

Mean

Add the values, divide by how many. Dragged about by extremes.

Median

The middle value when sorted. Barely moved by extremes.

Standard deviation

The typical distance of a value from the mean. Small means consistent.

Percentile

The 99th percentile is the value that 99% of observations fall below.

z-score = (value – mean) ÷ standard deviation

How many standard deviations a value sits from normal. Beyond about 3 is rare in well-behaved data.

WORKED EXAMPLE

An engineer's daily downloads in megabytes: **90, 100, 110, 120, 130**.

Mean = $550 \div 5 = \mathbf{110}$. Median = **110**.

Add one strange day of **900**. Mean = $1,450 \div 6 = \mathbf{242}$. Median = $(110 + 120) \div 2 = \mathbf{115}$.

One outlier doubled the mean and hardly touched the median. Use the median to describe "normal".

Another user: mean 120 MB, standard deviation 40 MB. Today: 400 MB.

$z = (400 - 120) \div 40 = \mathbf{7.0}$. Seven deviations out: investigate.

Bayes' rule: what an alert really tells you

The question: a detector flags a login. It catches 95% of malicious logins and wrongly flags 2% of honest ones. Only 1 login in 1,000 is malicious. How likely is this flagged login to be an attack?

WORKED EXAMPLE · COUNT THE PEOPLE

Imagine 100,000 logins.

100 are malicious

95% are flagged:

95 true alerts

99,900 are honest

2% are flagged:

1,998 false alerts

Flagged in total: $95 + 1,998 = 2,093$. Real attacks among them: $95 \div 2,093 = 4.5\%$.

4.5%

Not 95%. More than 95 in every 100 alerts from this "95% accurate" detector are false alarms.

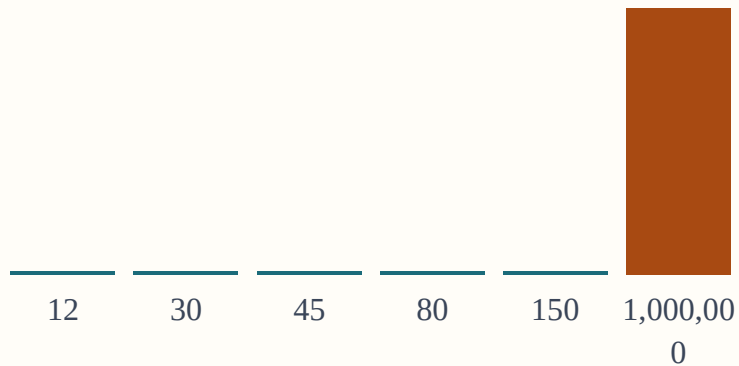
Bayes' rule in words

Chance it is real, given an alert = true alerts $\div$ all alerts. When attacks are rare, false alerts from the honest majority swamp the true ones.

Security data has heavy tails

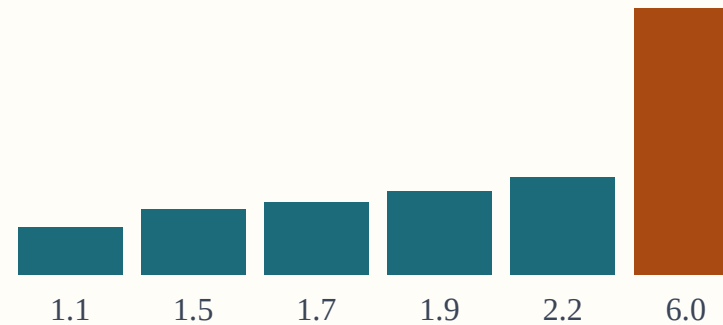
Most identities make a handful of API calls a day. A few automated service accounts make a million. Plot them together and the many vanish beside the few.

Raw daily API calls, six identities



Five humans are flattened to nothing by one service account.

The same six after taking log base 10



Now every identity is visible, and the differences between humans show.

Key term: logarithm

The log base 10 of a number is how many times you multiply 10 to reach it. 100 gives 2; 1,000,000 gives 6.

Rule of thumb

Counts, bytes and durations almost always need a log transform before modelling.

MODULE 3 · DATA SCIENCE FOUNDATIONS

The toolkit: what each tool is for

Python

The language. Readable, free, and where the libraries live.

Jupyter

A notebook: run code a cell at a time and see results beneath it.

pandas

Tables in memory, called DataFrames. Filter, group, join.

NumPy

Fast arithmetic on whole arrays of numbers at once.

scikit-learn

Classical machine learning, with one consistent interface.

XGBoost, LightGBM

Gradient-boosted trees: the usual winners on tabular data.

Matplotlib, seaborn

Charts, from quick histograms to finished figures.

PyTorch, TensorFlow

Deep learning: neural networks of any shape.

SQL

Querying logs where they live, in a warehouse or a SIEM.

Apache Spark

pandas-style work spread across many machines.

NetworkX

Graphs: who connects to what, and through whom.

SHAP

Explains why a model gave a particular score.

IN CODE · SET UP ONCE, IN A TERMINAL

```
pip install pandas numpy scikit-learn matplotlib jupyter
```

Checkpoint 3: data science foundations

Answer on paper first. The answers are in this slide's speaker notes.

- 1** A column that measures some property of each event is called a...
a) label · b) feature · c) model
- 2** Learning from past alerts that analysts marked "real" or "false alarm" is...
a) supervised learning · b) unsupervised learning · c) reinforcement learning
- 3** A model is right 99% of the time on training data and 60% on new data. It is...
a) underfitting · b) overfitting · c) well tuned
- 4** Mean daily download 200 MB, standard deviation 50 MB, today 450 MB. The z-score is...
a) 2.5 · b) 5 · c) 9
- 5** Why split security data by time instead of at random?
a) It is faster · b) It stops the model seeing the future · c) It uses less memory

MODULE 04

Cloud security data

Every model in this tutorial eats logs. This module shows what is on the plate.

04

YOU WILL LEARN

The eight families of cloud telemetry and which attacks each one reveals

YOU WILL LEARN

How to read an audit event, a flow record and a sign-in event field by field

YOU WILL LEARN

How raw logs become a clean, labelled dataset, and where that process goes wrong

The telemetry map: eight families of logs

Control-plane audit

Every management API call: who, what, when, from where.

Reveals: escalation, discovery, persistence.

Identity sign-ins

Each login: user, device, place, method, result.

Reveals: stolen credentials, password spraying.

Network flows

Who talked to whom: addresses, ports, bytes. No content.

Reveals: scanning, exfiltration, beaconing.

DNS queries

Every domain name a workload looked up.

Reveals: command and control, tunnelling.

Storage access

Each object read, written or deleted.

Reveals: data theft, ransomware.

Workload and OS

Processes, files and system calls inside servers and containers.

Reveals: malware, cryptominers.

Kubernetes audit

Every request to the cluster's own API.

Reveals: secret theft, rogue containers.

Application and web

Requests, responses and errors at the front door.

Reveals: web attacks, API abuse.



The two navy boxes are the control plane and are the best place to start: on by default, modest in size, and centred on identity. One extra source many teams forget: the **billing data**, which shows hijacked compute as money.

The same logs under three providers' names

What you want	Amazon Web Services	Microsoft Azure	Google Cloud
Control-plane audit	CloudTrail management events	Activity Log	Admin Activity audit logs
Identity sign-ins	CloudTrail sign-in events	Entra ID sign-in logs	Login audit logs
Network flows	VPC Flow Logs	Virtual network flow logs	VPC Flow Logs
DNS queries	Route 53 Resolver query logs	Azure DNS query logs	Cloud DNS logs
Storage access	CloudTrail data events; S3 access logs	Storage resource logs	Data Access audit logs
Built-in threat detection	GuardDuty	Defender for Cloud	Security Command Center
Security lake or SIEM	Security Lake	Microsoft Sentinel	Google Security Operations

Watch out: product names change often. Treat this table as a map of categories and confirm the current name in the provider's documentation.

Good news: the built-in detectors in row six are machine learning products too. Their findings make excellent extra features and rough labels for your own models.

Anatomy of a control-plane audit event

ONE AWS CLOUDTRAIL EVENT, SIMPLIFIED

eventTime: 2026-03-14T09:41:07Z

eventSource: iam.amazonaws.com

eventName: AttachUserPolicy

userIdentity.userName: amara.okafor

sourceIPAddress: 198.51.100.24

userAgent: aws-cli/2.15

awsRegion: us-east-1

requestParameters.policyArn: .../AdministratorAccess

errorCode: none, so the call succeeded

When eventTime, always in UTC. Feature: hour of day; time since this user's last call.

What eventSource and eventName. Feature: has this user ever made this call before?

Who userIdentity. The entity whose behaviour we will model.

Where sourceIPAddress and awsRegion. Feature: new address, new country, unused region.

With what userAgent: console, command line or a script library. Feature: tool never used by this user.

Outcome errorCode. Feature: share of calls denied in the last hour.

This is the 09:41 escalation from Meridian's breach. One record, six questions, at least six features.

Anatomy of a network flow record

ONE AWS VPC FLOW LOG LINE, DEFAULT FORMAT

2 123456789012 eni-0a1b2c3d 10.0.1.25 203.0.113.50 49152 443 6 1840 2650000 1741945200 1741945260 ACCEPT OK

Field	Value here	Meaning
srcaddr, dstaddr	10.0.1.25, 203.0.113.50	From an internal server to an outside address
srcport, dstport	49152, 443	Port 443 is HTTPS, encrypted web traffic
protocol	6	6 means TCP; 17 would mean UDP
packets, bytes	1,840 and 2,650,000	How much was sent in this direction
start, end	60 seconds apart	The capture window, in Unix time
action	ACCEPT	Whether the firewall rules allowed it

Plain English

A flow record is the envelope, never the letter: sender, recipient, weight and time, but not what was written inside.

Worked example

Bytes per packet = 2,650,000 ÷ 1,840 = 1,440. Packets that full, for a whole minute, outbound: a server is uploading something large.

Anatomy of a sign-in event

SIGN-IN RECORD · SIMPLIFIED

li.wei

Time · 03:07 UTC

Application · cloud management console

Address · 198.51.100.24

Location · a country never seen for this user

Device · unrecognised browser

Method · password only, no second factor

Result · success, after 6 failures

Three attack patterns that live in sign-in logs

Impossible travel

One user signs in from two places too far apart for the time between them. Worked through in Module 6.

Password spraying

One address tries a few common passwords against hundreds of users. Each user sees almost nothing; the address sees everything. Model the address, not the user.

Prompt bombing

The attacker has the password and triggers approval prompts until a tired user taps "yes". Look for many denied prompts followed by one approval.

Normalisation: one name for one thing

Three providers record the same three facts under nine different field names. A model cannot learn that they mean the same thing; you must tell it.

The fact	AWS CloudTrail	Azure Activity Log	Google Cloud audit log	Common schema
Who acted	userIdentity.arn	caller	authenticationInfo.principalEmail	actor.user
What they did	eventName	operationName	methodName	api.operation
From where	sourceIPAddress	callerIpAddress	requestMetadata.callerIp	src_endpoint.ip

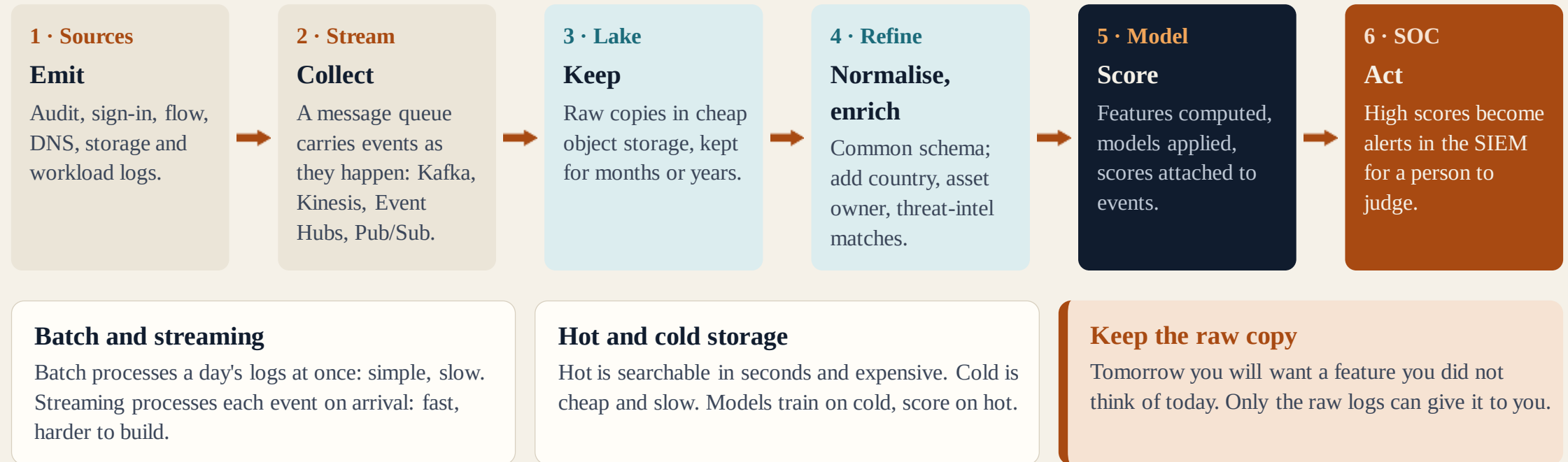
KEY TERMS

OCSF, the Open Cybersecurity Schema Framework, is a vendor-neutral standard for naming security fields; the right-hand column uses it. **ECS**, the Elastic Common Schema, is a widely used alternative.

Why it pays

Normalise once and every feature, model and dashboard you build works across all your clouds. Skip it and you build everything three times.

The security data pipeline, from log to alert



Where labels come from

Confirmed incidents

Events tied to an investigated breach.

Strength: the truth.

Weakness: a handful a year.

Analyst triage decisions

Each closed alert marked true or false positive.

Strength: thousands of them.

Weakness: only covers what old rules already flagged.

Red-team exercises

Hired attackers, with a record of what they did and when.

Strength: realistic, precisely timed.

Weakness: one team's habits, not every adversary's.

Honeypots and decoys

Fake keys and servers nobody should ever touch.

Strength: every touch is hostile.

Weakness: attracts noisy attackers more than careful ones.

Threat-intelligence matches

Events touching a known bad address or domain.

Strength: automatic.

Weakness: indicators go stale and misfire.

Simulated attacks

Scripts that replay known techniques in a test account.

Strength: unlimited supply.

Weakness: cleaner than anything real.

Watch out for the silent trap: everything unlabelled gets treated as "benign", yet undetected attacks are hiding in there. Labels in security are scarce, biased and sometimes wrong. Plan for it.

Public datasets to practise on

Dataset	What it holds	Good for	Caveat
NSL-KDD	Network connections, a cleaned KDD Cup 1999	First experiments	Traffic from the 1990s
UNSW-NB15	Network flows with nine attack families	Intrusion classification	Generated in a lab
CIC-IDS2017	Five days of labelled flows, many attacks	Flow-based detection	Known labelling errors
CSE-CIC-IDS2018	A larger successor, built on AWS	Practice at scale	Simulated users
CTU-13	Thirteen captures of real botnet traffic	Botnets and beaconing	Captured in 2011
LANL authentication	58 days of logins with red-team events	Lateral movement, graphs	Anonymised; few attacks
EMBER	Features of about a million Windows files	Malware classification	Static features only
flaws.cloud logs	Real CloudTrail from a public training lab	Cloud API behaviour	No labels supplied

Watch out: a model that scores 99.9% on a public benchmark has learned that benchmark. It tells you little about your own cloud.

How to use them: to learn the workflow and compare methods. Then repeat every step on your own organisation's logs.

Six data-quality traps in cloud logs

1 · Silent gaps

Logging was never switched on for one region or service. Silence there does not mean safety.

2 · Clocks and time zones

Some logs use UTC, some local time, and many arrive minutes late and out of order.

3 · Duplicates

The same event delivered twice inflates every count you compute.

4 · Schema drift

The provider renames or adds a field. Your parser quietly fills the column with blanks.

5 · Shared addresses

An office of 300 people appears as one IP address; one traveller appears as ten.

6 · Sampling and truncation

Flow logs summarise windows of time; long fields are cut short. Detail you assume is there may not be.



The habit that saves you: monitor the data before you monitor the attackers. Chart events per hour for every source, and alert when a source goes quiet.

Loading audit logs into a table

IN CODE • PYTHON

```
import json
import pandas as pd
with open("cloudtrail.jsonl") as f:
    events = [json.loads(line) for line in f]
df = pd.json_normalize(events)
df["eventTime"] = pd.to_datetime(df["eventTime"], utc=True)
df = df.drop_duplicates("eventID").sort_values("eventTime")
print(df.shape)
print(df["eventName"].value_counts().head(10))
```

- 1** **import** brings in two libraries: one reads JSON text, one builds tables.
- 2** Open the file, which holds one event per line, and turn each line into a Python dictionary.
- 3** **json_normalize** flattens nested fields, so `userIdentity.userName` becomes its own column.
- 4** Turn timestamp text into real times, all in UTC.
- 5** Drop repeated events, then put the rest in time order.
- 6** Two sanity checks: how many rows and columns, and the ten most common API calls.

Each numbered box explains one paragraph of code, top to bottom. Traps 2 and 3 from the previous slide are both handled here, in two lines.

Checkpoint 4: cloud security data

Answer on paper first. The answers are in this slide's speaker notes.

- 1 Which log records who called which management API, and when?**
a) Network flow logs · b) Control-plane audit logs · c) DNS query logs
- 2 A network flow record contains...**
a) the full content of each packet · b) addresses, ports and byte counts · c) the user's name
- 3 Why normalise logs into a common schema?**
a) To shrink them · b) So one set of features and models works across providers · c) Because the law requires it
- 4 Labels taken only from alerts your existing rules raised will...**
a) cover every attack · b) miss whatever those rules never flagged · c) be free of mistakes
- 5 "No events from that region last night" means...**
a) nothing happened · b) logging was off · c) either one, so check before trusting it

MODULE 05

Exploring the data

Look before you model. An hour of exploration saves a week of training the wrong thing.

05

YOU WILL LEARN

Five questions to ask of any log source before trusting it

YOU WILL LEARN

How to see what is typical, what follows the clock and what is rare

YOU WILL LEARN

How observations become hypotheses, and hypotheses become features

Exploratory data analysis: five questions

KEY TERM

Exploratory data analysis, or EDA, means studying a dataset with summaries and charts before building any model on it.

PLAIN ENGLISH

A detective walks the whole scene before naming a suspect. EDA is the walk.

1

How much, how complete?

Rows, date range, gaps, empty fields.

2

What is one row?

An API call? A minute of traffic? A whole session?

3

What is typical?

Medians and distributions, per user and overall.

4

What follows the clock?

Working hours, weekends, nightly jobs.

5

What is rare?

Values seen once, extremes, things that never repeat.

The next five slides take the questions in order, using one month of Meridian's audit log as the example.

First look: size, span and gaps

IN CODE · PYTHON

```
print(df.shape)
print(df["eventTime"].min(), df["eventTime"].max())
print(df["userIdentity.userName"].nunique())
print(df.isna().mean().sort_values().tail(5))
hourly = df.set_index("eventTime").resample("1h").size()
hourly.plot()
```

Line 1 rows and columns. **Line 2** earliest and latest event. **Line 3** how many distinct identities. **Line 4** the five emptiest columns, as a share of rows. **Lines 5 and 6** count events in each hour and chart them.

AT MERIDIAN · ILLUSTRATIVE RESULTS

1,284,310 events across **30 days**, from **212 identities**. One row is one API call.

errorCode is empty in 96% of rows. Not a problem: empty means the call succeeded. Always learn what "missing" means before filling it.

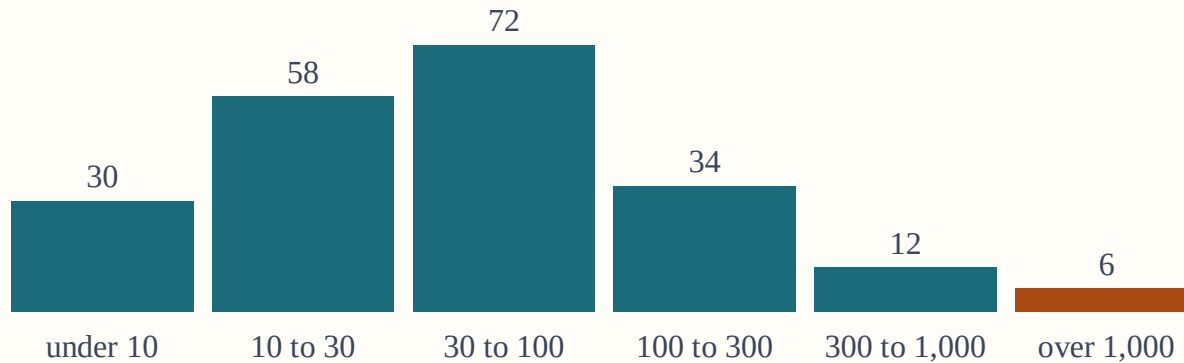
The hourly chart drops to zero for four hours on day 12.

Nobody stopped working. The log delivery failed. Mark those hours as "no data", never as "quiet".

MODULE 5 • QUESTION 3

What is typical? Draw the distribution

Identities at Meridian, grouped by API calls per day



Number above each bar: identities in that band. 212 in total. Illustrative data.

48

median calls a day

202

mean calls a day

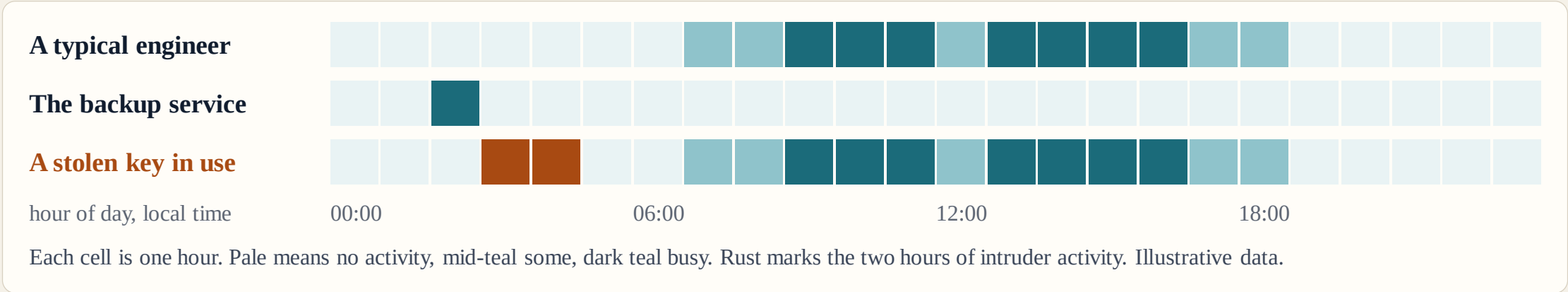
Why the two disagree

The six identities in the rust bar are automated service accounts. Together they make about 70% of all calls and pull the mean to four times the median.

What to take from it

There is no single "normal". People and machines need separate baselines, and each identity is best compared with its own history.

What follows the clock? Activity by hour of day



People keep office hours
With a lunch dip, quiet weekends and holidays. This repeating shape is called seasonality.

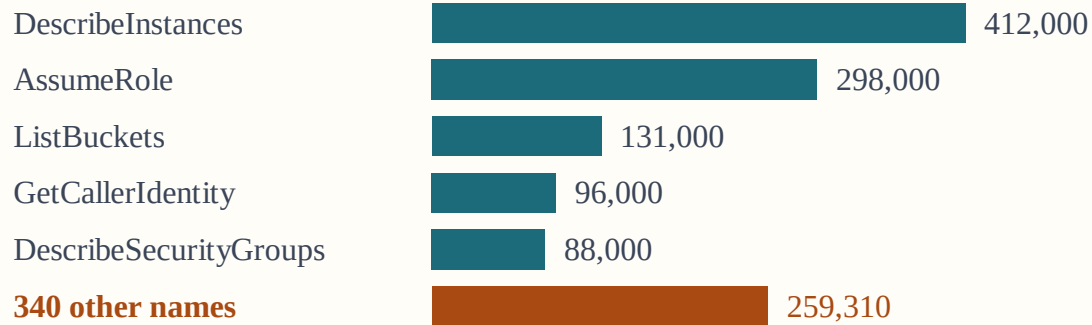
Machines keep schedules
A 2 a.m. spike is normal for the backup service and would be alarming for an engineer.

So "unusual hour" is personal
Compare each identity with its own rhythm, never with a single company-wide rule.

MODULE 5 • QUESTION 5

What is rare? Categories and the long tail

API calls in one month, by name



The top five names are 80% of all events (1,025,000 of 1,284,310). Illustrative data.

Deep in the tail, this month

CreateAccessKey · 7 times
PutBucketPolicy · 4 times
AttachUserPolicy · 3 times
StopLogging · once

The lesson of the tail

The calls an attacker needs are the calls almost nobody makes. In security data the rare end of the chart is where the story is.

Key term: cardinality is the number of distinct values in a column. 345 event names is high cardinality, which matters when we encode it in Module 6.

MODULE 5 • WORKED EXAMPLE

Finding outliers with the interquartile range

One engineer's API calls on eleven working days, sorted from smallest to largest:



STEP BY STEP

Median (navy), the middle value: 220.

First quartile Q1, the middle of the lower five: 180. **Third quartile Q3**, the middle of the upper five: 260.

Interquartile range = $Q3 - Q1 = 260 - 180 = 80$.

Upper fence = $Q3 + 1.5 \times 80 = 260 + 120 = 380$.

950 is far above 380, so it is an outlier.

Why not just use the z-score?

The outlier corrupts its own yardstick. With 950 included, the mean jumps to 282 and the standard deviation to about 215, so its z-score is only 3.1. Quartiles ignore the extreme and judge it fairly.

Key term: robust statistics are measures such as the median and the interquartile range that a few extreme values cannot distort.

From observation to hypothesis to feature

WE SAW	SO WE SUSPECT	SO WE WILL BUILD
Six service accounts make 70% of all calls.	People and machines behave too differently to share one model of "normal".	An identity-type flag, and a separate baseline for each type.
Engineers work 08:00 to 19:00; services run on fixed schedules.	Activity that breaks an identity's own rhythm is informative.	"Share of this identity's past activity that fell in this hour."
Sensitive calls such as AttachUserPolicy sit in the rare tail.	Rarity matters more than volume.	"First time this identity made this call" and a rarity score per call.
A four-hour hole in the logs on day 12.	Silence can mean "no data" as easily as "no activity".	A data-quality flag; exclude that window from training.

This table is the real output of EDA. Charts are only the means. The right-hand column is the to-do list for Module 6.

MODULE 06

Feature engineering

Turning a stream of events into numbers that describe behaviour. This is where security knowledge becomes mathematics.

06

YOU WILL LEARN

Nine families of feature that can be built from cloud logs

YOU WILL LEARN

Windows, aggregation, encoding and scaling, the four basic moves

YOU WILL LEARN

Three features worked by hand: clock time, impossible travel and entropy

A simple model with the right features beats a brilliant model with the wrong ones.

WHAT THE LOG SAYS

"amara.okafor called AttachUserPolicy at 09:41 from 198.51.100.24 using the command line."

A sentence. A model cannot do arithmetic on a sentence.



WHAT THE MODEL NEEDS

first_time_call = 1	new_address = 1
changes_privilege = 1	calls_last_10_min = 46
denied_share_1h = 0.35	usual_hour_share = 0.11

Six numbers. Each one encodes a piece of what an analyst would notice.

Feature engineering is the work of the arrow: turning raw events into numbers that describe behaviour. It is where your knowledge of clouds and attackers enters the model.

Nine families of feature

1 Counts

How many. API calls by this identity in the last hour.

2 Rates and ratios

One count over another. Denied calls $\div$ all calls.

3 Diversity

How many different. Distinct regions touched today.

4 Novelty

Never seen before for this identity. A new address, a first-ever call.

5 Time

When and how regularly. Hour, weekday, seconds since the last event.

6 Location

Where from. Country, network owner, speed of travel between logins.

7 Text

What names look like. Length and randomness of a domain name.

8 Sequence

In what order. List, then read, then delete.

9 Graph

Who touches what. Resources this identity reached for the first time.



If you remember one family, remember the navy one. For stolen credentials, **novelty** features are consistently the strongest signal: a thief rarely repeats the owner's habits.

Windows: how much time does one row cover?

A single API call says little. Behaviour appears when you summarise many events over a span of time, called a **window**. There are three shapes.

Tumbling

fixed blocks, no overlap



"Calls in each clock hour." Simple and cheap.

Sliding

overlapping, always up to date



"Calls in the last 60 minutes", refreshed every few minutes.

Session

ends after a pause



"Everything until 30 idle minutes pass." Follows the work itself.

time runs left to right



Which length? Short windows catch fast, loud attacks. Long windows catch slow, quiet ones. Compute the same feature over several: 5 minutes, 1 hour, 1 day, 7 days.

Key term: aggregation is the summary applied inside a window: count, sum, mean, maximum, or number of distinct values.

Behaviour profiles: compared with what?

KEY TERM

An **entity** is the thing whose behaviour you profile: a user, a role, an access key, an IP address, a server, a bucket. Every feature belongs to an entity and a window.

One number, three yardsticks. Today Amara downloaded 400 MB. Is that unusual?

Against herself

Her own median is 120 MB a day.

Verdict: more than three times her norm. Unusual.

Feature: $\text{today} \div \text{own 30-day median} = 3.3$

Against her peers

The data-engineering team's median is 350 MB.

Verdict: ordinary for someone in her role.

Feature: $\text{today} \div \text{peer-group median} = 1.1$

Against everyone

The company-wide median is 15 MB.

Verdict: extreme, but so is her whole team.

Feature: $\text{today} \div \text{company median} = 26.7$

Give the model all three ratios and let it weigh them. A jump against herself *and* against her peers is far more alarming than either alone.

Novelty and rarity: the thief's fingerprints

Novelty: "first time seen"

Keep a memory of what each identity has done in the last 90 days: its addresses, countries, API calls, regions, tools.

For each new event, ask: is this value in the memory? If not, the feature is 1.

new_address new_country new_api_call new_region
new_user_agent

Add them up for a simple and surprisingly strong score: "how many things about this event are new for this identity?"

WORKED EXAMPLE · RARITY

Rarity score = log base 2 of (1 ÷ how often it happens). The rarer the event, the bigger the score, measured in bits.

DescribeInstances is 1 call in 3: $\log_2(3) = 1.6$ bits. Unsurprising.

StopLogging is 1 call in 1,000,000: $\log_2(1,000,000) = 19.9$ bits. Very surprising.

Why the logarithm? It turns "a million times rarer" into "about 18 bits more", a scale a model can use.

Watch out: a new starter, or a team beginning a new project, is novel in every way and perfectly innocent. Add "age of this identity in days" as a feature, and give new identities a warm-up period.

Encoding: turning words into numbers

Models do arithmetic. A column holding "us-east-1" or "AttachUserPolicy" must become numbers first. Five ways, from simplest to richest:

One-hot	One new 0-or-1 column per value: is_us_east_1, is_eu_west_2, and so on.	Use when there are a few dozen values at most.
Frequency	Replace each value with how common it is: us-east-1 becomes 0.62, a never-used region 0.0001.	The security favourite: rarity is exactly the signal.
Target	Replace each value with the share of past samples with that value that were malicious.	Powerful, but leaks the answer unless computed on past data only.
Hashing	Scramble each value into one of, say, 1,000 numbered buckets.	For columns with endless values, such as user-agent strings.
Embedding	Let a neural network learn a short list of numbers for each value, so similar values sit close together.	For deep learning on sequences of calls. Module 10.

Watch out: never number categories 1, 2, 3 in arbitrary order. The model will believe region 3 is "three times" region 1.

Scaling: putting features on a common footing

"Bytes sent" runs into the millions; "is the country new" is 0 or 1. Many algorithms would let the big numbers shout the small ones down. Scaling fixes that.

Standardise

$(x - \text{mean}) \div \text{standard deviation}$

Centres on 0. The z-score from Module 3. The usual default.

Min-max

$(x - \text{min}) \div (\text{max} - \text{min})$

Squeezes into 0 to 1. With min 10 and max 110, the value 30 becomes 0.2.

Robust

$(x - \text{median}) \div \text{interquartile range}$

Like standardising, but unmoved by outliers. Good for security data.

Log

$\log(1 + x)$

Tames heavy tails. 2,650,000 bytes becomes 6.4 in base 10. Apply before the others.

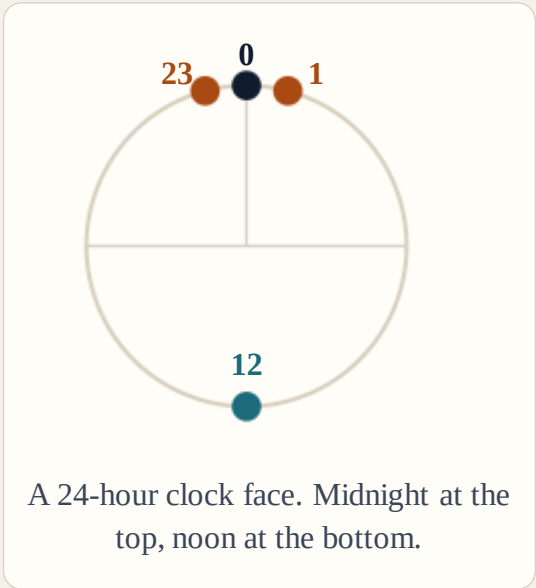
Who needs it?

Needs scaling: logistic regression, k-nearest neighbours, support vector machines, k-means, neural networks. **Does not care:** decision trees, random forests, gradient boosting.

Watch out

Work out the mean, minimum and so on from the training set only, then reuse those numbers on validation and test data. Computing them on everything lets the test set leak into training.

Time is a circle: encoding the hour



The problem. 23:00 and 01:00 are two hours apart, yet the numbers 23 and 1 are 22 apart. Fed the raw hour, a model thinks late evening and early morning are opposites.

The fix. Place each hour on a clock face and give the model its two coordinates: $\sin(2\pi \times \text{hour} \div 24)$ and $\cos(2\pi \times \text{hour} \div 24)$.

Hour	sin value	cos value	Where on the clock
23	-0.26	0.97	just left of the top
0	0.00	1.00	the top
1	0.26	0.97	just right of the top
12	0.00	-1.00	the bottom

Now 23:00 and 01:00 are 0.52 apart, while midnight and noon are 2.00 apart, as they should be. The same trick works for day of week and month of year.

Impossible travel: speed between two logins

LOGIN A

London • 09:00

the user's usual city and device

about 5,570 km

45 minutes later

LOGIN B

New York • 09:45

same account, unfamiliar device

WORKED EXAMPLE

Time = 45 minutes = 0.75 hours.

Speed = distance ÷ time = $5,570 \div 0.75 = 7,427 \text{ km/h}$.

An airliner cruises near 900 km/h. No person made this journey. Either two people hold the account, or the location data is wrong.

The feature: travel_speed_kmh. Many teams flag anything above about 1,000.

Key term: haversine distance is the shortest distance between two points over the curved surface of the Earth, computed from their latitudes and longitudes.

Watch out

VPNs, company proxies and mobile networks make honest users "teleport" daily, and address-to-city lookups are often wrong. Treat speed as one feature among many, never as a verdict.

Entropy: measuring how random a name looks

PLAIN ENGLISH

Entropy asks: if I pick one character from this name, how hard is it to guess? Names people invent repeat letters and are easy to guess. Names machines generate use every character once and are hard.

KEY TERM

Shannon entropy: for each distinct character, take its share p of the name and compute $p \times \log_2(1 \div p)$. Add the results. The unit is bits per character.

amazon

Six characters. "a" appears twice (share $2/6$); m, z, o, n once each (share $1/6$).

a: $(2/6) \times \log_2(3) = 0.53$

four others: $4 \times (1/6) \times \log_2(6) = 1.72$

Entropy = $0.53 + 1.72 = 2.25$ bits

xk7qz9vb2mfw

Twelve characters, all different (share $1/12$ each).

twelve characters: $12 \times (1/12) \times \log_2(12)$
which is simply $\log_2(12)$

Entropy = **3.58 bits**, the maximum for 12 characters

Where it is used

Spotting machine-generated domains, data smuggled out inside DNS names, and randomly named malicious resources.

Watch out: content-delivery networks and cloud services use random-looking names too. High entropy is a clue, not proof.

Order and connection: sequence and graph features

Sequence features

Counts forget order. Order is often the whole story.

ListBuckets ➔ GetObject ordinary

CreateUser ➔ AttachUserPolicy ➔ CreateAccessKey

The second chain is a recipe for a hidden back door: new user, admin rights, fresh key.

Key term: n-gram. A run of n consecutive items. Two calls in a row make a bigram; three, a trigram. Feature: how many rare n-grams appear in this session?

Graph features

Draw a line from each identity to every resource it touches. The drawing is a graph, and its shape can be measured.

Degree: how many distinct resources this identity touched today.

New edges: how many of those it had never touched before.

Peer overlap: what share of them its team-mates also use.

An engineer who suddenly reaches 40 buckets, 38 of them new and none used by their team, is exploring, not working.

Both ideas return with full methods in Module 10. Here they are simply two more columns in the feature table.

Leakage: when the model sees the answer sheet

KEY TERM

Leakage is any information in the features that would not be available at the moment of prediction, or that gives the label away. The model looks superb in testing and fails in production.

1 · Time leakage

A feature uses events from after the moment being scored.

Example: scoring a call made at 09:41 with "total calls that day". At 09:41 the day was not over.

2 · Label leakage

A feature is a consequence of the investigation itself.

Example: "account was disabled" or "ticket number is present". Only compromised accounts have these, and only afterwards.

3 · Group leakage

Pieces of one attack land in both training and test sets.

Example: 500 events from one intrusion, split at random. The model recognises the intruder's address, not the behaviour.

The symptom: results that are too good to be true. A near-perfect score on a hard security problem means "look for the leak", not "celebrate".

The cure: for every feature ask, "at the instant of this event, could I have known this number?" If not, remove it.

Building a feature table in ten lines

IN CODE · PYTHON

```
import numpy as np
df["hour"] = df["eventTime"].dt.floor("1h")
df["denied"] = df["errorCode"].notna()
feats = df.groupby(["user", "hour"]).agg(
    calls=("eventName", "size"),
    distinct_calls=("eventName", "nunique"),
    distinct_ips=("sourceIPAddress", "nunique"),
    denied_share=("denied", "mean"),
).reset_index()
feats["log_calls"] = np.log1p(feats["calls"])
```

- 1 NumPy supplies the logarithm used on the last line.
- 2 Round each timestamp down to its hour: a tumbling window. Mark each call as denied or not.
- 3 **groupby** gathers the events of one user in one hour. **agg** summarises each group four ways: a count, two diversity features and a ratio.
- 4 The mean of a true-or-false column is the share that are true: here, the share of calls denied.
- 5 **log1p** is $\log(1 + x)$: the heavy-tail fix from Module 3.

What changed: 1.28 million rows of raw API calls became a table where each row is one user-hour and each column describes behaviour. That table is what every model from here on consumes.

Checkpoint 5: exploring and features

Answer on paper first. The answers are in this slide's speaker notes.

- 1** "Number of distinct regions an identity used today" belongs to which family?
a) Counts · b) Diversity · c) Text
- 2** A window covering "the last 60 minutes", refreshed every 5 minutes, is...
a) tumbling · b) sliding · c) a session
- 3** Why encode the hour of day with a sine and a cosine?
a) So 23:00 and 01:00 end up close together · b) To save memory · c) To hide the time
- 4** Two logins by one account, 6,000 km and 2 hours apart. The implied speed is...
a) 300 km/h · b) 3,000 km/h · c) 12,000 km/h
- 5** Using "account was disabled by the SOC" to predict compromise is...
a) a novelty feature · b) label leakage · c) robust scaling

MODULE 07

Supervised learning

Teaching a model to tell malicious from benign, using examples where the answer is already known.

07

YOU WILL LEARN

Eight algorithms, from logistic regression to neural networks, each through an analogy

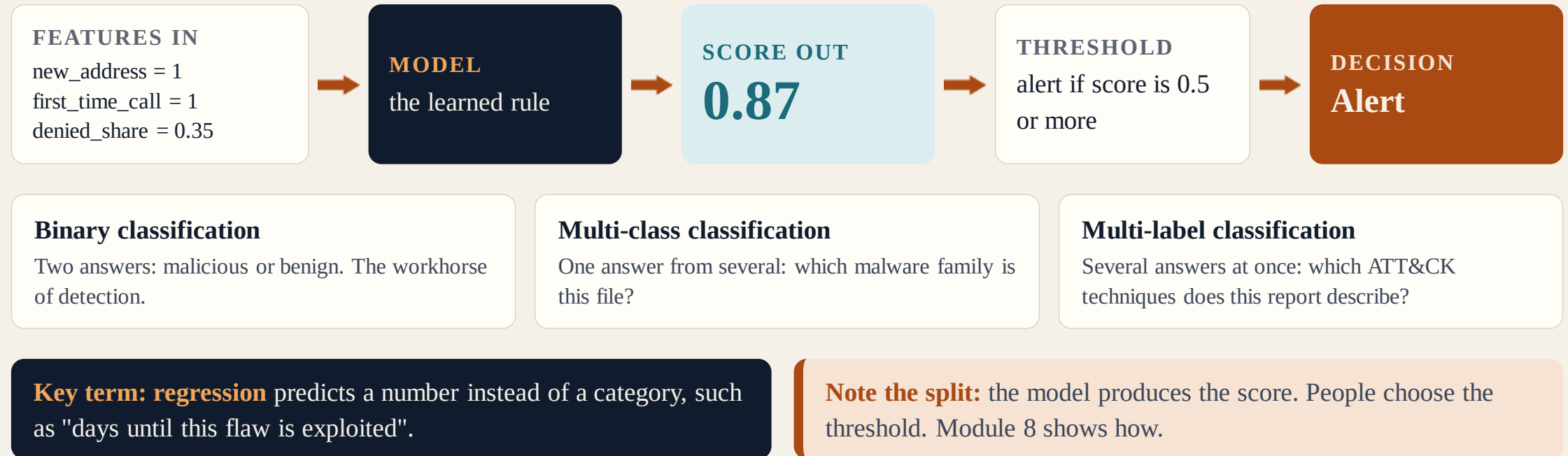
YOU WILL LEARN

How to train, tune and compare models in a few lines of Python

YOU WILL LEARN

Four classic applications: phishing, malware, generated domains and intrusions

Classification: from features to a decision



Logistic regression: a weighted checklist

PLAIN ENGLISH

An experienced analyst keeps a mental checklist: new country, plus two points; several failed logins, plus one each; second factor used, minus three. Add up the points. A high total means trouble. Logistic regression learns the points from data.

Step 1

Multiply each feature by its learned **weight**.

Step 2

Add them up, with a starting value called the **bias**.

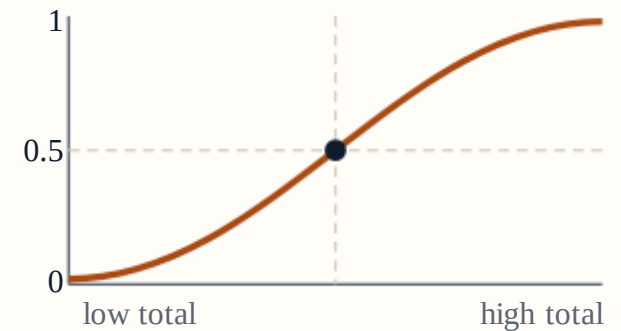
Step 3

Squash the total into a probability between 0 and 1.

Strengths: fast, and every weight can be read and explained. The best first model.

Weakness: it can only add evidence up. It cannot learn "A matters only when B is also true" unless you build that feature.

The squashing curve, called the sigmoid



A total of zero gives 0.5. Large positive totals approach 1; large negative totals approach 0.

MODULE 7 · WORKED EXAMPLE

Logistic regression by hand: scoring one login

bias
-4.0
logins start out presumed safe

weight on failed logins in the last hour
+0.4 each

weight on "country is new"
+2.0

weight on "outside usual hours"
+1.0

LOGIN A · SUSPICIOUS

6 failed logins, new country, outside usual hours.

$$\begin{aligned}\text{Total} &= -4.0 + (0.4 \times 6) + 2.0 + 1.0 \\ &= -4.0 + 2.4 + 2.0 + 1.0 = \mathbf{1.4}\end{aligned}$$

$$\begin{aligned}\text{Probability} &= 1 \div (1 + e \text{ to the power } -1.4) \\ &= 1 \div (1 + 0.247) = \mathbf{0.80}\end{aligned}$$

An 80% chance of being malicious: alert.

LOGIN B · ORDINARY

No failed logins, usual country, usual hours.

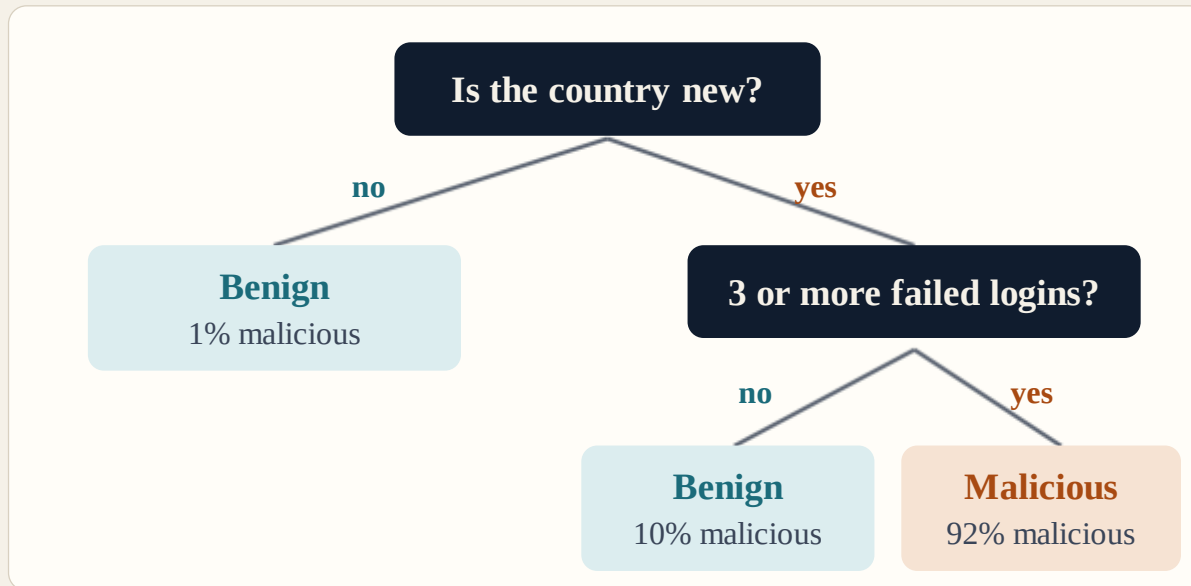
$$\text{Total} = -4.0 + 0 + 0 + 0 = \mathbf{-4.0}$$

$$\begin{aligned}\text{Probability} &= 1 \div (1 + e \text{ to the power } 4.0) \\ &= 1 \div (1 + 54.6) = \mathbf{0.018}\end{aligned}$$

Under 2%: let it pass.

Where do -4.0, 0.4, 2.0 and 1.0 come from? Training. The algorithm tries values, measures how wrong the predictions are on labelled examples, and nudges each weight to be less wrong, thousands of times.

Decision trees: a game of twenty questions



Key terms. A **node** asks a yes-or-no question about one feature. A **leaf** gives the answer. **Depth** is the number of questions on the longest path.

Strengths: anyone can read it. No scaling needed. Learns "A matters only if B" naturally, as this tree does.

Weakness: let it grow deep and it memorises the training data, one leaf per example. A lone tree overfits.

How does it choose which question to ask first? Next slide.

How a tree picks its question: Gini impurity

KEY TERM

Gini impurity measures how mixed a group is: $1 - (\text{share benign})^2 - (\text{share malicious})^2$. A pure group scores 0; a half-and-half group scores 0.5, the worst.

Before the question

100 logins: 80 benign, 20 malicious

$$1 - 0.80^2 - 0.20^2 \\ = 1 - 0.64 - 0.04$$

Gini = 0.32



Ask: "Is the country new?"

Yes: 25 logins

7 benign, 18 malicious

$$1 - 0.28^2 - 0.72^2$$

Gini = 0.403

No: 75 logins

73 benign, 2 malicious

$$1 - 0.973^2 - 0.027^2$$

Gini = 0.052

Weighted by size: $(0.25 \times 0.403) + (0.75 \times 0.052) = 0.101 + 0.039 =$
0.140



The gain

$$0.32 - 0.14$$

0.18

less mixed than before

The tree runs this sum for every feature and every possible cut-off, keeps the question with the biggest gain, then repeats inside each branch. That is the whole learning algorithm.

Random forests: the wisdom of a crowd of trees



PLAIN ENGLISH

Ask one expert and you inherit their blind spots. Ask three hundred who each studied a different part of the evidence and take a vote: individual mistakes cancel out.

Two sources of randomness make the trees differ

Random rows. Each tree trains on a random sample of the data, drawn with repeats. This is called **bagging**.

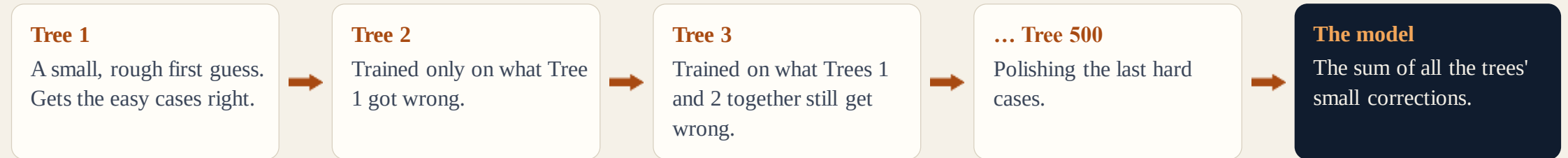
Random features. At each question a tree may only choose among a random handful of the features.

Strengths: strong with almost no tuning; shrugs off noisy features; reports which features mattered most.

Weaknesses: three hundred trees cannot be read like one; larger and slower to score.

Key term: ensemble. Any model built by combining many simpler models.

Gradient boosting: each tree fixes the last one's mistakes



PLAIN ENGLISH

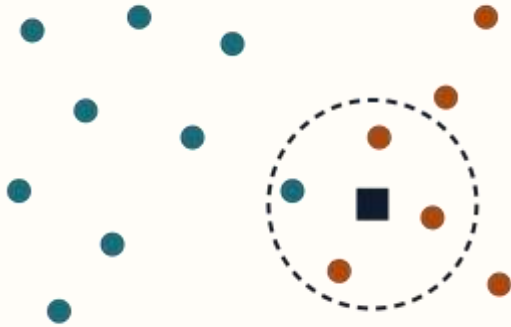
A relay of editors. The first fixes the glaring errors. Each later editor reads only the passages still wrong and corrects a little more.

	Random forest	Gradient boosting
Trees are built	independently	one after another
Each tree is	deep and strong	shallow and weak
Combined by	voting	adding corrections
Main risk	size	overfitting if run too long

Key terms. The **learning rate** sets how big each correction is. **Early stopping** halts training when validation results stop improving.

In practice: XGBoost, LightGBM and CatBoost are the usual top performers on tabular security features.

k-nearest neighbours: judged by the company you keep



Teal: past benign events. **Rust:** past malicious events. **Navy square:** the new event. Its five nearest neighbours are inside the dashed circle.

The whole algorithm. To classify a new event, find the k most similar past events and let them vote. Here k is 5: four malicious, one benign. Verdict: malicious, with a score of $4 \div 5 = 0.8$.

"Similar" means "close". Each event is a point whose coordinates are its feature values. Similarity is the straight-line distance between points, which is why features must be scaled first.

Strengths: no training step. Gives analysts "here are the five most similar past cases".

Weaknesses: slow on millions of events, and distance loses meaning with hundreds of features.

Naive Bayes: counting the evidence in words

Naive Bayes applies the rule from Module 3 to many clues at once. It asks how often each clue appears in phishing emails and how often in honest ones, then multiplies.

WORKED EXAMPLE

An email contains the words "**urgent**" and "**invoice**". From past mail we know:

	Phishing	Legitimate
Share of all email	0.20	0.80
Contains "urgent"	0.60	0.05
Contains "invoice"	0.50	0.10

Phishing score = $0.20 \times 0.60 \times 0.50 = 0.060$

Legitimate score = $0.80 \times 0.05 \times 0.10 = 0.004$

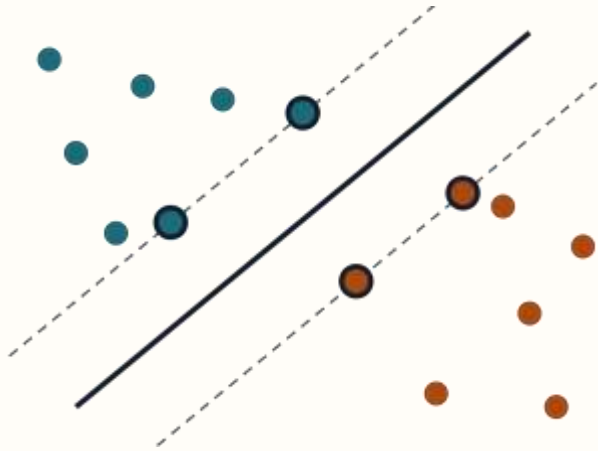
Chance of phishing = $0.060 \div (0.060 + 0.004) = \mathbf{0.94}$

Why "naive"? It assumes each word appears independently of the others. That is false, since "urgent" and "invoice" travel together, yet the method works surprisingly well.

Strengths: tiny, extremely fast, learns from few examples. A classic for spam and phishing text.

Weakness: its probabilities are overconfident, and attackers defeat it by padding messages with innocent words.

Support vector machines: the widest street



Solid line: the boundary. Dashed lines: the kerbs. Outlined points sit on the kerb and are called **support vectors**.

PLAIN ENGLISH

Many lines could separate the two groups. An SVM chooses the one that leaves the widest empty street between them, because a wide street tolerates new points that land a little off.

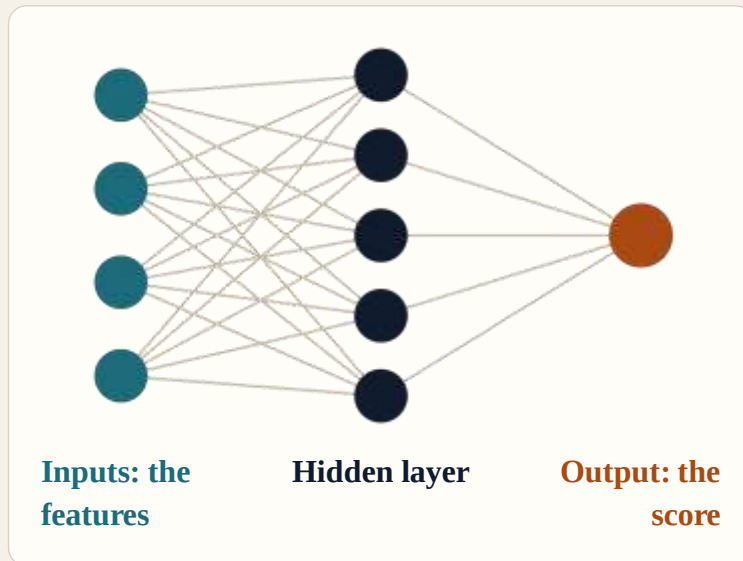
Key terms. The street's width is the **margin**. When no straight street exists, a **kernel** bends the space so that a curved boundary becomes possible.

Strengths: strong on small and medium datasets, and on text.

Weaknesses: slow on millions of rows; needs scaling; hard to explain.

A one-sided variant, the one-class SVM, returns in Module 9 for anomaly detection.

Neural networks: layers of simple units



Each circle is a small logistic regression. It weights its inputs, adds them, and squashes the result. Stack layers of them and the first layer's outputs become the second layer's features. The network invents its own features.

Key terms. **Weights** are the numbers on the lines. **Backpropagation** is how training passes the error backwards, line by line, to adjust them. An **epoch** is one pass through the training data. **Deep** learning means many hidden layers.

Strengths: learns directly from raw bytes, text and sequences.

Weaknesses: needs a lot of data, and is hard to explain to an analyst.

Five deep-learning shapes and their security jobs

Convolutional network

CNN

Slides a small pattern-detector along the input, finding local shapes wherever they occur.

Malware read as a raw sequence of bytes; phishing pages seen as screenshots.

Recurrent network

RNN, LSTM

Reads a sequence one step at a time, carrying a memory of what came before.

Chains of API calls; the characters of a suspicious domain name.

Transformer

the basis of language models

Lets every element look at every other element at once, a mechanism called attention.

Log lines, threat reports, phishing text, source code. Module 11.

Autoencoder

compress, then rebuild

Learns to squeeze normal data through a bottleneck and reconstruct it.

Anomaly detection: what it cannot rebuild is unusual. Module 9.

Graph neural network

GNN

Each node updates itself using information from its neighbours.

Identity-to-resource graphs; lateral movement. Module 10.

A rule to keep you honest: on a table of engineered features, try gradient-boosted trees first. Reach for deep learning when the input is raw, sequential or textual, and you have a great deal of it.

Choosing an algorithm: the eight side by side

Algorithm	Strength	Weakness	Reach for it when
Logistic regression	Fast; weights can be read	Only adds evidence up	You need a baseline you can explain
Decision tree	Rules a person can follow	Overfits on its own	An auditor must see the logic
Random forest	Robust with little tuning	Large; slower to score	You want a strong first model
Gradient boosting	Usually the most accurate	Needs careful tuning	Accuracy on tabular data matters most
k-nearest neighbours	Finds similar past cases	Slow at scale	Analysts ask "seen this before?"
Naive Bayes	Tiny and instant on text	Crude; overconfident	Triage of text at high volume
Support vector machine	Strong on medium data	Slow on millions of rows	A clean dataset of modest size
Neural network	Learns from raw input	Data-hungry; opaque	Sequences, text, bytes, images

A sound default path: logistic regression as the baseline, then a random forest, then gradient boosting. Stop when extra complexity stops buying accuracy.

Remember slide 73: a new feature usually helps more than a new algorithm. If results disappoint, go back to Module 6 before trying a fancier model.

Training a first classifier

IN CODE • PYTHON

```
from sklearn.ensemble import RandomForestClassifier
cols = ["log_calls", "distinct_calls",
        "distinct_ips", "denied_share"]
train = feats[feats["hour"] < "2026-03-01"]
test = feats[feats["hour"] >= "2026-03-01"]
model = RandomForestClassifier(
    n_estimators=300,
    class_weight="balanced",
    random_state=0)
model.fit(train[cols], train["label"])
scores = model.predict_proba(test[cols])[:, 1]
```

- 1 Bring in the random forest from scikit-learn.
- 2 Name the feature columns built on slide 85.
- 3 Split by time: everything before 1 March trains the model; everything after tests it.
- 4 300 trees. **class_weight** makes rare malicious rows count for more. **random_state** makes the run repeatable.
- 5 **fit** is training: features and labels go in.
- 6 **predict_proba** is inference. It returns two columns; column 1 is the probability of "malicious".

Every scikit-learn model follows the same two verbs, fit then predict. Swap one line and you have a different algorithm.

Pipelines and cross-validation

IN CODE • PYTHON

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import (
    TimeSeriesSplit, cross_val_score)

pipe = make_pipeline(
    StandardScaler(),
    LogisticRegression(class_weight="balanced"))

cv = TimeSeriesSplit(n_splits=4)
ap = cross_val_score(pipe, X, y, cv=cv,
    scoring="average_precision")
print(ap.mean(), ap.std())
```

Key term: pipeline. Scaling and model chained into one object. The scaler is then fitted on training data only, every time, so nothing leaks.

Key term: cross-validation. Train and validate several times on different slices, then average. One split can be lucky; four rarely are.

TimeSeriesSplit: four folds, always past before future



Navy: train. **Rust:** validate. Time runs left to right.

X is the feature table and y the label column. The score "average precision" suits rare attacks; Module 8 explains why.

Tuning: choosing the settings the model cannot learn

KEY TERM

A **hyperparameter** is a setting you fix before training. For boosted trees the three that matter most are the number of trees, the depth of each tree and the learning rate.

PLAIN ENGLISH

The model learns the recipe's quantities by itself. Hyperparameters are the oven temperature and the baking time: you set them, bake, taste, and adjust.

Grid search

Try every combination on a fixed grid. Thorough, and slow: 5 values each of 3 settings is 125 training runs.

Random search

Try, say, 30 combinations picked at random. Usually finds settings nearly as good for a fraction of the cost.

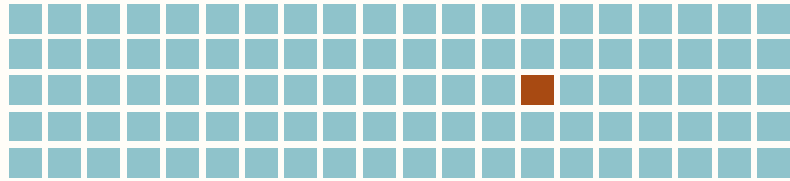
Bayesian optimisation

Use the results so far to guess where good settings lie, and try there next. Tools such as Optuna automate it.

Watch out: judge every setting on the validation data. Touch the test set during tuning and it stops being a fair exam.

Keep it in proportion: tuning typically adds a few points of performance. Better features and cleaner labels add far more.

Class imbalance: attacks are needles in haystacks



One hundred events, **one malicious**. That is 1 in 100. In real cloud logs the ratio is nearer 1 in 100,000: picture a thousand of these grids with a single rust square among them.

THE LAZIEST MODEL IN THE WORLD

"Everything is benign." On the grid it is right 99 times in 100. It is 99% accurate and it will never catch a single attack.

Three things imbalance breaks

Training. The model learns that ignoring the rare class costs it almost nothing.

Evaluation. Accuracy becomes meaningless. Module 8 replaces it.

Confidence. With 40 attacks in the test set, each one you catch or miss moves the result by 2.5 points.

Five ways to handle imbalance

1 • Class weights

Tell the model that a mistake on a malicious row costs, say, 1,000 times a mistake on a benign row.

Start here. One setting, no data thrown away.

2 • Undersampling

Keep every malicious row and only a random fraction of the benign ones.

Fast training; loses some knowledge of what "normal" looks like.

3 • Oversampling

Repeat malicious rows, or invent new ones between real ones. The best-known method is SMOTE.

Can help; can also invent attacks that never could exist.

4 • Threshold moving

Train normally, then alert at a score of 0.05 instead of 0.5.

Always needed anyway. Module 8 shows how to choose it.

5 • Change the question

With only a few dozen confirmed attacks, stop classifying. Model "normal" and flag departures from it.

This is anomaly detection: Module 9.

Watch out: rebalance the training set only. The validation and test sets must keep the real-world ratio, or every number you report will flatter the model.

Phishing: classifying a web address

A LINK SENT TO A MERIDIAN ENGINEER · INVENTED FOR ILLUSTRATION

<http://meridian-retail.com.account-verify.example/login/confirm>

Teal: a decoy that looks like the real company. Amber: the domain that is actually registered, and that the attacker owns.

Features a model can compute from the address alone

total length

number of dots

share of digits

entropy

a brand name outside the registered domain

words such as login, verify, secure

a raw IP address instead of a name

no encryption

And from a lookup: age of the domain in days, age of its certificate, reputation of the hosting network.

Model: gradient boosting on these features is a strong baseline. Character-level neural networks read the raw text and skip the feature step.

Labels: community phishing feeds for the malicious class; lists of popular sites for the benign class.

Watch out: phishing sites live for hours. Train on last year's links and test on this year's, or the score is fiction.

Malware: is this file dangerous?

Static analysis

EXAMINE THE FILE WITHOUT RUNNING IT

File size and structure. Entropy of each section (packed or encrypted code looks random). Which system functions it imports. Readable strings inside it. A histogram of its byte values.

Fast and safe. Defeated by packing and obfuscation.

Dynamic analysis

RUN IT IN A SEALED SANDBOX AND WATCH

The sequence of system calls it makes. Files it creates or encrypts. Network addresses it contacts. Processes it starts.

Sees true behaviour. Slow, and some malware detects the sandbox and behaves.

Models that work: gradient boosting on static features, as in the EMBER benchmark; convolutional networks reading raw bytes; sequence models over system calls.

In the cloud: scan files as they land in object storage, container images before they deploy, and code packages before a function runs.

Watch out for labels: antivirus vendors disagree on what a sample is. A common fix is to call a file malicious only when several engines agree.

Watch out for time: malware authors change their product weekly. A model's accuracy starts to fall the day it is deployed.

Machine-generated domains

KEY TERM

A **domain generation algorithm**, or DGA, lets malware compute hundreds of fresh domain names a day. The attacker registers one of them; the malware tries them all until one answers. A fixed blocklist can never keep up.

WRITTEN BY A PERSON

meridianretail

Pronounceable. Vowels and consonants alternate.
Entropy is low.

RANDOM-CHARACTER DGA

xkqzvbtmfw

Unpronounceable. Almost no vowels. Entropy is high. Easy to catch.

DICTIONARY DGA

tablehorsebrown

Real words glued together. Entropy looks normal.
Hard to catch.

Features from the name

Length, entropy, share of vowels, share of digits, and how likely each pair of letters is in ordinary language.

Features from behaviour

A server that looks up hundreds of names that do not exist is a stronger sign than any single name. This catches dictionary DGAs too.

Models and data

Random forests on the features above, or recurrent networks reading the characters. The data is the DNS query log.

Network intrusion detection from flow records

Two generations of detector

Signature-based. Matches traffic against patterns of known attacks. Precise, and blind to anything new.

Learning-based. A classifier trained on labelled flows, or an anomaly detector trained on normal traffic. Catches variants; raises more false alarms.

Good teams run both. The signature tool's alerts become features and labels for the model.

Attack class	The feature that gives it away
Port scan	One source, many destination ports, tiny flows
Brute force	Many short connections to one login port
Denial of service	A flood of packets to one destination
Botnet beaconing	Small flows at clockwork intervals
Exfiltration	Outbound bytes far above inbound, to a new place

A multi-class problem: the model names the type of attack, not only its presence.

Practise on: CIC-IDS2017 or UNSW-NB15 with a random forest, then gradient boosting.

Watch out: most traffic is encrypted. You have the envelope only, so sizes, timing and direction are the features.

Watch out: 99% on a lab benchmark does not transfer to your own network. Re-validate on your own traffic.

Checkpoint 6: supervised learning

Answer on paper first. The answers are in this slide's speaker notes.

- 1 In logistic regression, a large positive weight on a feature...**
a) pushes the score towards "malicious" · b) means the feature is ignored · c) means the feature is missing
- 2 A group of 10 logins holds 5 benign and 5 malicious. Its Gini impurity is...**
a) 0 · b) 0.25 · c) 0.5
- 3 Gradient boosting builds its trees...**
a) all at once, independently · b) one after another, each correcting earlier errors · c) only once
- 4 With 1 attack in 10,000 events, a model that always says "benign" has an accuracy of...**
a) 50% · b) 99.99% · c) 0.01%
- 5 Which of these needs its features scaled before training?**
a) Random forest · b) k-nearest neighbours · c) Decision tree

MODULE 08

Evaluating detectors

A model is only as good as the honesty of its test. This module is about not fooling yourself.

08

YOU WILL LEARN

Why accuracy fails, and the metrics that replace it: precision, recall and F1

YOU WILL LEARN

The base-rate fallacy: why a "99% accurate" detector can be 99% false alarms

YOU WILL LEARN

How to set a threshold from the cost of mistakes and the size of your team

99.99% accurate. Zero attacks caught.

1,000,000

events in a day

100

of them malicious

99.99%

accuracy of a "model" that calls
everything benign

0

attacks it will ever detect

Accuracy is the share of all predictions that are correct: here $999,900 \div 1,000,000$. It rewards getting the easy majority right and says nothing about the rare events we care about. We need measures that look only at the attacks and the alerts.

The confusion matrix: four kinds of outcome

	MODEL SAYS: MALICIOUS	MODEL SAYS: BENIGN
REALLY MALICIOUS	True positive A hit. The attack was caught.	False negative A miss. The attack went through unseen.
REALLY BENIGN	False positive A false alarm. An analyst's time wasted.	True negative Correct silence. The vast majority.

PLAIN ENGLISH

A smoke alarm. It rings and there is a fire: true positive. Fire and silence: false negative. It rings for burnt toast: false positive. Quiet, no fire: true negative.

How to read the names. The second word is what the model said. The first word is whether it was right. "False positive": it said positive, and that was false.

You want the two teal boxes full and the two rust boxes empty. The two errors cost different things: a miss costs a breach, a false alarm costs attention.

Precision, recall and F1

Precision

true positives ÷ all alerts raised

"When it rings, how often is there a fire?"

The analyst's metric. Low precision means a queue full of false alarms.

Recall

true positives ÷ all real attacks

"Of all the fires, how many set it off?"

The defender's metric. Low recall means attacks slipping past. Also called detection rate or sensitivity.

F1 score

$2 \times \text{precision} \times \text{recall} \div (\text{precision} + \text{recall})$

One number that is high only when both are high.

Handy for comparing models. It hides which of the two is weak, so always report all three.

The two pull against each other. Alert on everything and recall is 100% while precision collapses. Alert only when certain and precision soars while recall falls. The threshold sets the balance.

One more: the **false positive rate** is false positives ÷ all benign events. It looks tiny and is the number that decides your alert volume.

MODULE 8 · WORKED EXAMPLE

Every metric from one confusion matrix

A detector is tested on 10,000 events, of which 200 are malicious.

	Said malicious	Said benign
Really malicious (200)	160 true positives	40 false negatives
Really benign (9,800)	240 false positives	9,560 true negatives

THE SUMS

$$\text{Accuracy} = (160 + 9,560) \div 10,000 = 97.2\%$$

$$\text{Precision} = 160 \div (160 + 240) = 160 \div 400 = 40\%$$

$$\text{Recall} = 160 \div (160 + 40) = 160 \div 200 = 80\%$$

$$\text{F1} = 2 \times 0.40 \times 0.80 \div (0.40 + 0.80) = 0.64 \div 1.20 = 0.53$$

$$\text{False positive rate} = 240 \div 9,800 = 2.4\%$$

Read it aloud: "It catches four attacks in five. Three alerts in five are false alarms." The 97.2% accuracy told you neither.

MODULE 8 · THE MOST IMPORTANT SLIDE IN THIS MODULE

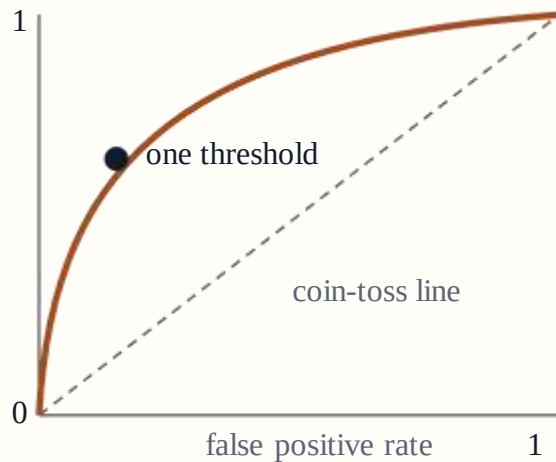
The base-rate fallacy at cloud scale

A vendor says: "Our detector catches 99% of attacks with only a 1% false positive rate." It sounds superb. Run it on one day of Meridian's logs.



This effect was set out for intrusion detection by Stefan Axelsson in 2000. It is why a false positive rate that sounds small is rarely small enough.

The ROC curve and the area under it



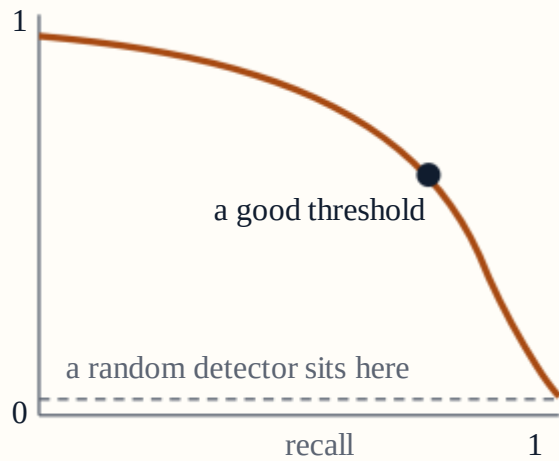
Vertical axis: recall, the share of attacks caught.
Each point on the curve is one choice of threshold.

Key terms. The **ROC curve** shows recall against false positive rate as the threshold slides from strict to lax. **AUC**, the area under it, is a single score: 0.5 is a coin toss, 1.0 is perfect.

What AUC means in words. Pick one malicious event and one benign event at random. AUC is the chance the model gives the malicious one the higher score.

Watch out. ROC flatters detectors of rare events. On the axis, a false positive rate of 1% is a sliver next to the corner; on the previous slide it was 9,999 false alerts. An AUC of 0.99 can still mean an unusable alert queue.

The precision-recall curve: the honest picture



Vertical axis: precision. Moving right, you catch more attacks and pay for it in false alarms.

Key terms. The **precision-recall curve** shows precision against recall across all thresholds. The area under it is called **average precision**.

Why it is more honest. Neither axis uses the true negatives, the mountain of correctly ignored benign events that makes every other metric look good. The curve shows only what analysts will live with.

The baseline moves. A random detector's precision equals the share of attacks in the data. At 1 in 10,000 that is 0.0001, so an average precision of 0.30 is three thousand times better than chance, and still far from finished.

MODULE 8 • WORKED EXAMPLE

Choosing a threshold by what mistakes cost

A missed attack costs Meridian about £50,000 on average.

A false alarm costs about £25: fifteen minutes of an analyst.

Alert when score is at least	Missed attacks	False alarms	Cost of the misses	Cost of the alarms
0.50 · strict	40	240	$40 \times £50,000 = £2,000,000$	$240 \times £25 = £6,000$
0.10 · moderate	10	1,500	$10 \times £50,000 = £500,000$	$1,500 \times £25 = £37,500$
0.01 · lax	4	12,000	$4 \times £50,000 = £200,000$	$12,000 \times £25 = £300,000$

Strict, total
£2,006,000

Moderate, total
£537,500

Lax, total
£500,000

On paper the lax threshold wins by £37,500. But 12,000 alarms is more than four analysts can open.

Unread alerts are misses. Past the team's capacity the sums stop holding. The moderate threshold is the real optimum.

Alert budgets: evaluate what the team can read

Start from capacity. Meridian has four analysts. Each can investigate about 15 alerts a day. The budget is **60 alerts a day**, whatever the model says.

Key terms. Sort the day's events by score and take the top k . **Precision at k** is the share of those k that are real. **Recall at k** is the share of all the day's attacks that made it into the k .

Worked example

Today held 20 real attacks. The top 60 scores contain 12 of them.

Precision at 60 = $12 \div 60 = 20\%$. Recall at 60 = $12 \div 20 = 60\%$.

Spend the budget in tiers

Top 5 scores

Page the on-call analyst immediately.

Next 55

Into the queue, to be worked today.

Next few hundred

No alert. Kept as leads for weekly threat hunting.

Everything else

Scored and stored, ready if an investigation needs it.

A checklist for an honest evaluation

✓ **Split by time.** Train on the past, test on what came after.

✓ **Keep each incident whole.** All events of one attack go on the same side of the split.

✓ **Compute features as of the moment.** No feature may use anything stamped later than its event.

✓ **Keep the real ratio in the test set.** Never rebalance the data you report on.

✓ **Tune on validation only.** Open the test set once, at the end.

✓ **Report precision and recall at your alert budget,** not accuracy and not AUC alone.

✓ **Compare with a simple baseline.** If a three-line rule does as well, ship the rule.

✓ **Read the errors.** Open twenty false alarms and twenty misses. They name your next features.

KEY TERM

Backtesting replays a past period day by day as if the model had been live, and counts what the team would have seen each morning. It is the closest thing to a rehearsal.

Beyond the model: the numbers the business feels

A model earns its place only if it moves an operational measure. Six worth tracking before and after deployment:

Mean time to detect

From the attacker's first action to the first alert.
Hours, ideally minutes.

Mean time to respond

From the alert to the attacker being shut out.

Dwell time

How long an intruder was inside in total. The sum of the two above.

Alerts per analyst per day

Above the team's capacity, quality collapses.

Share of alerts closed as real

Precision as the analysts experience it, week by week.

Technique coverage

How many relevant ATT&CK techniques at least one detector can see.

At Meridian: the breach on slide 18 ran for 78 minutes with no alert at all. A detector that fired at the 09:24 discovery burst would have cut the time to detect to five minutes after first use.

Checkpoint 7: evaluating detectors

Answer on paper first. The answers are in this slide's speaker notes.

- 1 Precision answers which question?**
a) Of all attacks, how many did we catch? · b) When we alert, how often are we right? · c) How many events are benign?
- 2 30 true positives and 10 false negatives. Recall is...**
a) 25% · b) 75% · c) 30%
- 3 30 true positives and 90 false positives. Precision is...**
a) 25% · b) 33% · c) 75%
- 4 Lowering the alert threshold usually...**
a) raises recall and lowers precision · b) raises both · c) lowers both
- 5 2,000,000 benign events a day and a false positive rate of 0.1%. False alarms per day?**
a) 20 · b) 200 · c) 2,000

MODULE 09

Anomaly detection

Finding attacks that nobody has labelled, by learning thoroughly what normal looks like.

09

YOU WILL LEARN

The three kinds of anomaly, and why context decides which is which

YOU WILL LEARN

Seven methods, from a z-score to an autoencoder, with one worked in code

YOU WILL LEARN

Why "unusual" is not "malicious", and how to close that gap

Changing the question

SUPERVISED LEARNING ASKS

"Does this look like an attack we have seen before?"

Needs labelled attacks. Precise on the known. Blind to the new.



ANOMALY DETECTION ASKS

"Does this look like us?"

Needs only normal data, which is plentiful. Can catch what nobody has named. Noisier.

PLAIN ENGLISH

Bank tellers are not shown every forgery ever made. They handle so many real notes that a fake simply feels wrong. Anomaly detection trains that feel.

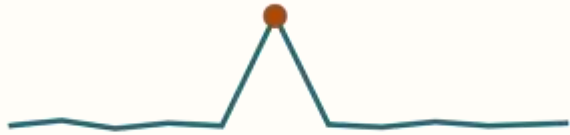
Three reasons security needs it

Labels are scarce. A few incidents a year cannot train a classifier.

Attackers innovate. A brand-new technique has no label by definition.

Every organisation is different. Your "normal" exists only in your own logs.

Three kinds of anomaly



Point anomaly

One value is extreme all by itself.

Example: an engineer who downloads 40 GB in an hour when nobody ever exceeds 2 GB.

Found by: any method in this module.



Contextual anomaly

An ordinary value in the wrong context.

Example: 200 API calls at 3 a.m. Routine for the backup service; strange for an accountant.

Found by: per-entity and per-hour baselines.



Collective anomaly

Each event is normal. The pattern is not.

Example: 300 small file reads, each ordinary, that together cover every customer record.

Found by: windows, sequences and graphs.

Skilled attackers avoid point anomalies. They work "low and slow", keeping each action small. Most of the craft in this module is about catching the second and third kinds.

Statistical baselines: start simple

The z-score

$(\text{value} - \text{mean}) \div \text{standard deviation}$. Simple, but the outlier distorts its own yardstick, as slide 70 showed.

The robust z-score

$0.6745 \times (\text{value} - \text{median}) \div \text{MAD}$, where MAD is the median of the absolute distances from the median. Extremes cannot bend it.

A moving baseline

Compare today with a running average that slowly forgets the past, so the baseline follows gradual change. Worked in Module 10.

WORKED EXAMPLE · ROBUST Z

The eleven days from slide 70: 150, 170, 180, 190, 210, **220**, 240, 250, 260, 280, 950.

Median = 220.

Distances from 220: 70, 50, 40, 30, 10, 0, 20, 30, 40, 60, 730. Sorted, the middle one is 40.

MAD = 40.

Robust z of 950 = $0.6745 \times (950 - 220) \div 40 = 0.6745 \times 730 \div 40 = \mathbf{12.3}$

The ordinary z-score gave 3.1, a borderline case. The robust score says 12.3: unmistakable.

Do this first. One robust baseline per identity per feature is cheap, explainable in a sentence, and catches a large share of loud attacks. Fancier models must beat it to earn their keep.

k-means clustering: finding the natural groups



Each dot is an identity, placed by its behaviour. **Crosses** are cluster centres. The **rust dot** belongs to no group.

Step 1. Choose k , the number of groups, and drop k centres at random.

Step 2. Assign every point to its nearest centre.

Step 3. Move each centre to the average position of its points.

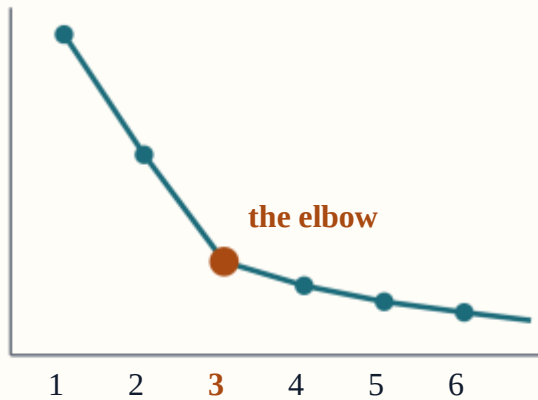
Step 4. Repeat steps 2 and 3 until nothing moves.

AT MERIDIAN

Clustering the 212 identities finds three groups no one had written down: engineers, automated deployment pipelines and scheduled jobs. These become the peer groups of slide 76.

As an anomaly detector: the score is the distance from a point to its nearest centre. The rust dot is far from all three, so it scores highest.

How many clusters? The elbow and the silhouette



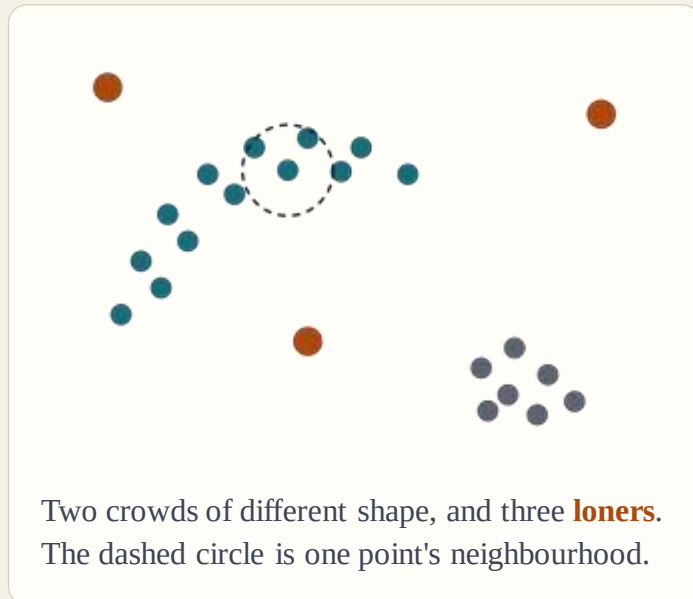
Across: number of clusters k . Up: how spread out points are around their centres. Lower is tighter.

The elbow method. More clusters always fit tighter, so plot tightness against k and look for the bend where extra clusters stop helping much. Here the bend is at 3.

Key term: silhouette score. For each point, compare its distance to its own cluster with its distance to the next nearest one. Near 1: well placed. Near 0: on a border. Negative: probably in the wrong cluster. Pick the k with the highest average.

Watch out. k -means assumes round groups of similar size and needs scaled features. Always open each cluster and read who is in it. A cluster you cannot name is a cluster you should not trust.

DBSCAN: clusters are crowds, outliers stand alone



The idea. Draw a small circle round every point. A point with enough neighbours inside its circle is in a crowd. Crowds that touch are one cluster. A point in no crowd is labelled **noise**: an outlier, found for free.

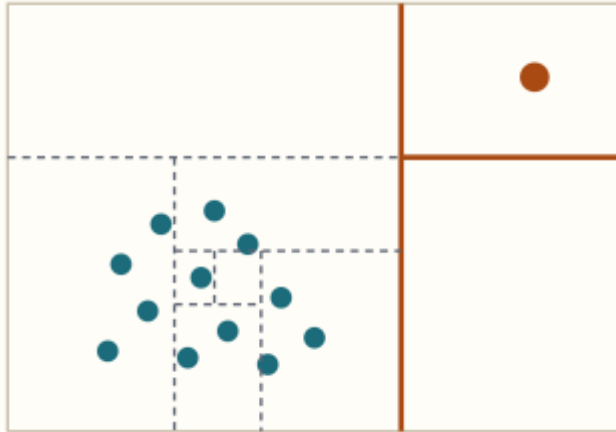
Two settings. The circle's radius, called **eps**, and the number of neighbours that counts as a crowd, called **min_samples**. There is no *k* to choose.

Strengths: finds clusters of any shape; labels outliers itself; no need to guess the number of groups.

Weakness: one radius must suit every crowd. When densities differ a lot, use its successor, HDBSCAN.

A common security use: grouping thousands of similar alerts into a few dozen clusters, so analysts handle a cluster at a time.

Isolation Forest: oddities are easy to fence off



Two solid cuts leave the rust point alone in its box. Fencing off one teal point inside the crowd takes **six dashed cuts**.

PLAIN ENGLISH

Play "guess who" with random questions. A person in a huge purple hat is singled out in two questions. An ordinary face takes ten. Anomalies are few and different, so they are quick to isolate.

The algorithm. Pick a feature at random and cut it at a random value. Repeat inside each half until every point is alone. Count the cuts each point needed. Do this for a few hundred random trees and average. **Fewer cuts means more anomalous.**

Why it is the usual first choice: fast on millions of rows, no distances to compute, no scaling needed, and only one or two settings. Introduced by Liu, Ting and Zhou in 2008.

An anomaly detector in eight lines

IN CODE • PYTHON

```
from sklearn.ensemble import IsolationForest
cols = ["log_calls", "distinct_calls",
        "distinct_ips", "denied_share"]
iso = IsolationForest(n_estimators=200,
                      contamination=0.001, random_state=0)
iso.fit(train[cols])
test["score"] = -iso.score_samples(test[cols])
top = test.sort_values("score",
                      ascending=False).head(60)
```

- 1 The same feature columns as the classifier on slide 101.
- 2 200 random trees. **contamination** is your guess at the share of anomalies: here one user-hour in a thousand.
- 3 **fit** takes features only. Notice what is missing: there is no label column anywhere.
- 4 **score_samples** returns high values for normal points, so the minus sign flips it: now high means strange.
- 5 Sort by strangeness and keep the top 60: the alert budget from slide 120.

Watch out: if an intruder was already active during the training period, the model learns their behaviour as normal. Train on a period you have reason to believe was clean.

Two neighbours: LOF and the one-class SVM

Local Outlier Factor

COMPARED WITH YOUR OWN NEIGHBOURHOOD

For each point, measure how crowded its surroundings are, then compare with how crowded its neighbours' surroundings are.

A score near 1 means "as crowded as my neighbours": normal. A score well above 1 means "much lonelier than my neighbours": an outlier.

Best when normal behaviour comes in groups of different tightness: tightly scheduled services beside loosely behaved humans. A single global rule fails there; a local comparison works.

One-class SVM

A FENCE AROUND NORMAL

Shown only normal data, it draws the tightest smooth boundary that encloses nearly all of it.

Anything landing outside the fence is flagged. A setting called **nu** states roughly what share of the training data may be left outside.

Best when you have a clean, modest-sized sample of normal behaviour. Needs scaled features and slows down on millions of rows.



No single detector wins everywhere. Each has a different idea of "strange". Running two or three and combining their rankings is more reliable than trusting one. Slide 137 shows how.

PCA: compress, rebuild, and measure the damage

1 • Learn

From normal data, find the few combined directions along which behaviour really varies.



2 • Compress

Describe each event using only those few directions: 40 features become 5 numbers.



3 • Rebuild

Expand the 5 numbers back into 40 features.



4 • Compare

The gap between original and rebuilt is the anomaly score.

PLAIN ENGLISH

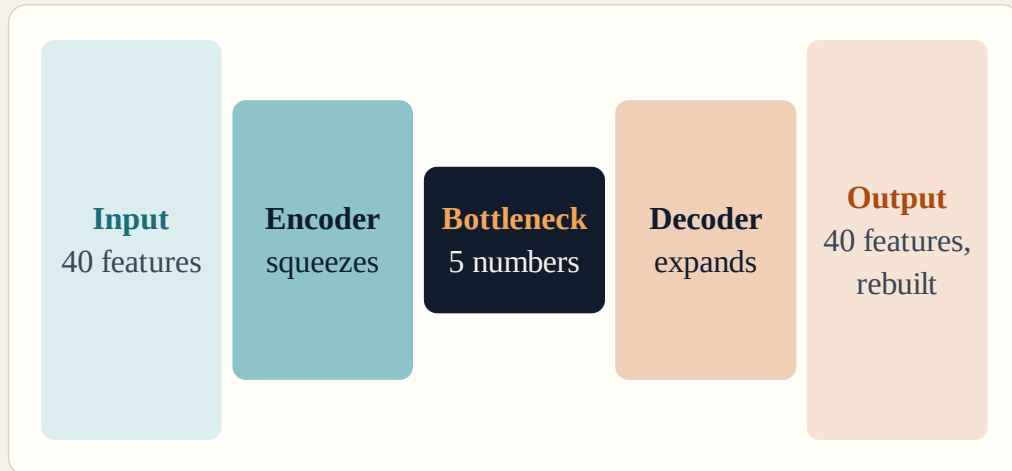
A caricaturist who has drawn a thousand ordinary faces can capture a new one in five strokes. Show them a truly unusual face and the five strokes come out wrong. How wrong is the score.

KEY TERMS

Principal component analysis, PCA, finds those main directions. Each is a **principal component**. Describing data with fewer numbers is **dimensionality reduction**. The gap after rebuilding is the **reconstruction error**.

A second use: squeeze 40 features down to 2 and you can plot every identity on a flat chart and simply look for the stragglers.

Autoencoders: the same idea with a neural network



Training. Feed it normal events only, and reward it for making the output match the input. The narrow middle forces it to learn what normal events have in common.

Scoring. An event unlike anything in training cannot pass through the bottleneck intact. It comes out distorted, and the size of the distortion is its anomaly score.

Over PCA: it can learn curved, tangled patterns, and versions exist for sequences of events.

The price: it needs far more data and tuning, and "the reconstruction was poor" is a thin explanation for an analyst.

Watch out: give it too wide a bottleneck and it learns to copy anything, attacks included.

Behaviour analytics: profiling every identity

KEY TERM

User and entity behaviour analytics, UEBA, is the practice of learning a baseline for every user and machine, scoring each new action against it, and adding the scores into a running risk level per identity.

1 • Entities

List who and what you will profile: users, roles, keys, servers.

2 • Baselines

Learn each one's habits from 30 to 90 days of history.

3 • Peer groups

Group similar identities, by team or by clustering.

4 • Score

Rate each new action against the identity's own past and its peers.

5 • Accumulate

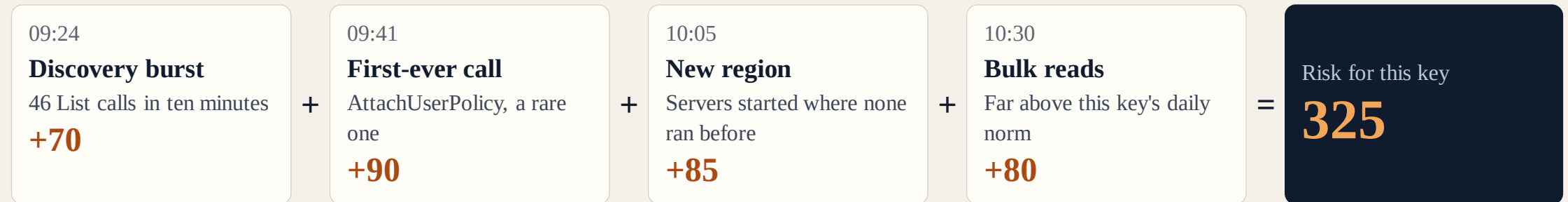
Add the scores into one risk level per identity that fades over time.

Why it suits the cloud: identity is the perimeter (Module 1), so the identity is the natural thing to watch. One profile follows a user across every service and region.

Watch out: profile machine identities too. Service accounts and access keys outnumber people, never take holidays, and are what intruders steal most.

From many weak signals to one strong alert

Replay the breach from slide 18. Four detectors each notice something about Amara's access key. None is sure enough to alert alone; the threshold for a page is a risk of 150.



The running total crosses 150 at 09:41, after the second signal: $70 + 90 = 160$. That is 22 minutes after the key was first used, and before any servers were hijacked or data taken.

The principle: alert on identities and their stories, not on single events. The analyst receives one case with four pieces of evidence instead of four puzzling alerts.

Unusual is not the same as malicious

Innocent things that look like attacks

a new starter's first week a product launch

a migration to a new region month-end reports

an engineer working abroad an emergency fix at 2 a.m.

the security team's own investigation

An anomaly detector with no context will fire on every one of these. Most anomaly projects fail here, not in the mathematics.

Four ways to close the gap

Add context as features. Deployment calendar, change tickets, joiner and leaver dates, travel bookings.

Weight by danger. A strange read of a public file matters less than a strange change to permissions.

Require agreement. Alert when several independent signals point at one identity, as on the previous slide.

Learn from the analysts. Every closed alert is a label. Train a classifier on those labels to re-rank the anomalies. Asking people about the cases the model is least sure of is called **active learning**.

The mature design is a pair: an anomaly detector to propose, and a supervised model trained on analyst feedback to dispose.

Checkpoint 8: anomaly detection

Answer on paper first. The answers are in this slide's speaker notes.

1 A 3 a.m. burst of calls that is routine for a backup job but odd for an accountant is...
a) a point anomaly · b) a contextual anomaly · c) a collective anomaly

2 Isolation Forest gives a high anomaly score to points that...
a) are fenced off in few random cuts · b) have the most neighbours · c) sit nearest a cluster centre

3 Median 100, MAD 20, today's value 300. The robust z-score is about...
a) 2.0 · b) 6.7 · c) 10.0

4 An autoencoder flags an event when...
a) its reconstruction error is high · b) it compresses well · c) its label is "malicious"

5 The detector fires on every deployment day. The best response is to...
a) switch it off · b) feed it the deployment calendar as context · c) lower its threshold

MODULE 10

Time, sequences and graphs

Attacks unfold over time, in a particular order, across connected things. Three families of method respect that.

10

YOU WILL LEARN

How to forecast a baseline and flag departures from it, worked by hand

YOU WILL LEARN

How to catch clockwork beacons, sudden shifts and improbable chains of actions

YOU WILL LEARN

How to treat the cloud as a graph and find the paths an attacker could walk

Anatomy of a time series

KEY TERM

A **time series** is one quantity measured at regular intervals and kept in order: API calls per hour, bytes out per minute, logins per day.



Trend

The slow drift. The company grows; traffic creeps up month by month.

+



Seasonality

The repeating rhythm. Office hours, weekends, the month-end billing run.

+



Noise

Random wobble that no model can or should explain.

=

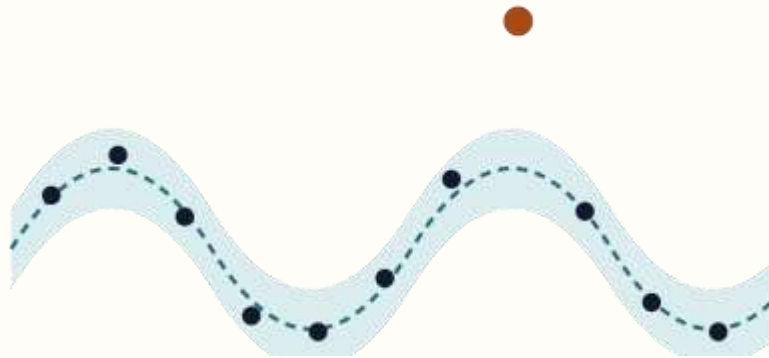


What you observe

All three at once, and occasionally an event that fits none of them.

The whole strategy of this module in one line: explain away the trend and the seasonality, estimate the size of the noise, and whatever is left over that is too big to be noise is your anomaly.

Forecast, then compare



Dashed line: the forecast. **Shaded band:** the range noise should stay within. **Navy dots:** what happened. **Rust dot:** outside the band.

1 • Forecast what the next value should be, from history.

2 • Observe what it actually is.

3 • Flag if the gap is larger than noise allows.

Ways to forecast, simplest first

Same hour last week. Crude and often hard to beat.

Weighted moving average. Recent values count more. Next slide.

Seasonal models. Holt-Winters, ARIMA and Prophet model trend and rhythm explicitly.

Learned models. Gradient boosting or neural networks fed with past values as features.

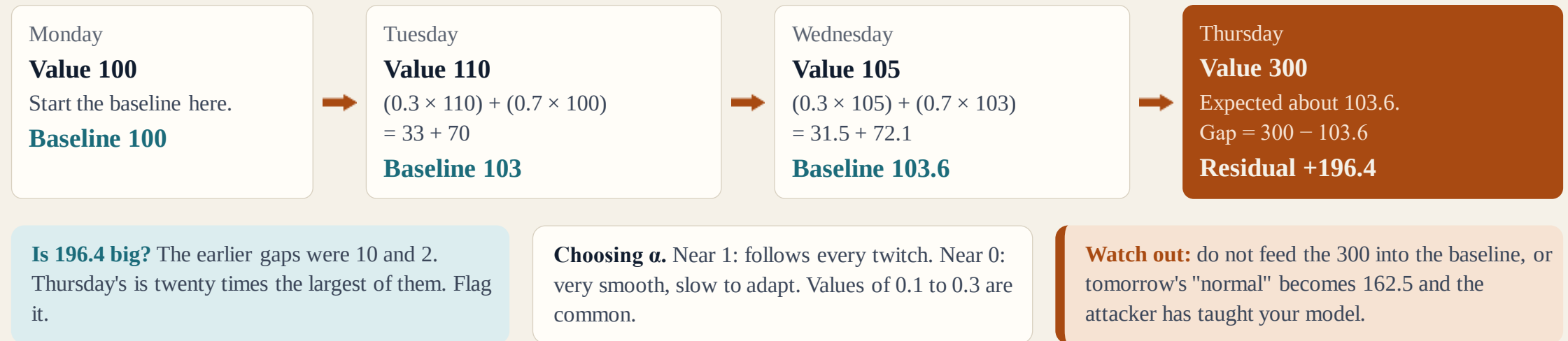
Key term: the gap between forecast and reality is the **residual**. The shaded band is a **prediction interval**.

A moving baseline by hand

KEY TERM

An **exponentially weighted moving average**, EWMA, updates a baseline with each new value: $\text{new baseline} = (\alpha \times \text{new value}) + ((1 - \alpha) \times \text{old baseline})$. The setting α , between 0 and 1, is how fast it forgets.

A service's daily outbound data in gigabytes, with $\alpha = 0.3$:



Beaconing: when traffic is too regular to be human

KEY TERM

A **beacon** is a compromised machine checking in with its controller at fixed intervals to ask for instructions. The controller is called a command-and-control, or C2, server.

A person browsing



Seconds between connections: 5, 140, 32, 600, 12

Mean = 157.8. Standard deviation = 226.

$$226 \div 157.8 = \mathbf{1.43}$$

Malware calling home



Seconds between connections: 60, 61, 59, 60, 60

Mean = 60. Standard deviation = 0.63.

$$0.63 \div 60 = \mathbf{0.011}$$

The feature: standard deviation $\div$ mean of the gaps, called the **coefficient of variation**. Near 1 or above: irregular, human. Near 0: clockwork.

Strengthen it with: identical byte counts each time, persistence over days, and a destination almost no other machine contacts.

Watch out, twice. Attackers add random delay, called jitter, to blur the rhythm. And honest software beacons too: update checks, monitoring agents, time synchronisation. Allow-list the known ones.

Change-point detection: catching the slow leak

A careful thief raises their daily take by a little, never enough to trip a z-score. **CUSUM**, the cumulative sum method, adds up small excesses until they can no longer be coincidence.

Set-up. Normal level 100 GB a day. Ignore the first 5 above that as slack. Raise the alarm when the running total passes 20.

Day	Value	Excess = value – 105	Running total
1	104	–1	0
2	108	+3	3
3	112	+7	10
4	109	+4	14
5	115	+10	24 • alarm
6	111	+6	30

The running total never goes below zero: day 1's –1 resets it to 0.

Alarm on day 5.

No single day was more than 15% above normal. If the usual day-to-day spread is 10 GB, the largest z-score here was 1.5: invisible to the methods of Module 9.

Key term: change point. The moment a series shifts to a new level and stays there. Point-anomaly methods look for spikes; change-point methods look for new plateaus.

Honest causes too: a new product launch shifts the level for good. A change point is a question to ask, not a verdict.

Markov chains: how likely is this order of events?

KEY TERM

A **Markov chain** is a table saying, for each action, how likely each possible next action is. It assumes the next step depends only on the current one.

Learn the table by counting. In Meridian's history, ListBuckets was followed by another call 10,000 times:

After ListBuckets came	Times	Probability
GetObject	7,000	0.70
ListBuckets again	2,000	0.20
PutObject	990	0.099
DeleteBucket	10	0.001

An ordinary session

ListBuckets, GetObject, GetObject

$$0.70 \times 0.80 = \mathbf{0.56}$$

A destructive session

ListBuckets, DeleteBucket, DeleteBucket

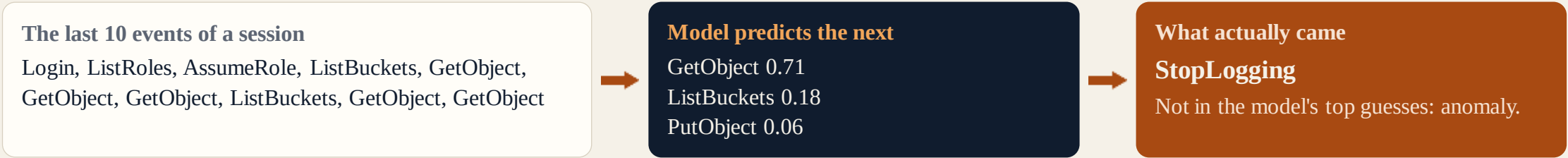
$$0.001 \times 0.01 = \mathbf{0.00001}$$

The second factors, 0.80 and 0.01, come from the rows of the table for GetObject and DeleteBucket. Multiply along the chain.

The feature: the probability of the session's sequence. The second session is 56,000 times less likely than the first, even though every call in it is a legitimate API call that the identity is allowed to make.

When one step of memory is not enough

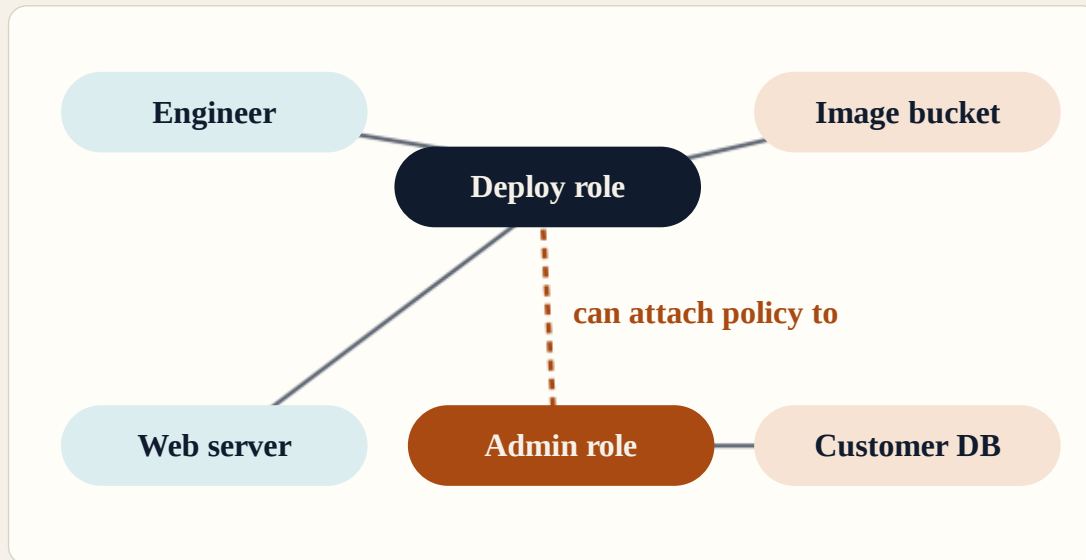
A Markov chain remembers only the last action. Real attack chains span dozens of steps. Deep sequence models carry a longer memory and work on the same principle: predict the next event, and be surprised when you are wrong.



Model	Memory	Needs	Explaining its alerts
Markov chain	One step	Counts only; trains in seconds	Easy: "this pair is rare"
LSTM network	Tens of steps	Many sessions; hours of training	Moderate
Transformer	Hundreds of steps	Very many sessions; heavy compute	Hard

The next-event approach was popularised for system logs by DeepLog (Du and colleagues, 2017). Start with the Markov chain; move down the table only if it misses attacks you care about.

The cloud is a graph



Key terms. A **graph** is a set of things, called **nodes**, joined by relationships, called **edges**. A **path** is a route along edges from one node to another.

Two graphs worth building. The **permission graph**: what each identity *could* do, from policies. The **activity graph**: what each identity *did* do, from logs. The gap between them is unused, risky privilege.

Follow the dashed edge. No list of permissions says "the web server can read the customer database". The graph does: server, deploy role, admin role, database. Three hops.

A saying among defenders: those who think in lists lose to attackers who think in graphs. This part of the module is about thinking in graphs.

Measuring a graph, and finding attack paths

Degree

How many edges a node has. An identity touching 400 resources stands out.

Shortest path

The fewest hops from one node to another. Short paths to valuable data are dangerous.

Betweenness

How many shortest paths pass through a node. High scorers are the bridges everything depends on.

Blast radius

Everything reachable from a node. If this identity is stolen, this is what the thief gets.

AT MERIDIAN • AN ATTACK PATH THE GRAPH REVEALS

Internet-facing web server



runs as the deploy role



which can grant itself admin



which reads the customer database

Three hops from the internet to the crown jewels. The middle box has the highest betweenness in the account: fix that one permission and the path is cut.

Preventive use: list every path from an internet-facing node to sensitive data and rank the edges that appear on the most paths.

Detective use: blast radius becomes a feature. An anomaly on an identity that can reach everything deserves a higher score.

Machine learning on graphs: four ideas

Community detection

Finds groups of nodes that connect mostly to each other: a team and its resources.

Security use: an edge that suddenly bridges two communities that never mixed is a candidate for lateral movement.

Link prediction

Learns which connections are likely, given everyone's past connections.

Security use: when a connection appears that the model rated nearly impossible, such as a payroll clerk reaching a build server, flag it.

Node embeddings

Turns each node into a short list of numbers so that well-connected neighbours get similar lists.

Security use: graph position becomes ordinary feature columns for any classifier or anomaly detector from Modules 7 and 9.

Guilt by association

Spreads a known label outward along edges, weakening with each hop.

Security use: one address is confirmed malicious; the domains it hosts and the machines that called it inherit some suspicion.



Where to start: NetworkX in Python handles graphs of a few million edges on a laptop. Graph neural networks are the heavy machinery; reach for them last.

Checkpoint 9: time, sequences and graphs

Answer on paper first. The answers are in this slide's speaker notes.

- 1** The daily office-hours rhythm in API calls is an example of...
a) trend · b) seasonality · c) noise
- 2** Baseline 100, new value 140, $\alpha = 0.5$. The updated EWMA baseline is...
a) 110 · b) 120 · c) 140
- 3** Gaps between connections have a coefficient of variation of 0.02. That suggests...
a) a person browsing · b) automated beaconing · c) nothing at all
- 4** Which method is built to catch a small, sustained rise that no single day reveals?
a) The z-score · b) CUSUM · c) One-hot encoding
- 5** "Everything a thief could reach if this identity were stolen" is its...
a) degree · b) blast radius · c) entropy

MODULE 11

Language models for threat intelligence

Most intelligence arrives as prose. This module turns reports and advisories into data.

11

YOU WILL LEARN

How text becomes numbers: word weights and embeddings

YOU WILL LEARN

How to pull indicators, actors and ATT&CK techniques out of a report

YOU WILL LEARN

Where large language models help, and where they must be checked

Threat intelligence is mostly written in sentences

Where the words come from

vendor threat reports government advisories vulnerability descriptions

security blogs social media posts criminal forums

your own incident tickets your analysts' notes

Thousands of pages a week. A team of four reads a handful. The two rust tags are the most valuable and the most neglected: nobody else has them.

Four jobs for a machine that reads

Extract

Pull out addresses, hashes, actor names, malware names, techniques.

Classify

Which technique is this? Does it concern our sector and our software?

Link

Are these three reports describing the same campaign?

Summarise

Forty pages into five lines: what happened, why it matters, what to do.

KEY TERM

Natural language processing. NLP, is the branch of machine learning that works on human language. It is step 3, "processing", of the intelligence cycle on slide 31, done by machine.

From words to numbers: TF-IDF

1 · Tokenise

Split the text into words, called tokens.



2 · Count

How often does each word appear in this document?



3 · Weight

Mark down words that appear in every document.



4 · A row of numbers

One column per word. Ready for any model in Module 7.

KEY TERM

TF-IDF = term frequency × inverse document frequency.

TF: the word's share of this document.

IDF: the natural log of (all documents ÷ documents containing the word). Rare words score high; a word in every document scores zero.

WORKED EXAMPLE

A 100-word report, in a library of 1,000 reports.

"**mimikatz**" appears 3 times here, and in 10 reports overall.

$TF = 3 \div 100 = 0.03$. $IDF = \ln(1,000 \div 10) = \ln(100) = 4.61$.

$TF-IDF = 0.03 \times 4.61 = \mathbf{0.138}$

"**the**" appears 7 times here, and in all 1,000 reports.

$TF = 0.07$. $IDF = \ln(1,000 \div 1,000) = \ln(1) = 0$.

$TF-IDF = 0.07 \times 0 = \mathbf{0}$

The name of a credential-stealing tool tells you what the report is about; "the" tells you nothing. TF-IDF makes that judgement automatically.

Extracting indicators from a report

IN CODE • PYTHON

```
import re

text = text.replace("[.]", ".")
text = text.replace("hxxp", "http")

ips = re.findall(
    r"\b(?:\d{1,3}\.){3}\d{1,3}\b", text)

cves = re.findall(r"CVE-\d{4}-\d{4,}", text)

sha256 = re.findall(r"\b[a-fA-F0-9]{64}\b", text)
```

re is Python's pattern-matching library. Each **findall** returns every piece of text that fits a pattern: four numbers joined by dots; "CVE", a year and a number; 64 characters drawn from 0 to 9 and a to f.

Key term: defanging. Reports write dangerous links so they cannot be clicked: **hxxp://bad[.]example**. Lines 2 and 3 undo that, called refanging, before matching.

Key term: regular expression. A compact pattern language for text. "\d{1,3}" reads "one to three digits".

Watch out: patterns have no judgement

The version number "10.2.1.4" matches the address pattern. A victim's address and the attacker's address look identical. A legitimate site named as "abused by the group" is not an indicator.

Patterns find the candidates. The next slides add the context that decides what each one means.

Who did what to whom: entities and relations

ONE SENTENCE FROM A REPORT • THE GROUP NAME IS INVENTED

"**Amber Kestrel** used **Mimikatz** to steal credentials, then exploited **CVE-2021-44228** against **retailers** in **Europe**."

Amber Kestrel: threat actor

Mimikatz: tool

CVE-2021-44228: vulnerability

retailers: sector

Europe: region

Key term: named entity recognition. A model that reads a sentence and labels each span with its type, as in the tags above. Language models fine-tuned on security text do this well.

Key term: relation extraction. A second step that finds the links: Amber Kestrel *uses* Mimikatz; Amber Kestrel *exploits* the vulnerability; Amber Kestrel *targets* retailers.



Why bother: entities and relations are nodes and edges. Do this to every report and you have a knowledge graph, in STIX form, that answers "which groups that target retailers exploit software we run?"

Mapping sentences to ATT&CK techniques

"The actor created a second access key for the compromised user."



T1098.001 · Additional Cloud Credentials

"Before continuing, they switched off the audit trail."



T1562.008 · Disable or Modify Cloud Logs

"Large instances were launched in several regions to mine cryptocurrency."



T1496 · Resource Hijacking

The task

Multi-label text classification: one sentence may describe several techniques, and there are hundreds to choose from.

Three ways to build it

TF-IDF with logistic regression as a baseline; a fine-tuned language model; or a large language model given the ATT&CK descriptions to choose from.

What you gain

Reports become countable. "Which techniques were used against our sector this year, and for which do we have no detector?" That gap list drives your roadmap.

MITRE publishes an open-source project for this task, the Threat Report ATT&CK Mapper, with labelled training sentences. A good place to start.

Embeddings: meaning as a position in space

Key term: embedding. A language model turns a piece of text into a list of a few hundred numbers. Texts with similar meaning get similar lists, even when they share no words.

What TF-IDF cannot do. "Stole the login details" and "harvested credentials" share no words, so TF-IDF sees no connection. Their embeddings sit side by side.

Worked example • cosine similarity

Two tiny embeddings: $A = (3, 4)$ and $B = (4, 3)$.

Multiply matching entries and add: $(3 \times 4) + (4 \times 3) = 24$.

Length of each: the square root of $9 + 16 = 5$.

Similarity = $24 \div (5 \times 5) = 0.96$

1 means pointing the same way; 0 means unrelated.

Four uses in a security team

"Have we seen this before?" Embed a new alert's description and retrieve the most similar past incidents and how they were resolved.

Deduplication. Five vendors describe one campaign in five styles. Their embeddings cluster; you read one summary instead of five reports.

Search by meaning. Ask "how do we contain a stolen access key?" and find the runbook even if it is titled "credential compromise procedure".

Features. The embedding of a command line or a user-agent string is a ready-made set of feature columns for Modules 7 and 9.

Large language models in the intelligence workflow

Summarise

A forty-page report becomes three lines for the board and a page for the analysts.

Extract to structure

Actor, techniques, indicators and targets, returned as fields a program can load.

Translate

Advisories and forum posts written in languages the team does not read.

Draft detections

Turn "they list all buckets, then read each" into a first-draft query for the SIEM.

Explain alerts

Turn a row of feature values into a sentence an analyst can act on.

Answer questions

"What do we know about attacks on object storage this quarter?", answered from your own library.

Key term: retrieval-augmented generation. Instead of trusting what the model remembers, first fetch the relevant documents, then ask it to answer from those and cite them.

The analyst's question



Search the library by embedding



Give the top passages to the model



Answer with sources

Four risks of language models, and the guardrails

Hallucination

The model writes a fluent, confident falsehood: an address that was never in the report, a vulnerability number that does not exist, an attribution nobody made.

Guardrail: require a quoted source for every claim. Check every extracted indicator against the original text by program.

Prompt injection

Text inside the material being read is taken as an instruction. Attackers control what appears in logs, emails and web pages, so they can write to your model.

Guardrail: treat everything ingested as untrusted data. Give the model no power to act without a person approving.

Data leakage

Logs and incident notes hold names, addresses and secrets. Pasted into an outside service, they have left your control.

Guardrail: a written policy on what may be sent where; remove secrets and personal data first; prefer deployments your contracts cover.

Over-trust

A well-written summary feels authoritative. People stop reading the source and stop doubting.

Guardrail: measure the model on a labelled test set, as with any other model in this tutorial, and keep an analyst accountable for what is published.

A language model is a fast, well-read junior colleague who is sometimes wrong and can be lied to. Use it that way and it is a real gain.

Checkpoint 10: language models for CTI

Answer on paper first. The answers are in this slide's speaker notes.

- 1 TF-IDF gives its highest scores to words that are...**
a) common in every document · b) frequent in this document but rare overall · c) the longest
- 2 Writing a link as `hxxp://bad[.]example` is called...**
a) hashing · b) defanging · c) encryption
- 3 Labelling "Mimikatz" as a tool and "retailers" as a sector in a sentence is...**
a) named entity recognition · b) one-hot encoding · c) change-point detection
- 4 Two texts whose embeddings have a cosine similarity of 0.97 are...**
a) unrelated · b) close in meaning · c) written in different languages
- 5 An attacker plants "ignore earlier instructions" in a log field your model will read. This is...**
a) hallucination · b) prompt injection · c) overfitting

MODULE 12

Predictive threat intelligence

From "what happened?" to "what is likely next, and what should we do first?"

12

YOU WILL LEARN

How to frame a prediction so that somebody can act on it

YOU WILL LEARN

Eight predictive models: exploited flaws, attack volume, the next move and more

YOU WILL LEARN

How to score a forecast honestly, and where prediction reaches its limits

Four kinds of analytics, climbing in value

DESCRIPTIVE

What happened?

Counts and dashboards. "We had 412 phishing emails last week."

DIAGNOSTIC

Why did it happen?

Investigation and root cause. "They all used one leaked supplier contact list."

PREDICTIVE

What is likely next?

Probabilities about the future. "There is a 60% chance of a larger wave before the sale next month."

Most of this module lives here.

PRESCRIPTIVE

What should we do?

Ranked actions. "Enforce second-factor login for the 30 staff on that list this week; it removes most of the risk for the least effort."

The point of predicting at all.

What can be predicted, and what cannot

Predictable, as probabilities

Which known vulnerabilities will be exploited soon

How much routine attack traffic to expect next week

Which alerts will turn out to be real incidents

Which newly registered domains will be used for harm

Which step an intruder is likely to take next

How quickly a threat indicator loses its value

Not predictable

The day and hour of a specific attack on you

A flaw nobody has discovered yet

The exact choices of one determined, well-funded adversary

A kind of attack with no precedent in any data

The pattern: the left column is about groups of similar events with a history. The right is about single events with none.

PLAIN ENGLISH

A weather forecast says "70% chance of rain tomorrow", never "a drop will strike this window at 3.07 p.m." It is still worth having: you take an umbrella. Predictive intelligence tells a security team where to carry the umbrella.

Framing a prediction: the five-line brief

PREDICTION BRIEF · WORKED FOR VULNERABILITIES	
1 · Unit	What is one row? One vulnerability on one of our systems.
2 · Target	What exactly is predicted? Whether attackers exploit it somewhere in the world.
3 · Horizon	By when? Within the next 30 days.
4 · Features	What is known today? Its description, its age, whether exploit code is public.
5 · Action	What will change? The order in which engineers apply patches.

If line 5 is blank, do not build the model.

A prediction nobody acts on is a curiosity with a running cost.

Line 3 matters more than it looks. "Will this ever be exploited?" cannot be scored until the end of time. "Within 30 days" can be checked next month.

Line 4 is the leakage test from slide 84, asked before any code is written: only what is known on the day of the prediction.

Which flaws will be exploited? CVSS, EPSS and KEV

Tens of thousands of vulnerabilities are published every year, each with an identifier called a **CVE**. No team can patch them all at once. Three public resources help decide the order.

CVSS

HOW BAD WOULD IT BE?

The Common Vulnerability Scoring System. A severity score from 0 to 10, from the flaw's technical properties.

It does not say how likely exploitation is. Thousands of flaws score 9 or above.

EPSS

HOW LIKELY IS IT?

The Exploit Prediction Scoring System, from the industry body FIRST. A probability from 0 to 1 that the flaw will be exploited in the next 30 days.

A machine learning model, retrained on fresh evidence and published daily, free.

KEV

IS IT HAPPENING ALREADY?

The Known Exploited Vulnerabilities catalogue, kept by the US agency CISA. A list of flaws confirmed as exploited in real attacks.

Not a prediction: a fact. Anything on it goes to the front of the queue.

Severity is not likelihood. The research behind EPSS found that only a small minority of published vulnerabilities are ever exploited in the wild. Patching by severity alone spends most of the effort on flaws no attacker will touch.

Inside an exploit-prediction model

Features, each known on the day of prediction

The description. Its words, as TF-IDF or an embedding.

How it is reached. Over the network? Without logging in?

The product. Which vendor; how widely deployed.

The weakness type. Memory error, injection, broken login.

Public exploit code. Is a working attack already posted?

Chatter. Mentions in security news and on social media.

Age. Days since it was published.

Severity. The CVSS score and its parts.

The two rust boxes usually carry the most signal: attackers are pragmatic and reuse what is already written.

Label: was exploitation observed within 30 days? Evidence comes from sensors, honeypots and incident reports.

Model: gradient-boosted trees suit this mix of text, categories and numbers. The published EPSS model uses them.

Evaluation: precision and recall, renamed. **Efficiency:** of the flaws we patched first, what share were exploited? **Coverage:** of the exploited flaws, what share had we patched first?

Watch out: "not seen exploited" is not "not exploited". Quiet, targeted use leaves no public trace, so labels undercount.

Ranking patches by risk instead of severity

Four unpatched flaws. Engineers can fix one a day. **Risk = probability of exploitation × impact on Meridian**, where impact is scored 1 to 10 by how much the affected system matters.

Flaw	Where it sits	CVSS	EPSS	Impact	Risk = EPSS × impact
A	Internal test server	9.8	0.02	2	$0.02 \times 2 = 0.04$
B	Internet-facing payment service	7.5	0.85	10	$0.85 \times 10 = 8.5$
C	Internal staff system	8.8	0.30	6	$0.30 \times 6 = 1.8$
D	Public web server	5.3	0.60	7	$0.60 \times 7 = 4.2$

Order by severity alone

A, C, B, D

The payment service waits until day three.

Order by risk

B, D, C, A

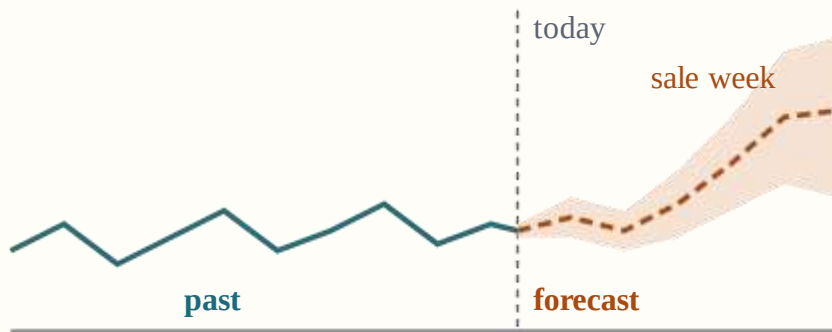
The "critical" 9.8 goes last, and rightly.

This is prescriptive analytics

A predicted probability, multiplied by local knowledge of what matters, turned into an ordered to-do list. Add one override: anything on the KEV list goes first regardless.

Forecasting attack volume

Failed login attempts per day at Meridian • illustrative



The method is Module 10's forecasting, pointed forward by weeks instead of hours: trend, weekly rhythm, plus known future events such as a sale, a holiday or a product launch.

Three decisions it feeds

Staffing: who is on call during the peak.

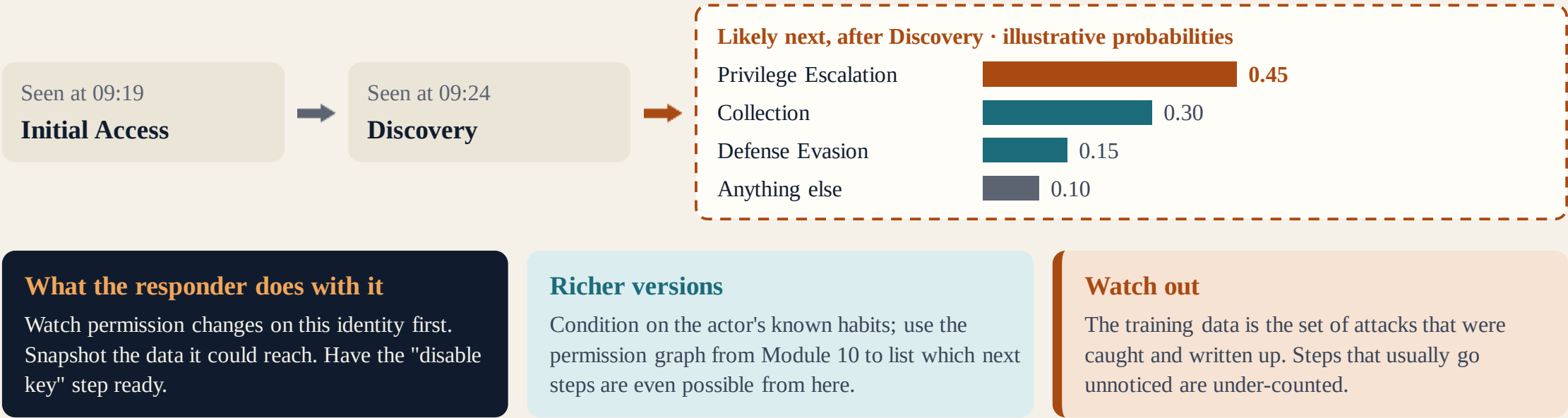
Capacity: rate limits and login protections sized for the surge.

Detection: a threshold that rises with the forecast, so the surge itself does not bury real alerts.

Notice the band widening. Honest forecasts grow less certain the further ahead they look. A forecast drawn as a single line is hiding something.

Predicting the attacker's next move

An incident is under way. Two techniques have been seen so far. Which tactic is likely next? A Markov chain over ATT&CK tactics, learned from past incidents and published reports, gives an answer.



Triage scoring: which alerts will prove real?

PREDICTION BRIEF

Unit: one alert, at the moment it is raised.

Target: will an analyst close it as a true incident?

Horizon: by the time it is closed.

Features: see right.

Action: the order of the queue; which alerts page someone; which are closed automatically.

Features that predict a real incident

How often this rule has been right before

The identity's current risk level

How many different detectors fired on it today

How valuable the asset is; its blast radius

A match with current threat intelligence

How the five most similar past alerts ended

Labels come free: every alert an analyst has ever closed is a training example. A gradient-boosted classifier on these features is usually enough.

Why it matters so much: it multiplies every other detector. The same four analysts spend their sixty investigations on the sixty alerts most likely to be real.

Watch out for the loop: alerts ranked low are never opened, so never labelled, so the model never learns it was wrong about them. Each day, open a small random sample from the bottom.

Spotting malicious infrastructure before it is used

Day 0 Attacker registers a domain	Day 0 to 3 Sets up hosting and a certificate	Day 3 Phishing emails go out	Day 4 Reported; added to blocklists
---	--	--	---

A blocklist acts on day 4, after the damage. A predictive model scores the domain on day 0. The days in between are the **lead time**. Timings are illustrative.

What can be known on day 0

- looks like a known brand, one letter off
- registered in a batch of hundreds
- the registrar's past record
- the hosting network's past record
- entropy and length of the name
- name servers shared with known bad domains
- a certificate issued within minutes

At Meridian: a daily job scores every new domain resembling "meridian-retail" and warns staff before the first email lands.

Watch out: blocking an innocent new business is a real harm. Use high scores to warn and watch, and reserve blocking for the very top.

Scoring domains at registration was shown at scale in the research system PREDATOR (Hao and colleagues, 2016).

Indicator decay: intelligence has a shelf life

The problem. An address used in an attack last month may belong to an innocent cloud customer today. A feed that never forgets fills your queue with false alarms.

The model. Let each indicator's score halve at a fixed interval, its **half-life**:

score now = starting score × 0.5 raised to (age ÷ half-life)

Addresses
days to weeks

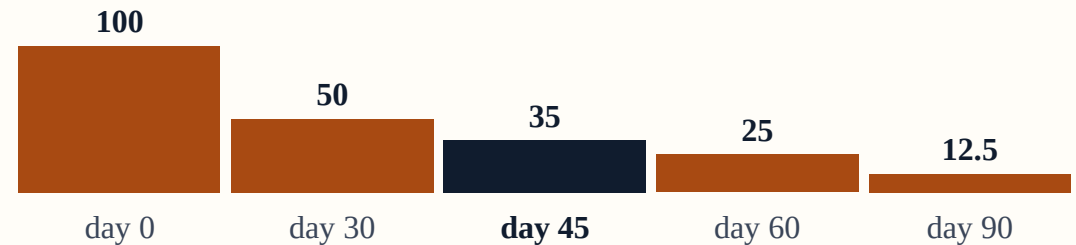
Domains
weeks to months

Behaviours
months to years

Typical half-lives, by kind. Estimate your own from how long past indicators kept producing true alerts.

WORKED EXAMPLE

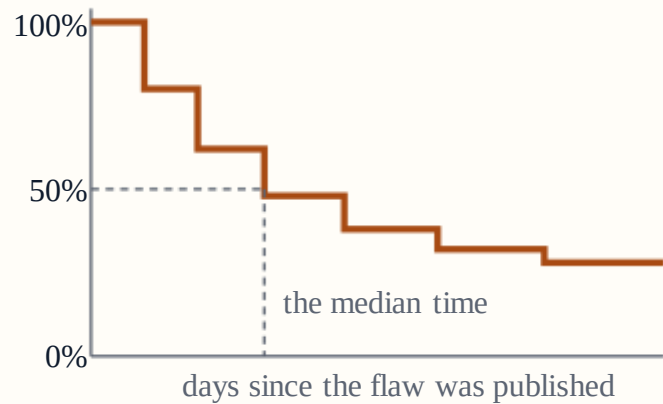
An address arrives with a score of 100. Addresses get a half-life of 30 days.



At day 45: $100 \times 0.5 \text{ raised to } (45 \div 30) = 100 \times 0.5 \text{ raised to } 1.5 = 100 \times 0.354 = 35.4$

Retire below 20: that happens after about 70 days. A fresh sighting resets the clock.

Survival analysis: not whether, but how soon



Key terms. Survival analysis models the time until an event. The **survival curve** shows the share for whom it has not happened yet. A case still waiting when the data ends is **censored**: you know only "not yet".

Why a classifier is not enough. "Exploited within 30 days, yes or no" throws away whether it took 2 days or 29. The curve keeps it, and handles the flaws published last week that simply have not had time.

Questions it answers

How short must our patch deadline be to beat half the attackers? How long does a leaked key last before first misuse? Do flaws with public exploit code fall faster?

The standard tools are the Kaplan-Meier curve and the Cox model; the Python library is called lifelines.

Bayesian updating: revising a belief as evidence arrives

The question at Meridian: is the extortion campaign now hitting retailers going to target us this month? Start with a belief, then let each piece of intelligence move it.

Starting belief

5%

the base rate for a firm like ours

Evidence 1: a peer retailer was hit this week

Seen 60% of the time when we are a target;
10% of the time when we are not.

$(0.05 \times 0.60) \div [(0.05 \times 0.60) + (0.95 \times 0.10)]$
 $= 0.030 \div (0.030 + 0.095)$

Belief now

24%

Evidence 2: their style of phishing email reaches our staff

Seen 70% of the time when we are a target;
5% when not.

$(0.24 \times 0.70) \div [(0.24 \times 0.70) + (0.76 \times 0.05)]$
 $= 0.168 \div (0.168 + 0.038)$

Belief now

82%

Key terms. The starting belief is the **prior**. The revised belief is the **posterior**. Yesterday's posterior is today's prior. The rule is the one from slide 47.

Why analysts like it: it turns "I have a bad feeling" into a number with a visible chain of reasoning. Disagree with the 60%? Change it and see what follows.

Watch out: the two pieces of evidence must be reasonably independent. Two reports of the same event are one piece of evidence, not two.

Simulating a year of risk, 100,000 times

Key term: Monte Carlo simulation. When a sum involves chance, let a computer roll the dice many thousands of times and study the spread of outcomes, not just the average.

Two inputs for Meridian's storage-breach risk

How often: 0.3 incidents a year on average.

How costly: typically £150,000, occasionally many times more.

Each simulated year draws a number of incidents and a cost for each. The result of 100,000 simulated years:

74%

of years: no incident, no loss

£74k

the average loss per year

£237k

exceeded in 1 year in 10

£423k

exceeded in 1 year in 20

£1.03m

exceeded in 1 year in 100

What the average hides: "£74k a year" sounds affordable. Three years in four cost nothing; the bad years are many times worse. Insurance and controls are bought for the right-hand boxes.

Where prediction enters: the "0.3 a year" need not be a guess. It can come from the models in this module, and fall measurably when a control is added. The FAIR framework formalises this approach.

Fusing intelligence into detection

Behaviour

What the identity just did, against its baseline. Modules 6 to 10.

Asset context

How valuable and how exposed the target is; its blast radius.

Threat intelligence

What attackers are doing in the world right now. This module.



ONE MODEL

All three as feature columns of the same row.



ONE SCORE

Risk that reflects what happened, to what, in which climate.

Intelligence as features: five examples

1. The source address's decayed indicator score (slide 173).
2. The reputation of the network the address belongs to.
3. Whether the client software matches a known attack tool.
4. The highest EPSS among the flaws on the target system.
5. Whether the sequence of calls matches a technique reported this month against our sector.

The effect. The same odd login scores higher when it comes from infrastructure seen in a live campaign and lands on a system with an actively exploited flaw.

Watch out: join each event to the intelligence as it stood at that moment. Using today's blocklist on last month's events is time leakage.

Scoring a forecaster: calibration and the Brier score

Key term: calibration. Of all the times the model said "70%", did the thing happen about 70% of the time? If so, its probabilities can be taken at face value.

Why it matters here. Slide 168 multiplied a probability by an impact. If "0.85" really means 0.40, every ranking built on it is wrong. Detection needs good ordering; prediction needs honest numbers.

Two more things to track. Lead time: how far ahead of the event the warning came. **Uptake:** whether anyone acted on it.

WORKED EXAMPLE · BRIER SCORE

Brier score = the average of (forecast – outcome)², where outcome is 1 if it happened and 0 if not. Lower is better; 0 is perfect.

Flaw	Forecast	Exploited?	Squared gap
B	0.9	yes, 1	0.1 ² = 0.01
C	0.2	no, 0	0.2 ² = 0.04
D	0.6	no, 0	0.6 ² = 0.36
A	0.1	no, 0	0.1 ² = 0.01

Brier = (0.01 + 0.04 + 0.36 + 0.01) ÷ 4 = **0.105**

A forecaster who always says 0.5 scores 0.25. One confident miss, flaw D, cost more than the other three together.

Five limits of prediction in security

The opponent reads the forecast

Weather does not change because it was predicted. Attackers do. A published model shifts what it measures.

The ground moves

Tools, targets and your own systems change. Patterns learned last year fade. Models need retraining and expiry dates.

The worst events are the rarest

The breach that ends a company has few precedents to learn from. Models are best where stakes are routine.

Success erases its evidence

You patch the flaw the model flagged. It is never exploited. Was the model right, or was that a false alarm? You cannot tell.

We learn from what was caught

Training data is the set of attacks someone noticed and reported. The stealthiest work is missing from it.

Prediction tells you where to look first. It never tells you where it is safe not to look.

Use it to order the work. Keep the basics, such as patching, least privilege and logging, for everything, including what the model ranked last.

Checkpoint 11: predictive intelligence

Answer on paper first. The answers are in this slide's speaker notes.

- 1** **CVSS tells you...**
a) how likely a flaw is to be exploited · b) how severe it would be if exploited · c) how old it is
- 2** **A flaw has EPSS 0.5 and sits on a system of impact 8. Its risk score is...**
a) 4 · b) 8.5 · c) 40
- 3** **An indicator starts at 80 with a half-life of 10 days. After 20 days it scores...**
a) 40 · b) 20 · c) 10
- 4** **A forecaster says "90%" on ten occasions and is right on nine. The forecaster is...**
a) overconfident · b) well calibrated · c) leaking data
- 5** **Which line of the prediction brief, if blank, means "do not build it"?**
a) Horizon · b) Features · c) Action

MODULE 13

Cloud use cases

Eight detections, each assembled from the parts built in the last twelve modules.

13

YOU WILL LEARN

A one-page recipe for each of the eight most common cloud threats

YOU WILL LEARN

For each: the data, the features, the model and the trap

YOU WILL LEARN

How to map what you have built on to ATT&CK and see the gaps

The catalogue: eight use cases at a glance

Use case	Main data	Features that carry it	Method
1 · Stolen credentials	Audit and sign-in logs	Novelty, travel speed, first-time calls	Baselines, then a classifier
2 · Privilege escalation	Audit logs, permission graph	Rarity of the call, path to admin	Rarity scores and graphs
3 · Data theft from storage	Storage access logs, flows	Volume against baseline, new buckets	Robust z-score, CUSUM
4 · Hijacked compute	Audit logs, billing	New region, new server type, cost	Forecast, then compare
5 · Misconfiguration risk	Configuration inventory	Exposure, sensitivity, path length	Risk ranking, graphs
6 · Container compromise	Cluster audit, runtime events	Processes never seen in this image	Allow-profiles, sequences
7 · Covert channels	DNS and flow logs	Entropy of names, regular timing	Classifier, beacon score
8 · Insider misuse	All of the above, HR context	Slow drift, distance from peers	Peer groups, change points

Read the last column again. There is no new mathematics in this module. Every method was taught earlier; use cases are a matter of pointing them at the right data.

Where to begin: use case 1. Most cloud intrusions pass through a stolen identity, and use cases 2, 3 and 4 are usually what the thief does next.

Stolen credentials in use

THE THREAT

A password, access key or session token is used by someone who is not its owner. Every action is "authorised", so no permission rule objects.

ATT&CK: Valid Accounts: Cloud Accounts, T1078.004

The tell: the right identity, behaving like a different person.

Data

Sign-in logs, control-plane audit logs, and an inventory saying which identities are people and which are machines.

Model

Per-identity baselines and an Isolation Forest to begin with. Add a boosted classifier once analysts have confirmed enough cases. Sum into one risk level per identity.

Features

New address, country, device or client tool. Travel speed. First-time API calls. A burst of List calls. Share of calls denied. Hour against own rhythm.

Watch out

VPNs, business travel, new starters and on-call emergencies all look like this. Machine identities need baselines of their own.

WHEN IT FIRES

End the identity's sessions and rotate its keys. Require a fresh login with a second factor. Then review everything it did since the first odd signal.

Privilege escalation

THE THREAT

An identity gives itself, or an accomplice, more power than it was granted: attaching an admin policy, minting keys for another user, handing a strong role to a server it controls.

ATT&CK: Account Manipulation, T1098

The tell: a rare, powerful call by someone who never makes it.

Data

Audit logs, narrowed to the few dozen calls that change permissions, and the permission graph from Module 10.

Model

Rarity scoring for the dangerous calls, a graph check on each change, and a Markov chain for the pattern "discover, then escalate".

Features

Rarity of the call in bits. First time for this identity. Blast radius after the change minus before. Was there a discovery burst just earlier?

Watch out

Platform teams and deployment pipelines change permissions all day. Baseline them apart, and check that each of their changes matches an approved code commit.

WHEN IT FIRES

Undo the permission change at once; it costs little if the change was innocent. This detector can reach high precision, so treat a hit as real until shown otherwise.

Data theft from cloud storage

THE THREAT

Data is copied out of buckets or databases in bulk, or a bucket or backup is quietly shared with an outside account.

ATT&CK: Data from Cloud Storage, T1530; Transfer Data to Cloud Account, T1537

The tell: reading far more, or far more widely, than this identity ever has.

Data

Storage data-access logs, network flow logs, and audit events for changes to sharing and bucket policies.

Features

Bytes and objects read against own baseline. Distinct buckets touched. Buckets never touched before. Ratio of listing to reading. A new destination.

Model

Robust z-scores per identity and per bucket for the fast grab. CUSUM for the slow leak. A plain rule for "made public" or "shared outside the company".

Watch out

Data-access logging is often switched off to save money. No log, no detection. Backups, analytics jobs and migrations are the usual false alarms.

WHEN IT FIRES

Block the identity and remove any outside sharing. Then list exactly which objects were read: that list decides who must be told, and the access log is the only source for it.

Hijacked compute

THE THREAT

An intruder starts servers of their own in your account, usually large ones, to mine cryptocurrency or run other heavy workloads. You receive the bill.

ATT&CK: Resource Hijacking, T1496

The tell: new kinds of machine, in new places, working flat out.

Data

Audit events for starting servers, hourly billing data, processor-usage metrics, and DNS or flow logs for where the new machines connect.

Model

Novelty flags, plus forecast-then-compare on server counts and on spend. A threat-intelligence match on known mining destinations settles it.

Features

Region never used. Server type never used. Servers started per hour against forecast. Spend per hour against forecast. Processor pinned near 100%.

Watch out

Load tests, machine-learning projects and sale-day scaling look the same. Billing data can lag by hours, so do not rely on cost alone.

WHEN IT FIRES

Stop the servers, keeping a copy of one for investigation. Then find the credential that started them: this use case is nearly always the visible end of use case 1.

Misconfiguration risk: ranking the open doors

THE THREAT

Not an attack but an invitation: a public bucket, an admin port open to the world, a role with far too much power, logging switched off.

Key term: tools that scan for these are called cloud security posture management, CSPM.

The problem: they return thousands of findings, all marked "high".

Data

The configuration of every resource, its change history, tags saying how sensitive it is, and the permission graph.

Features

Reachable from the internet? Sensitivity of what it holds. Hops from the internet to it. Blast radius behind it. Highest EPSS on the same system.

Model

This is prediction, not detection. Score each finding as likelihood times impact, as on slide 168, and rank the attack paths the findings open.

Watch out

Some exposure is intended: a website's images must be public. Record the intent as a tag, or the model will nag about it for ever.

WHAT TO DO WITH IT

Give engineers the ten findings that sit on the most attack paths, not the ten thousand. Then stop the same mistake recurring with an automatic check at deployment time.

Container and Kubernetes compromise

THE THREAT

A flaw in an application lets an attacker run commands inside its container. From there they steal its credentials, query the cluster and try to break out to the host.

ATT&CK: Deploy Container, T1610; Escape to Host, T1611

Key term: a container is a sealed package holding one application and what it needs. Kubernetes runs fleets of them.

Data

Kubernetes audit logs for requests to the cluster.
Runtime sensors for processes, files and connections inside each container.

Features

A process this image has never run. A command shell opened. A connection to a new destination. Secrets listed. A container created with host-level powers.

Model

An allow-profile per image: the short list of processes and calls it normally makes. Anything off the list is anomalous. Sequence models for finer cases.

Watch out

Containers live for minutes. Baseline by image and deployment, never by the individual container, or every one is "new".



The easiest anomaly problem in this tutorial. A container image is built to do one job the same way every time. "Normal" is tiny and stable, so departures stand out sharply. Response is cheap too: destroy the container and start a clean one.

Covert channels: DNS tunnelling and beacons

THE THREAT

Malware inside a server needs to receive orders and send out what it steals. It hides both in traffic every network allows: name lookups and ordinary web requests.

ATT&CK: Application Layer Protocol: DNS, T1071.004

Key term: DNS tunnelling packs data into the names being looked up, such as a long scrambled word in front of the attacker's domain.

Data

DNS query logs and network flow logs, grouped by the domain looked up and by each pair of server and destination.

Features

Per domain: number of distinct sub-names, their average length and entropy. Per pair: regularity of timing, sameness of sizes, how few other servers contact it.

Model

A random forest on the per-domain features; the beacon score from slide 144; the generated-domain classifier from slide 108.

Watch out

Content-delivery networks, antivirus products and monitoring agents also use long encoded names and regular check-ins. Keep an allow-list.

WHEN IT FIRES

Block the domain at the resolver, isolate the server, and work backwards to how it was infected. Share the domain with your peers: it is fresh, first-hand intelligence.

Insider misuse: the most delicate use case

THE THREAT

A person with legitimate access uses it for the wrong purpose: copying data before leaving, sabotage after a grievance, or simple carelessness.

Why it is hard: there is no break-in to detect. Every action is within the person's permissions, from their usual device.

Data

The access logs of the earlier use cases, with a small amount of context from HR: role, team, and whether a notice period has begun.

Features

Slow drift from the person's own baseline over weeks. Distance from peers in the same role. Reach into data outside the role. Copies to personal destinations.

Model

Peer-group comparison and change-point detection, summed into a slowly fading risk level. The signal is weak; expect modest precision.

Watch out

A false alarm here accuses a colleague. Most "anomalies" are people doing their jobs slightly differently.



Rules before models. Agree the purpose with legal and HR first. Collect the minimum. Tell staff that monitoring exists. Never act automatically: a small, authorised group reviews each case discreetly. Module 15 returns to this.

The coverage map: what can we see, and what can we not?

ATT&CK tactic	Detector Meridian has built	Coverage
Initial Access	Credential-use anomalies	Good
Persistence	New-key and new-user rarity	Partial
Privilege Escalation	Permission-change rarity and graph	Good
Defense Evasion	One rule for logging changes	Partial
Credential Access	Nothing yet	None
Discovery	List-call burst detector	Good
Lateral Movement	New edges in the activity graph	Partial
Collection and Exfiltration	Storage volume baselines, CUSUM	Good
Command and Control	DNS and beacon scoring	Partial
Impact	Hijacked-compute forecast	Good

How to build one. List the techniques relevant to your cloud. For each, ask: which detector would fire, and has it ever been tested against that technique?

The rust row is next quarter's project. A map like this plans a roadmap better than any accuracy figure.

"Good" must be earned. Have a red team perform the technique and see whether the alert arrives. Untested coverage is a hope.

MODULE 14

Models in production

A model in a notebook protects nobody. This module is about the day after it works.

14

YOU WILL LEARN

How a model is wired into the data pipeline and the analysts' queue

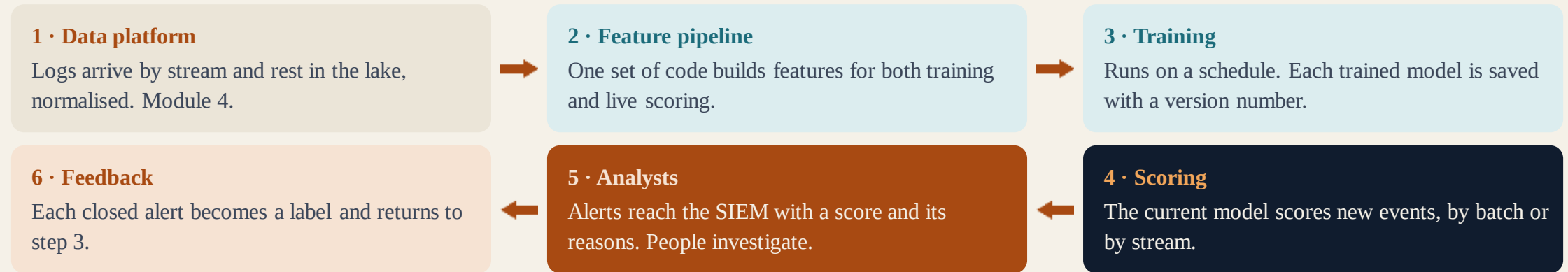
YOU WILL LEARN

How models decay, how to measure it, and when to retrain

YOU WILL LEARN

How to explain a score, automate safely and keep alert volume humane

From notebook to production: the loop



Read the top row left to right, drop from box 3 to box 4, then the bottom row right to left.

Key term: MLOps. The engineering practice of running this loop reliably: automated, versioned, monitored and repeatable.

The model is the small box. In a working system the algorithm is a few hundred lines. The other five boxes are most of the code and most of the failures.

Batch or streaming: how fast must the answer be?

Batch

Score everything once an hour or once a night.

For: simple, cheap, easy to rerun and debug.

Against: an attack can be an hour old before it is scored.

Suits: patch ranking, misconfiguration risk, slow insider drift.

Micro-batch

Score the last few minutes, every few minutes.

For: most of the speed of streaming with the simplicity of batch.

Against: still a delay of one interval.

Suits: most identity and storage detections. The sensible default.

Streaming

Score each event within seconds of its arrival.

For: the fastest possible detection.

Against: windows must be kept in memory; harder to build, test and pay for.

Suits: blocking a login as it happens; stopping a transfer in progress.

The rule: the scoring delay must be shorter than the time the attacker needs to do damage. Anything faster than that is cost without benefit.

At Meridian: the breach took 46 minutes from first use to hijacked servers. A nightly batch misses it entirely. A five-minute micro-batch raises the alert by 09:46 at the latest.

The silent failure: training-serving skew

KEY TERM

Training-serving skew: a feature is computed one way when the model is trained and a slightly different way when it scores live events. Nothing crashes. The model simply sees numbers it was never taught, and its scores quietly lose meaning.

Four ways it creeps in

Time zones. Training used UTC; the live job uses local time. "Hour of day" is now off by several hours.

Windows. Training counted whole clock hours; live scoring counts the last 60 minutes.

Blanks. Training filled a missing value with the median; live code fills it with zero.

Lateness. Training saw every event; live scoring runs before late logs arrive, so counts are low.

The cure: define each feature once

A **feature store** holds one definition per feature and serves it to both training and live scoring, with values recorded as they stood at each moment in time.

The cheap version

Log the feature values the live model actually received, and train the next model on those logged values, not on a fresh recomputation.

The test

Recompute yesterday's features offline and compare them with what was served. They should match to the last digit.

Drift: the world moves and the model does not

Data drift

The inputs change. Meridian opens a new region; a new team joins; a service is re-architected. Features take values the model rarely saw.

Concept drift

The meaning changes. Attackers adopt a new technique, so the pattern that used to say "malicious" no longer does, and a new one goes unseen.

WORKED EXAMPLE · POPULATION STABILITY INDEX

Sort the feature "calls per hour" into three bands and compare shares.

Band	Training	This week	(now – then) × ln(now ÷ then)
Low	0.50	0.30	$-0.20 \times \ln(0.60) = -0.20 \times -0.511 = 0.102$
Medium	0.30	0.30	$0 \times \ln(1) = 0$
High	0.20	0.40	$0.20 \times \ln(2.0) = 0.20 \times 0.693 = 0.139$

$PSI = 0.102 + 0 + 0.139 = 0.24$

Reading the index

under 0.1
stable

0.1 to 0.25
look into it

over 0.25
act

A widely used rule of thumb. At 0.24, find out what changed before the model's scores are trusted further.

Monitor three things weekly: each important feature's PSI, the distribution of scores, and the share of alerts analysts close as real.

Retraining, and promoting a new model safely

Four triggers for retraining

The calendar. Weekly or monthly, on a sliding window of recent data.

A drift alarm. The index on the previous slide crosses its limit.

A drop in results. Fewer alerts are being closed as real.

A new technique. Intelligence reports something the model has never seen; add examples and retrain.

Make feedback one click. When an analyst closes an alert, a single choice of "real", "benign" or "unsure", with a reason, is the label for the next training run.

Promotion in three stages

1 • Offline test

The new model must beat the current one on the most recent weeks.



2 • Shadow mode

It scores live events silently beside the current model. Nobody sees its alerts yet.



3 • Switch

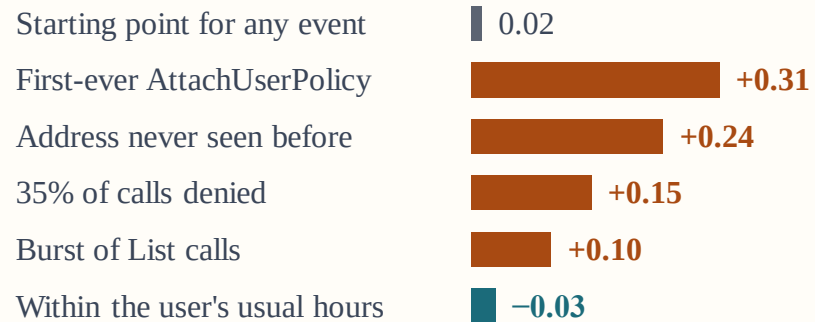
It takes over. The old model is kept, ready to be restored in one step.

Key terms. The model in service is the **champion**; the candidate is the **challenger**. A **model registry** stores every version with its training data, settings and test results.

Watch out: never retrain blindly on whatever arrived. An attacker who can shape your data can teach your model. Module 15 explains.

Explaining a score: why did the model say that?

How one alert's score of 0.79 was built up • illustrative



$0.02 + 0.31 + 0.24 + 0.15 + 0.10 - 0.03 = \mathbf{0.79}$. Rust bars push the score up; the teal bar pulls it down.

Key term: SHAP values. A method that splits one prediction into a contribution from each feature, so that the contributions add up exactly to the score. It works with any model.

What the analyst reads

"Risk 0.79. This key made a permission change it has never made, from a new address, after a burst of failed and listing calls."

Why it is not optional. An unexplained score cannot be investigated, argued with or trusted. Explanations also expose leaks: if "ticket number" tops the list, you have found slide 84.

From score to action: alerts and automated response

A WELL-FORMED ALERT CARRIES

Who and what: the identity, the resource, the time.

How sure: the score, and the identity's running risk.

Why: the top three reasons, in a sentence.

Which technique: the ATT&CK tactic it suggests.

Evidence: a link to the raw events behind it.

Next step: the runbook to follow.

Three tiers of response, by how reversible the action is

Tier 1 • Automatic, harmless if wrong

Ask for a second factor again. Open a ticket. Tag the resource. Notify the owner.

Tier 2 • Automatic at very high confidence, and undoable

Suspend one access key for an hour pending review. Block one address. An analyst is told at once and can reverse it.

Tier 3 • A person decides

Disable a member of staff. Stop production servers. Delete anything. No model takes these on its own.

The principle: automate what is cheap to undo; keep people on what is not. A false alarm that asks for a second login costs ten seconds. A false alarm that shuts the shop on sale day costs the trust the whole programme depends on.

Engineering against alert fatigue

KEY TERM

Alert fatigue is what happens when the volume of alerts numbs the people reading them. Real ones are closed unread among the false. It is the commonest way a good detector fails.

Deduplicate

The same identity, for the same reason, within the same hour, is one alert, not forty.

Group into cases

Gather every signal about one identity into a single case with a timeline, as on slide 137.

Rank

Order the queue with the triage model from slide 171, so the top is worth opening.

Suppress, with an expiry date

Silence a known harmless pattern for 30 days, never for ever. Permanent exceptions are where intruders hide.

Hold to a budget

Fix the number of alerts the team can work each day, as on slide 120, and set thresholds to fit it.

Retire the losers

A detector whose alerts are never real after tuning is taking attention from those that are. Switch it off.

Watch three numbers every week: alerts per analyst per day; the share closed as real; and the time an alert waits before someone opens it. If the third is rising, the first is too high.

Detection as code: test detectors like software

Key term: detection as code. Every rule, feature and model lives in version control, is reviewed by a second person, and is tested automatically before it reaches production.

Three layers of test

Feature tests. Feed in six hand-made events; check that "distinct regions" comes out as exactly 2.

Replay tests. Keep a library of recorded attacks from red-team exercises and simulations. After every change, each must still raise its alert.

Volume tests. Replay a normal week. The change must not double the number of alerts.

DETECTOR RECORD · ONE PER DETECTOR

Name: first-time permission change

Purpose: catch privilege escalation, T1098

Data: control-plane audit log

Owner: a named person, not a team

Known false alarms: platform team migrations

Last tested against a real technique: a date

Runbook: what the analyst does when it fires

A detector with no owner, no test and no runbook is a liability with a dashboard. The record on the right takes ten minutes to write and saves hours on the night it matters.

Checkpoint 12: use cases and production

Answer on paper first. The answers are in this slide's speaker notes.

- 1 Why are containers an unusually good fit for anomaly detection?**
a) They are large · b) Each image does one job the same way, so "normal" is small and stable · c) They are never attacked
- 2 A feature computed one way in training and another way in production is...**
a) concept drift · b) training-serving skew · c) label leakage
- 3 Attackers change technique, so yesterday's malicious pattern no longer applies. This is...**
a) data drift · b) concept drift · c) skew
- 4 Running a new model silently beside the current one before switching is called...**
a) shadow mode · b) grid search · c) oversampling
- 5 Which response should never be left to a model alone?**
a) Opening a ticket · b) Asking for a second factor · c) Deleting production servers

MODULE 15

Attacks on the models, and governance

A detector is software an adversary wants to defeat, and a system that passes judgement on people. Both facts demand care.

15

YOU WILL LEARN

Four ways attackers go after the model itself, and the defences

YOU WILL LEARN

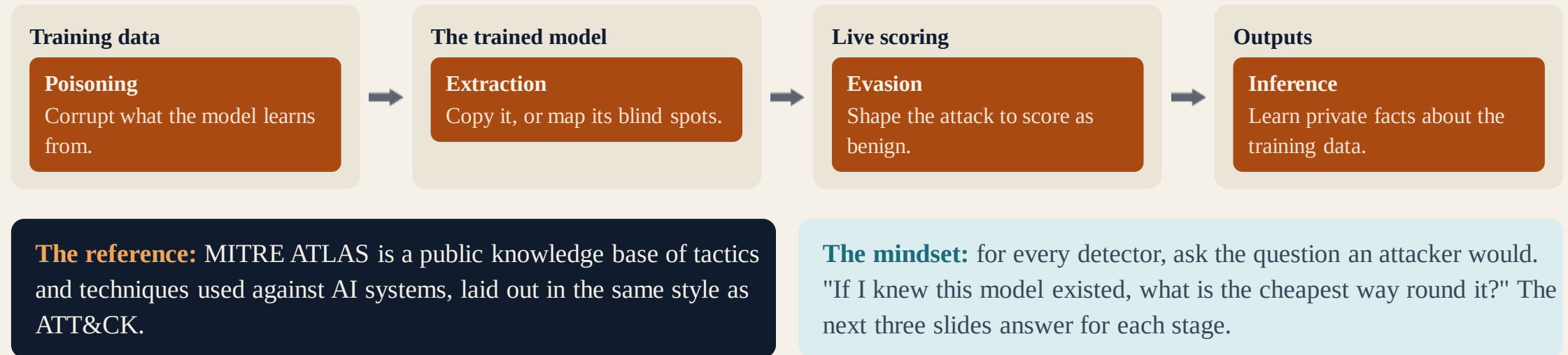
How to protect the machine learning pipeline like any other critical system

YOU WILL LEARN

The privacy, fairness and documentation duties that come with monitoring people

The model is an attack surface too

Most machine learning faces an indifferent world: tomorrow's weather does not try to fool the forecaster. Security models face an opponent who studies them. That study has a name: **adversarial machine learning**.



Evasion: looking innocent on purpose

How attackers evade, by detector

Malware classifiers. Pad the file with harmless text and code until the statistics look benign.

Phishing detectors. Swap letters for look-alikes, or put the text in an image.

Volume baselines. Go "low and slow": take a little each day, under the threshold.

Novelty features. Route through the victim's own city and work in the victim's own hours.

Beacon detectors. Add random delay so the rhythm blurs.

Five defences

Use features that are costly to fake. A file's padding is free to change. A month of behaviour and a position in the permission graph are not.

Layer different detectors. Evading volume, novelty, sequence and graph models all at once is far harder than evading one.

Accumulate. CUSUM and running risk catch what stays under every single threshold.

Keep scores private. An attacker who can see their score can tune against it.

Attack yourself first. Have a red team try to slip past each detector, and train on what gets through.

The consolation: every evasion has a price. Going slow gives defenders time. Mimicking the victim limits what the attacker can do. A model that merely forces attackers to be slower and quieter has already paid for itself.

Poisoning: teaching the model the wrong lesson

The boiling frog

An intruder already inside raises their activity by a few percent a day. The baseline adapts each night. After a month, large transfers are "normal".

Hits: any detector that relearns from recent data.

Bad labels

False reports are fed into shared intelligence feeds, or a compromised account marks real alerts as benign. The classifier learns the attacker's version of the truth.

Hits: supervised models and triage scoring.

The back door

A downloaded model or dataset has been tampered with, so that anything carrying a secret marker is always scored benign.

Hits: anyone reusing models or data from outside.

Four defences

Do not learn from what you flagged. Keep anomalous periods out of the baseline, as on slide 143.

Limit how fast a baseline may move. Compare with last quarter as well as last week.

Know where data and models came from. Verify checksums; prefer sources you can audit.

Keep a canary set. A fixed library of known attacks that every retrained model must still catch.

Why the canary set matters most

Poisoning is silent. Scores look healthy; the alert count even falls. The only way to notice is a test whose right answer you already know and the attacker cannot touch.

Stealing the model, and reading its memory

Extraction

Send the model thousands of probes and record its answers. From those, build a copy, or simply chart where it says "benign".

Risk: the attacker rehearses against the copy until the attack passes, then uses it on you.

Defence: limit and log queries to any scoring service. Return a decision, not a precise score.

Inference

Models can leak what they were trained on. An attacker may learn whether a given person's records were in the training data, or recover fragments of it.

Risk: a detector trained on staff behaviour becomes a source of information about staff.

Defence: train on the minimum; restrict who can query; keep raw identities out of features.

Instruction hijack

For language models: text planted in a log, email or web page is read as a command. Covered on slide 160 as prompt injection.

Risk: the model summarises what the attacker wants, or takes an action on their behalf.

Defence: treat all ingested text as data; no unsupervised actions.



Where to read further: MITRE ATLAS catalogues these techniques with real case studies. NIST publishes a taxonomy of adversarial machine learning, and OWASP maintains top-ten risk lists for machine learning and for language-model applications.

Securing the machine learning pipeline itself

The pipeline that builds your detectors is a production system holding sensitive data. Apply to it everything Module 1 said about clouds.



Least privilege. Few people can change training data, labels or the model store, and every change is logged.



Signed models. Record a checksum of each model at training. Refuse to load one that does not match.



Distrust model files. Python's common "pickle" format can run code when opened. Never load one from an unknown source.



Supply chain. Pin and scan the libraries, base images and pre-trained models the pipeline depends on.



Watch the watcher. Log who queries the scoring service and how often. A burst of probing is itself an alert.



Repeatable training. Same data and settings must give the same model, so that tampering shows up as a difference.

Add one line to the incident plan: "If we suspect the model or its data has been tampered with, we restore the last trusted version and fall back to rules while we investigate."

Privacy and fairness: security monitoring watches people

Privacy: six working rules

Purpose. Write down what the monitoring is for, and use it for nothing else.

Minimum. Collect the fields the features need, not everything available.

Proportion. Match the intrusiveness to the risk being managed.

Openness. Staff are told that monitoring exists and why.

Masking. Score on coded identifiers; reveal the name only when an alert is investigated.

Limits. Set retention periods, and log who looks at whose profile.

Fairness

A model that learned "normal" from the majority will flag the minority more: night-shift staff, people working from other countries, new joiners, part-timers. None of that is wrongdoing.

Check: compare alert rates across such groups. A large gap with no security reason is a defect to fix, not a finding.

The law, in outline. Under UK and EU data protection law, large-scale monitoring of staff normally calls for a data protection impact assessment, and people have rights concerning decisions made about them solely by automated means. Involve your data protection officer before building, not after.

This slide is orientation, not legal advice. Rules differ by country and change; take advice for your own situation.

Governance: who answers for the model?

MODEL CARD · ONE PAGE PER MODEL

Purpose: what it detects, and what it must not be used for

Owner: a named, accountable person

Data: sources, period covered, how labels were made

Performance: precision and recall at the live threshold

Known weaknesses: what it misses and falsely flags

Fairness check: alert rates by group, with date

Upkeep: retraining schedule, drift limits, how to roll back

Key term: model card. A short, standard document describing a model for those who rely on it. If nobody can fill in a line, that is a finding.

Three frameworks worth knowing by name

NIST AI Risk Management Framework. Four functions: govern, map, measure, manage. Voluntary and practical.

ISO/IEC 42001. An international standard for managing AI systems, auditable like the security standard ISO 27001.

The EU AI Act. Law that sorts AI uses by risk and sets duties accordingly. Check how it classes your use.

The test of good governance: when the model makes a serious mistake, can you say within the hour who owns it, what it was trained on, and how to switch it off?

MODULE 16

Capstone project

Detecting compromised access keys at Meridian, from the first meeting to the final report.

16

YOU WILL SEE

Every module of this tutorial used once, in order, on one problem

YOU WILL SEE

The dead ends as well as the result: a leak, a weak baseline, an awkward miss

A REMINDER

Meridian is fictional. Every figure in this module is illustrative, chosen to be realistic

Frame the problem

PREDICTION BRIEF · AGREED WITH THE SECURITY LEAD	
Unit	One identity in one hour: an "identity-hour".
Target	Was the identity under someone else's control in that hour?
Horizon	Now. This is detection, scored every five minutes.
Features	Built from audit and sign-in logs up to that moment only.
Action	An alert; the analyst suspends the key and reviews its activity.

Three targets, written down before any modelling

75%
recall, at least

20
alerts a day, at most

15 min
to detect, at most

Where the numbers came from. Twenty is a third of the team's daily capacity of sixty. Fifteen minutes is well inside the 46 minutes the last intruder needed to reach the servers.

Why first: without agreed targets, "is the model good?" has no answer and the project never ends.

Gather the data and the labels

90

days of audit and sign-in logs

3.85m

events after removing duplicates

180,000

identity-hours with any activity

150

of them malicious: 1 in 1,200

Where the 150 labels came from

Three real incidents, each reconstructed hour by hour from the investigation notes.

Two red-team exercises, with the testers' own timed record of what they did.

Every other identity-hour is treated as benign, knowing a few probably are not.

The split, by time

Train · days 1 to 60 120,000 rows, 100 malicious	Validate 30,000; 20	Test 30,000; 30
--	---	---------------------------------------

Each incident sits whole inside one part. The test part holds four separate intrusions the model will never see in training.

Be honest about 30. With 30 malicious rows in the test set, each one caught or missed moves recall by more than 3 points. Report results as rough, and keep collecting labels.

Explore: four findings that changed the plan

1 • People and machines are two populations

Six service accounts make about 70% of all calls. One baseline for everyone would be dominated by them.

Decision: an identity-type feature, and peer groups from clustering.

2 • A leak, caught early

A column called "ticket_id" was filled on exactly the malicious rows. It was written by the investigators afterwards.

Decision: deleted, along with two similar columns. A model using it would have scored 100% and learned nothing.

3 • The 150 malicious hours belong to 9 identities

A random split would put the same stolen identity in training and test.

Decision: confirms the split by time, with each incident kept whole.

4 • Novelty separates the classes best

An address new to the identity appears in 82% of malicious hours and 3% of benign ones.

Decision: invest most feature effort in the novelty family.



Two days of looking, no model trained. Finding 2 alone saved the project: the leak would have surfaced only after deployment, as a detector that never fired.

Engineer the features: eighteen columns, five families

Volume • 3 log of calls this hour · calls against own 30-day median · calls against peer-group median

Ratios • 3 share of calls denied · share that are List or Describe · share that change something

Diversity • 3 distinct API calls · distinct services · distinct regions

Novelty • 5 new address · new country · new client tool · count of first-time API calls · rarity in bits of the rarest call

Time and place • 2 share of own past activity in this hour of day · travel speed since last login

Context • 2 identity type, person or machine · blast radius from the permission graph

The discipline: every feature is computed as of the end of its hour, from data stamped earlier. The "30-day median" for day 40 uses days 10 to 39 and nothing later.

Built in two rounds. The first ten came from the brief. The last eight, mostly novelty and graph, came from reading the first model's mistakes in step 8.

Two baselines before anything clever

BASELINE A · A RULE

"Alert on any new country"

What many teams already run. One line, no training.

0.03

average precision on the validation set. Fires constantly on travel and VPNs; misses intruders who use a nearby exit.

BASELINE B · NO LABELS

Isolation Forest

The eight lines from slide 132, on the first ten features.

0.12

Four times the rule. Catches the loud hours: discovery bursts and server hijacks. Misses quiet use of a stolen key.

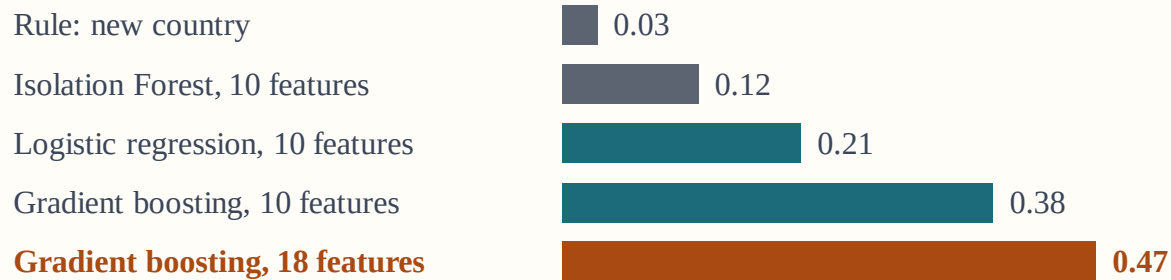
Why bother with baselines? They set the bar. Every later model must justify its extra complexity by beating 0.12. And a random guess here scores about 0.0008, so both baselines are already far above chance.

An unplanned win. Reading the forest's top 50 unlabelled anomalies turned up a contractor's key being shared by three people. Not an attack; still a risk, now fixed.

Average precision, from slide 118, is the yardstick throughout the capstone: it suits rare events and needs no threshold yet.

Train supervised models and compare

Average precision on the validation set · illustrative



Each supervised model used class weights for the 1-in-1,200 imbalance and four-fold time-series cross-validation on the training days.

Where the gains came from

Labels: 0.12 to 0.21. Even a simple model, once told what an attack looks like, beats a detector that only knows "strange".

Algorithm: 0.21 to 0.38. Trees learn combinations: a new address matters far more when a first-time call follows it.

Features: 0.38 to 0.47. Eight more columns, no change of algorithm. Tuning the settings added less than 0.01.

Kept for deployment: the last model, with logistic regression retained as an explainable fallback.

Set the threshold, then open the test set once

Threshold chosen on validation data to fit the budget of 20 alerts a day. Then, on the 15 test days, 30,000 identity-hours:

	Alerted	Not alerted
Malicious (30)	24 caught	6 missed
Benign (29,970)	216 false alarms	29,754 correctly quiet

Recall · target 75%
 $24 \div 30 = 80\%$

Alerts a day · limit 20
 $240 \div 15 = 16$

Precision
 $24 \div 240 = 10\%$

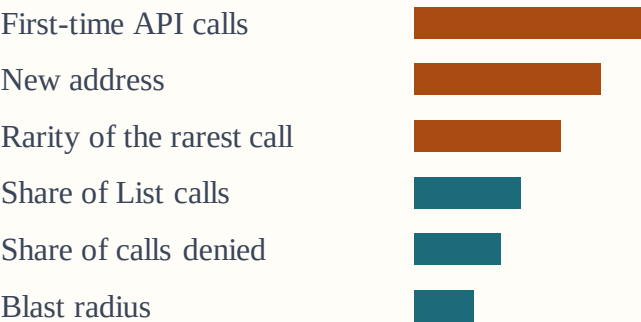
False positive rate
0.72%

As the analysts will see it. Grouped by identity, 240 alert-hours become 96 cases, about 6 a day. All four intrusions in the test period raised a case, at a median of 9 minutes after the intruder's first action.

Read the 10% honestly: nine alerts in ten are false. The targets are met only because the volume is small enough to work.

Explain the model and read its mistakes

What drives the scores overall



Bar length: average size of each feature's SHAP contribution. The three rust bars are all novelty features. Nothing suspicious, such as an identifier, appears: no sign of a leak.

Reading the 216 false alarms

About 130 came from two platform engineers during a planned migration to a new region. Everything they did was new.

Fix: add "an approved change ticket is open for this identity" as a feature.

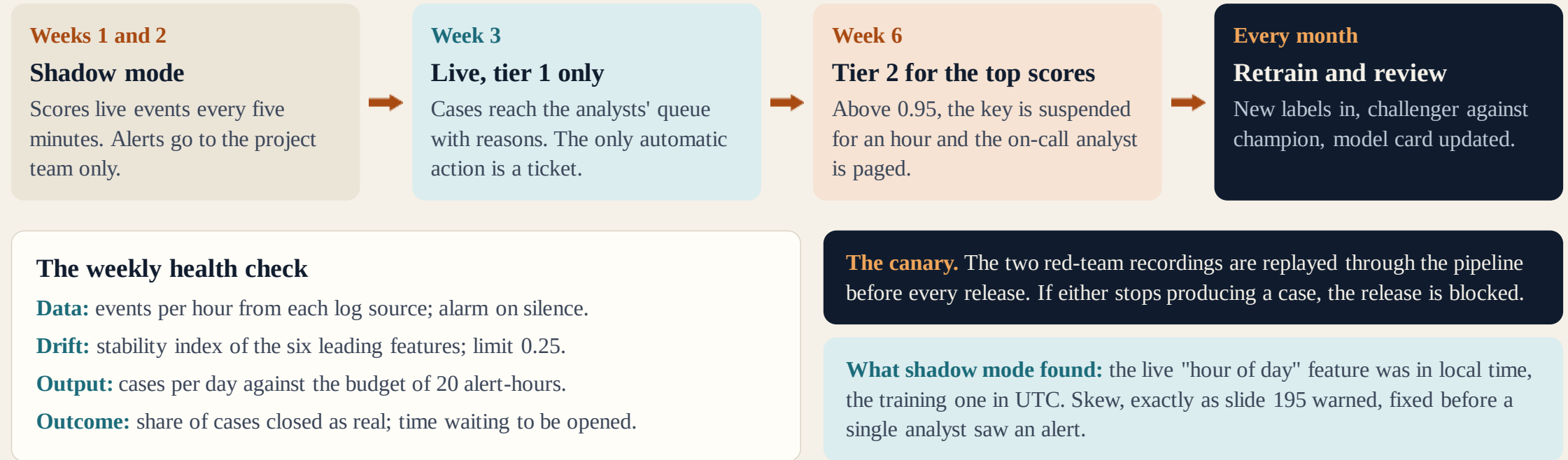
Reading the 6 misses

All six were one intruder, in later hours, using the key from the owner's own city in office hours to read storage slowly. Nothing was novel.

Fix: none inside this model. It needs the storage CUSUM detector of use case 3 as a second layer.

This step is where the next version is designed. The errors named one new feature and one whole new detector. No metric would have told us either.

Deploy, and keep watching



Report: one page for the security lead

COMPROMISED-KEY DETECTOR · SUMMARY

What it does. Scores every identity every five minutes for signs that someone else is using it.

What it catches. In a 15-day test it flagged all four intrusions, typically 9 minutes after the intruder began, and 80% of the hours in which they were active.

What it costs. About six cases a day, roughly an hour and a half of analyst time. Nine alerts in ten are false alarms.

What it misses. A patient intruder who copies the owner's location and hours. A storage detector is needed behind it.

What we ask. Approval to build that second detector, and a red-team exercise each quarter to keep both tested.

Why the report is written this way

No algorithm is named. The reader decides about risk and budget, not about trees.

Numbers are in the reader's units: intrusions, minutes, cases, analyst hours.

The weakness is stated plainly, next to the request that addresses it. Trust comes from the fourth line.

Against the brief: recall 80% against 75%. Alerts 16 a day against 20. Detection in 9 minutes against 15. All three targets met.

CLOSING REFERENCE

Ten mistakes this tutorial was written to prevent

- 1 Choosing an algorithm before a decision.** Fill in the five-line brief first. Slide 165.
- 2 Reporting accuracy.** With rare attacks it measures nothing. Use precision and recall. Slide 112.
- 3 Splitting at random.** Train on the past, test on the future, keep incidents whole. Slide 44.
- 4 Leaking the answer.** A near-perfect score means look for the leak. Slide 84.
- 5 One "normal" for everyone.** People and machines need separate baselines. Slide 67.
- 6 Ignoring the base rate.** A 1% false positive rate is thousands of false alarms a day. Slide 116.
- 7 Treating unusual as malicious.** Add context and analyst feedback. Slide 138.
- 8 Training once.** The world drifts; measure it and retrain. Slide 196.
- 9 Shipping a bare score.** No reasons, no investigation, no trust. Slide 198.
- 10 Forgetting the model has adversaries and subjects.** Defend it and govern it. Module 15.

Notice what is absent: "used the wrong algorithm". Projects in this field rarely fail on mathematics. They fail on framing, data, evaluation and upkeep.

CLOSING REFERENCE

Method cheat sheet: what to try first

If you need to...	Try first	Slide
Classify, with plenty of labels	Gradient-boosted trees	94
Classify, and explain every weight	Logistic regression	89
Find outliers, with no labels at all	Isolation Forest	131
Compare an identity with its own past	Robust z-score on its history	127
Catch a small rise that persists	CUSUM	145
Spot clockwork connections	Coefficient of variation of the gaps	144
Judge the order of a session's actions	Markov chain	146
Find risky routes to sensitive data	Shortest paths on the permission graph	149
Turn reports into features	TF-IDF, then embeddings	154
Decide which flaw to patch next	EPSS × impact, with KEV first	168

"First" is the important word. Each row names the simplest method that usually works. Move to something heavier only when you can show, on your own data, what the simple one misses.

CLOSING REFERENCE

Formula cheat sheet: fourteen sums worth knowing

Precision = true positives $\div$ all alerts

Recall = true positives $\div$ all real attacks

F1 = $2 \times \text{precision} \times \text{recall} \div (\text{precision} + \text{recall})$

False positive rate = false positives $\div$ all benign events

z-score = (value – mean) $\div$ standard deviation

Robust z = $0.6745 \times (\text{value} - \text{median}) \div \text{MAD}$

Upper fence = third quartile + $1.5 \times$ interquartile range

Coefficient of variation = standard deviation $\div$ mean

Gini impurity = $1 - (\text{share benign})^2 - (\text{share malicious})^2$

Entropy = sum over characters of $p \times \log_2(1 \div p)$

EWMA = $\alpha \times \text{new value} + (1 - \alpha) \times \text{old baseline}$

Logistic score = $1 \div (1 + e \text{ to the power } -\text{total})$

Risk = probability of exploitation $\times$ impact

Indicator score = start $\times 0.5$ to the power (age $\div$ half-life)

Rust: judging a detector. Teal: judging one value. Navy text: inside the models. Navy boxes: prediction. Each was worked by hand somewhere in this tutorial; redo one a day until they are reflexes.

CLOSING REFERENCE

Glossary, part 1: A to F

Anomaly detection. Finding events that do not fit a learned picture of normal behaviour.

ATT&CK. MITRE's public catalogue of the tactics and techniques attackers use.

Baseline. A learned description of what is normal for one identity or system.

Beacon. A compromised machine's regular check-in with the server that controls it.

Blast radius. Everything an attacker could reach if one identity were stolen.

Calibration. Whether predicted probabilities match how often things actually happen.

Class imbalance. One class, here benign, vastly outnumbering the other.

Control plane. The API actions that create, change or delete cloud resources.

CTI. Cyber threat intelligence: analysed knowledge about adversaries that supports a decision.

Drift. Change over time in the data a model sees, or in what that data means.

Entity. The thing whose behaviour is profiled: a user, a key, a server, a bucket.

EPSS. A published probability that a vulnerability will be exploited within 30 days.

False positive. A benign event that the model flagged as malicious: a false alarm.

Feature. A measurable property of a sample; one column of the training table.

Glossary, part 2: I to W

Indicator of compromise. An address, domain or file hash showing a known threat was present.

Label. The known answer attached to a training example, such as "malicious".

Leakage. Information in the features that would not be available at prediction time.

Novelty feature. A flag that something is new for this identity: an address, a call, a region.

Overfitting. Memorising the training data so well that new data is handled badly.

Precision. Of the alerts raised, the share that were real attacks.

Recall. Of the real attacks, the share that raised an alert.

Residual. The gap between what a forecast expected and what actually happened.

SIEM. The central system that stores security logs and raises alerts.

Supervised learning. Learning from examples that carry the right answer.

Threshold. The score at or above which the system raises an alert.

TTPs. Tactics, techniques and procedures: how an adversary operates.

UEBA. User and entity behaviour analytics: profiling each identity and scoring departures.

Window. The span of time over which events are summarised into one row.

CLOSING REFERENCE

A thirty-day practice plan

WEEK 1

Data

Install Python and Jupyter. Load one public dataset from slide 60. Answer the five exploration questions. Write your own version of the table on slide 71.

By Friday: a notebook that loads, cleans and charts the data.

WEEK 2

Features and classifiers

Build ten features from at least four families. Split by time. Train logistic regression, then a random forest. Draw the precision-recall curve.

By Friday: a confusion matrix at a threshold you can defend.

WEEK 3

Anomalies and time

Hide the labels. Run an Isolation Forest. Compute a robust z-score per entity. Apply EWMA and CUSUM to one series. Score one host for beaconing.

By Friday: your top 20 anomalies, each read and explained.

WEEK 4

Prediction and upkeep

Rank twenty real vulnerabilities by EPSS $\times$ impact. Write a prediction brief. Explain one score with SHAP. Write a model card and one replay test.

By Friday: a one-page report in the style of slide 221.



Then do it again on your own organisation's logs, with permission. The second pass, on data you understand and care about, is where the learning sticks.

TO TAKE AWAY

Seven sentences to keep

- 1 In the cloud, identity is the perimeter: **model identities, not machines.**
- 2 The right features beat the cleverest algorithm, and **novelty is the strongest feature.**
- 3 Attacks are rare, so **judge by precision, recall and the alert budget**, never by accuracy.
- 4 **Split by time and hunt for leaks**; a result too good to be true is not true.
- 5 **Unusual is not malicious**: add context, combine signals, and keep analysts in the loop.
- 6 **Predict in order to prioritise**: probability times impact decides what to fix first.
- 7 A model is a system: **monitor it, explain it, defend it and document it.**

Start small: one log source, one identity baseline, one honest evaluation.